



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Caracterización de jugadores a partir de datos de ligas fantasy

Realizado por
Pablo Arrabal Hermoso

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
José Antonio Troyano Jiménez

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de junio, curso 2025/26

*A mis padres, a mi hermano y a mi novia, por acompañarme y apoyarme en cada
paso de este camino*

Agradecimientos

Quiero agradecer a todos los profesores que me han enseñado a lo largo de la carrera, no solo por los conocimientos técnicos, sino también por todas las experiencias y aprendizajes que me llevo de esta etapa.

A mis amigos de toda la vida, por seguir estando ahí pase lo que pase, las risas y hacerme desconectar cuando más lo necesitaba. También a los compañeros que conocí en primero y que acabaron convirtiéndose en amigos y una parte fundamental de esta etapa. Ahora miro con cariño por todas las horas en clase, las bibliotecas, los trabajos de última hora, los agobios antes de los exámenes y todos los momentos que nos llevamos de estos 4 años.

A mis padres, por darme la oportunidad de estudiar fuera de casa y confiar en mí incluso cuando yo mismo dudaba, y por haberme dado la educación y los valores que me han acompañado durante toda mi vida. Gracias por enseñarme a esforzarme, a ser constante y a no rendirme nunca.

A mi hermano, por enseñarme que incluso después de los golpes más duros siempre hay que volver a levantarse y seguir adelante. Y a mi novia, por estar siempre ahí. Por aguantar los momentos difíciles, los agobios, las dudas y acompañarme en todo este camino.

En definitiva, gracias a todas las personas que formáis parte de mi vida y que, de una forma u otra, habéis empujado para que hoy esté aquí.

Resumen

El presente Trabajo de Fin de Grado describe el diseño e implementación de LigaMaster, una plataforma web orientada al análisis predictivo y la toma de decisiones en el Fantasy Football. En un entorno caracterizado por la alta volatilidad de los datos y la opacidad algorítmica de las herramientas actuales, este proyecto propone un sistema de apoyo al usuario basado en la Inteligencia Artificial Explicable (XAI).

Para la construcción de la plataforma se ha adoptado una arquitectura desacoplada, utilizando Django y Django REST Framework para el backend, y React para la interfaz de usuario. El pipeline de datos procesa información heterogénea extraída mediante web scraping de diversas fuentes deportivas, superando complejos retos técnicos como la normalización y el matching difuso de entidades.

El núcleo predictivo compite entre algoritmos de Machine Learning (Random Forest, XGBoost, Ridge y ElasticNet) entrenados y especializados por posición táctica (porteros, defensas, centrocampistas y delanteros) para pronosticar los puntos de los jugadores. Los resultados empíricos revelan que los modelos lineales con regularización generalizan mejor en este dominio, cumpliendo satisfactoriamente con los exigentes objetivos de Error Absoluto Medio (MAE) y del coeficiente de correlación de Spearman.

Como principal valor añadido frente a soluciones comerciales, el sistema integra la metodología SHAP para proporcionar predicciones interpretables. Este enfoque rompe la "caja negra" de los modelos tradicionales, permitiendo al mánager comprender qué factores estadísticos y de contexto influyen realmente en las recomendaciones. En conclusión, el proyecto culmina en una herramienta completa, robusta y transparente que eleva la gestión de plantillas virtuales a un nivel analítico avanzado.

Abstract

This Bachelor's Thesis describes the design and implementation of LigaMaster, a web platform oriented towards predictive analysis and decision-making in Fantasy Football. In an environment characterized by high data volatility and the algorithmic opacity of current tools, this project proposes a user support system based on Explainable Artificial Intelligence (XAI).

To build the platform, a decoupled architecture was adopted, using Django and Django REST Framework for the backend, and React for the user interface. The data pipeline processes heterogeneous information extracted through web scraping from various sports sources, overcoming complex technical challenges such as data normalization and fuzzy entity matching.

The predictive core compares Machine Learning algorithms (Random Forest, XGBoost, Ridge, and ElasticNet) trained and specialized by tactical position (goalkeepers, defenders, midfielders, and forwards) to forecast player points. Empirical results reveal that regularized linear models generalize better in this domain, satisfactorily meeting the demanding targets for Mean Absolute Error (MAE) and the Spearman correlation coefficient.

As its main added value compared to commercial solutions, the system integrates the SHAP methodology to provide interpretable predictions. This approach breaks the "black box" of traditional models, allowing the manager to understand which statistical and contextual factors actually influence the recommendations. In conclusion, the project culminates in a comprehensive, robust, and transparent tool that elevates the management of virtual squads to an advanced analytical level.

Índice general

I Introducción y conceptos

1. Introducción	1
1.1 Definición y contexto	1
1.2 Marco del juego y relevancia económica	1
1.3 Justificación del proyecto	7
2. Objetivos	8
2.1 Objetivo general	8
2.2 Objetivos funcionales	8
2.3 Objetivos técnicos	10
3. Análisis similares	12
3.1 AnalíticaFantasy.com	12
3.2 JornadaPerfecta.com	15
3.3 Repositorios	18
3.4 Conclusiones y valor añadido	19

II Metodología y planificación

4. Metodología	22
4.1 Evaluación de Scrum	22
4.2 Metodología seleccionada: Kanban	23
4.3 Planificación Temporal	24
4.4 EDT (Estructura Desglose de Trabajo)	25
4.5 Planificación de riesgos	28
4.6 Gestión de la Calidad	32
4.7 Análisis de tiempos, costes y desviaciones	32
5. Stack tecnológico	37
5.1 Visión general de la arquitectura del sistema	37
5.2 Elección del lenguaje de programación: Python	39
5.3 Backend: Django 5.1.4 + Django REST Framework	39
5.4 Persistencia y base de datos: SQLite y PostgreSQL	40
5.5 Procesamiento de datos y Machine Learning	42
5.6 Obtención de datos y scraping	44
5.7 <i>Frontend</i> con <i>React</i> y <i>Vite</i>	45

5.8	<i>APIs</i> y otras herramientas de soporte	45
5.9	Entorno de desarrollo, despliegue y calidad de software	46
6.	Identificación de requisitos	49
6.1	Alcance detallado	49
6.2	Fuera de alcance	49
6.3	Suposiciones, dependencias y restricciones	50
6.4	Actores	50
6.5	Requisitos funcionales	51
6.6	Requisitos no funcionales	60
6.7	Casos de uso	61
6.8	Prototipado de UI	64
 III Diseño y ejecución		
7.	Diseño del Sistema	68
7.1	Arquitectura global del sistema	68
7.2	Modelado predictivo	74
7.3	Decisiones de diseño estratégicas	78
7.4	Modelado de datos	80
7.5	Infraestructura de datos y búsqueda	84
7.6	Diseño de la interfaz	85
7.7	Diseño de seguridad y autenticación	88
7.8	Diseño del despliegue	88
8.	Recolección y procesamiento de datos	91
8.1	Fuentes de datos y proceso de scraping	91
8.2	Matching de entidades entre fuentes	93
8.3	Limpieza y saneamiento del dataset	94
8.4	Ingeniería de características	95
8.5	Prevención del data leakage	99
8.6	Estructura del dataset final	99
9.	Implementación y desarrollo	101
9.1	Implementación del backend con Django	101
9.2	Implementación del scraping de datos	114
9.3	Implementación del matching de entidades	119
9.4	Implementación del sistema de Machine Learning	124
9.5	Implementación del feature engineering	128
9.6	Implementación de IA Explicable (XAI)	131

9.7 Implementación del frontend	134
9.8 Principales problemas enfrentados	137
IV Resultados y conclusiones	
10. Calidad del software y pruebas	142
10.1 Organización del entorno de pruebas	142
10.2 Niveles de pruebas y resultados	143
10.3 Pruebas de carga y rendimiento	145
10.4 Resultados de cobertura y calidad	147
11. Resultados	150
12. Conclusiones	155
12.1. Cumplimiento de objetivos	155
12.2. Limitaciones encontradas	155
12.3. Líneas de trabajo futuro	156
12.4. Valoración personal	157
Bibliografía	159
V Anexos	
Anexo I. Glosario de términos	160
Anexo II. Repositorios consultados	162
Anexo III. Registros de Clockify	163
Anexo IV. Manuales de instalación y usuario	164
Anexo V. Autodeclaración del uso de IA	178

Índice de cuadros

1.	Asignación de puntos por valoración del cronista (Marca)	3
2.	Puntos adicionales por gol según posición del jugador	4
3.	Distribución de desempeño esperado por posición.	78
4.	Objetivos de rendimiento para modelos de ML por posición.	78
5.	Paleta de colores empleada en la interfaz.	87
6.	Bloques de variables del dataset final.	100
7.	Suite de pruebas automatizadas	142
8.	Ejecución focalizada de validación de la API	144
9.	Resultados agregados de las pruebas de estrés con Locust	146
10.	Cobertura de código por módulo	148
11.	Umbrales de rendimiento por posición táctica.	150
12.	Resultados de los modelos evaluados para la posición Defensas (DF). .	150
13.	Resultados de los modelos evaluados para la posición Delanteros (DT). .	151
14.	Resultados de los modelos evaluados para la posición Mediocentros (MC). .	151
15.	Resultados de los modelos evaluados para la posición Porteros (PT). . .	151
16.	Cumplimiento de los objetivos de rendimiento por posición.	152
17.	Resumen de la validación final por posición.	154

Índice de figuras

1.	Figura de referencia de la dificultad de un partido en AnalíticaFantasy.com.	13
2.	Figura de referencia de la vista de la plantilla de un equipo en JornadaPerfecta.com.	16
3.	Figura de referencia del apartado de noticias.	17
4.	Salida de la API de predicción mencionada en el análisis.	18
5.	Diagrama de Gantt de la línea base del cronograma.	25
6.	Estructura de Desglose de Trabajo (EDT) del proyecto.	25
7.	Matriz de riesgos del proyecto.	29
8.	Diagrama de resumen del stack tecnológico.	38
9.	Estructura del sistema y comunicación entre sus capas.	48
10.	Casos de uso para Usuario Anónimo.	61
11.	CCasos de uso para Usuario Registrado.	62
12.	Flujo de usuario en el caso de uso 4.	64
13.	Prototipo del menú principal de la aplicación, con algunas funcionalidades y donde el usuario puede navegar entre las distintas secciones desde la barra lateral.	65
14.	Prototipo de clasificación de LaLiga, mostrando equipos, posiciones y puntos.	65
15.	Prototipo de la ficha de equipo, con plantilla, posiciones, próximo partido y acceso a detalles individuales de los jugadores.	66
16.	Prototipo de la vista de jugador, mostrando estadísticas históricas y rendimiento reciente.	66
17.	Prototipo de la vista de plantilla de usuario, con predicción de puntos y explicación XAI.	67
18.	Resumen de alto nivel de la arquitectura.	68
19.	Distribución de puntos por posición comparada con distribución normal.	77
20.	Modelo conceptual del sistema.	82
21.	Diagrama Entidad-Relación.	83
22.	Esquema lógico del sistema.	84
23.	Diagrama de navegación de UI.	87
24.	Diagrama de despliegue del sistema en local.	89
25.	Diagrama de despliegue del sistema en fase de producción.	89
26.	Captura real de la organización del módulo de api/.	106
27.	Radar chart de Pedri González.	137

28.	Uso de percentiles y roles en la interfaz en la ficha de Pedri González. .	137
29.	Estado del <i>dashboard</i> de SonarQube tras el análisis del proyecto. . . .	149

Listings

1.	Algoritmo de reintentos con backoff exponencial	92
2.	Extracción de datos ocultos en comentarios HTML	92
3.	Función para calcular posición más frecuente de un jugador	102
4.	Validación de registro con verificación de contraseñas en Django REST Framework	104
5.	Serializer anidado con campos dinámicos usando SerializerMethodField	104
6.	Resolución ambigua de relaciones sociales en serializers mediante context	105
7.	Cálculo dinámico de percentiles relativos por posición	107
8.	Reconstrucción de eventos del partido mediante cruce relacional	107
9.	Construcción de objeto de análisis comparativo entre equipos rivales . .	108
10.	Generación de ranking top 3	109
11.	Codificador JSON personalizado	109
12.	Estrategia de predicción híbrida	109
13.	Detección de eventos en vivo con sistema de claves únicas para evitar redundancias	110
14.	Sistema de alertas para jugadores con bajo minutaje	110
15.	Signal de Django para emitir notificaciones en tiempo real vía WebSockets	112
16.	Inicialización asíncrona en hilo durante startup de Django	113
17.	Algoritmo de reintentos para descargas robustas	115
18.	Reintento con espera aleatoria para mitigación de detección de bots . .	116
19.	Extracción de nodos comentario HTML con BeautifulSoup para datos ocultos	118
20.	Estructura JSON de roles y rankings de eficiencia para enriquecimiento de features	118
21.	Normalización de texto y limpieza de minutos para entity matching . .	120
22.	Búsqueda contextual de candidatos por partido usando fuzzy matching acotado	121
23.	Fallback a WRatio para matching robusto	122
24.	Comparación de distancia Levenshtein vs WRatio	122
25.	Partial Ratio: búsqueda de subcadenas	122
26.	Token Sort Ratio: ordenación de tokens	123
27.	Clase base abstracta para trainers de modelos ML con split temporal .	124
28.	Espacio de búsqueda exhaustivo de hiperparámetros para grid search multi-algoritmo	125
29.	Agregación temporal con medias móviles y EWMA para capturar evolución de jugadores	129

30.	Inyección de interacciones entre elite y rol para enriquecer representación de características	131
31.	Diccionario semántico para traducir variables a narrativas deportivas interpretables	132
32.	Contrato de respuesta API con metadatos de predicción y explicación para transparencia	133

1. Introducción

1.1 Definición y contexto

El fútbol fantasy se define como un juego de simulación en el que los participantes asumen el rol de entrenadores de un equipo. Para componer su plantilla virtual, seleccionan jugadores activos de equipos reales que compiten en ligas auténticas cada fin de semana. El éxito de los participantes se determina mediante un proceso de conversión de la estadística al juego: el desempeño real de estos jugadores en el campo de juego se transforma en una puntuación numérica que se acumula semanalmente. El objetivo es conseguir la máxima puntuación posible, imponiéndose a sus rivales.

Los orígenes del concepto se remontan a 1962, cuando se establecieron las bases y reglas que rigen el actual fantasy football. Inicialmente, este concepto estuvo ligado al golf en Estados Unidos, popularizándose masivamente después con el béisbol y posteriormente con el fútbol americano. La idea del fútbol fantasy europeo, el *Fantacalcio*, fue concebida e introducida en Italia en 1990. No obstante, el fenómeno llegó a España cuatro años después, a través de la “Liga Fantasy Marca”.

Es fundamental contextualizar la evolución tecnológica: en sus primeras etapas en España, hasta el año 2002, las alineaciones y los resultados se gestionaban de manera manual, utilizando el correo ordinario y el fax. Esta limitación logística histórica fue el principal obstáculo para su crecimiento. El verdadero despegue y la popularidad masiva del fantasy a nivel mundial se deben a la aparición de plataformas en línea gratuitas y la era de internet, lo que garantizó la inmediatez de los resultados y permitió la gestión de la información en tiempo real, abriendo el camino para el análisis de datos a gran escala.

1.2 Marco del juego y relevancia económica

El presente trabajo se centra en la mecánica de juego de la plataforma líder en España, **LaLIGA FANTASY**, que opera principalmente bajo un modelo de liga privada con mercado abierto. La estrategia de gestión se divide en dos fases: la construcción inicial del equipo y la gestión semanal de las transacciones.

1.2.1 Construcción inicial y gestión de la plantilla

El juego comienza con la asignación de un presupuesto virtual idéntico a todos los participantes. Los jugadores son tratados como activos digitales con valor de mercado. Existen dos métodos principales para formar la plantilla inicial:

- **Equipo aleatorio:** los managers reciben un equipo predefinido y aleatorio, debiendo vender y comprar jugadores para adecuar su plantilla y presupuesto.
- **Presupuesto completo:** los managers reciben el presupuesto completo en efectivo y comienzan a construir su equipo desde cero a través del mercado.

La gestión posterior del equipo se realiza mediante el mercado de fichajes, donde la clave es la anticipación del rendimiento para comprar barato y vender caro.

1.2.2 La gestión económica

La estrategia del juego reside en el mercado de fichajes donde los managers deben operar constantemente en dos frentes de adquisición: jugadores libres y jugadores de otros equipos.

Cada futbolista dispone de un valor de mercado establecido por la plataforma, que actúa como referencia inicial para su adquisición. En el caso de los jugadores libres, el sistema emplea un mecanismo de pujas ciegas, en el que los managers compiten ofreciendo una cantidad igual o superior al valor de mercado. La oferta más alta resulta ganadora y fija el precio final de compra, introduciendo un componente estratégico y competitivo de carácter económico.

El valor de mercado de los jugadores es altamente volátil y puede variar diariamente en función de múltiples factores que reflejan la percepción de riesgo y oportunidad por parte de la comunidad de usuarios. Entre los más relevantes se encuentran el rendimiento reciente, especialmente el desempeño en los últimos encuentros; el estado físico, donde cualquier información relacionada con lesiones, sanciones o periodos de baja provoca ajustes inmediatos del precio; y los factores extradeportivos, como decisiones técnicas del entrenador, rotaciones, rumores de traspaso o incidentes ajenos al terreno de juego. Esta volatilidad convierte la gestión del mercado en un proceso dinámico que exige un seguimiento constante.

Además del mercado de jugadores libres, la economía del juego incorpora mecanismos avanzados de protección y transferencia de activos. Cada jugador tiene asociada una cláusula de rescisión, que puede ser incrementada estratégicamente por su manager

mediante el pago de una cantidad adicional de capital virtual. Esta inversión busca disuadir a los rivales de adquirir jugadores clave, pero conlleva un riesgo económico: si el jugador no mantiene un rendimiento acorde a su valor, el capital invertido en la subida de la cláusula no se recupera.

El mecanismo más agresivo de adquisición es el denominado “clausulazo”, que consiste en abonar la cláusula de rescisión (original o incrementada) de un jugador perteneciente a otro equipo. Esta operación no puede ser rechazada y provoca la transferencia inmediata del futbolista, lo que introduce un alto nivel de presión estratégica y refuerza la necesidad de realizar predicciones precisas sobre rendimiento futuro y evolución del valor de mercado.

En conjunto, la gestión económica del fantasy de fútbol se configura como un entorno complejo y dinámico, donde la toma de decisiones informadas resulta clave para maximizar el rendimiento deportivo y financiero del equipo virtual.

1.2.3 Sistema de puntuación

La predicción del rendimiento depende críticamente de entender el sistema de puntuación, ya que la variable objetivo a pronosticar está definida por las reglas específicas de la plataforma. Existen principalmente dos categorías de sistemas de valoración, y la elección del método por parte del mánager impacta directamente en la estrategia de juego y la complejidad del análisis.

A. Sistemas de puntuación subjetivos este es el sistema más tradicional, utilizado como opción principal por la plataforma líder **LALIGA FANTASY** (ligado a las puntuaciones de Marca). Las puntuaciones se asignan basándose en una calificación subjetiva otorgada por un cronista o periodista tras el final de cada encuentro.

Un ejemplo de este modelo es la puntuación con estrellas del diario Marca:

Valoración del cronista	Puntos
4 estrellas	14 puntos
3 estrellas	10 puntos
2 estrellas	6 puntos
1 estrella	2 puntos
0 estrellas	-2 puntos
SC (Sin Calificar)	0 puntos

Cuadro 1: Asignación de puntos por valoración del cronista (Marca)

Además de la valoración base, las acciones clave impactan directamente la puntuación final. Estas acciones pueden sumar o restar puntos en función de su naturaleza. Entre las más comunes se encuentran: goles (a favor o en propia puerta), asistencias, penaltis (marcados, fallados o parados), tarjetas (amarillas o rojas) y portería a cero.

B. Sistemas de puntuación objetivos sistemas como el algoritmo SofaScore calculan la puntuación en tiempo real a partir de acciones cuantificables y estadísticas en vivo. La evaluación del rendimiento comienza con una puntuación base de 6.5 y se ajusta dinámicamente: las acciones positivas del jugador (pases exitosos, recuperaciones, entradas completadas) elevan su nota, mientras que las negativas (pérdidas de balón, pases fallados) la reducen.

El resultado final del algoritmo es una nota pura de 1 a 10 que refleja el rendimiento global del jugador en el partido. Sin embargo, la transformación de esta nota en Puntos Fantasy finales es responsabilidad de la plataforma de juego y no del algoritmo externo. La plataforma toma esta nota de 1 a 10 como puntuación base convertida a puntos, a la cual posteriormente añade sus propios bonus fijos y penalizaciones como se ha mencionado en el punto 3.1.

A nivel estratégico, es fundamental para el mánager conocer qué factores puntúan y con qué peso según la posición de cada jugador al elegir su equipo, ya que el valor de un activo se define por una combinación de estadísticas. Todos los jugadores se engloban en una de las cuatro posiciones establecidas: Portero (PT), Defensa (DF), Mediocentro (MC) y Delantero (DL). El valor de una acción clave depende directamente de su posición en el campo. Esto significa que la misma acción estadística puede tener un peso diferente.

Los principales valores que se ajustan según la posición incluyen, por ejemplo, goles marcados, donde el gol de un portero o defensa es más valorado por su infrecuencia. Del mismo modo, las entradas exitosas se valoran más positivamente por un defensor dada su relevancia en el deporte. Por ejemplo, los goles marcados:

Posición	Puntos Adicionales
PT (Portero)	6 puntos
DF (Defensa)	5 puntos
MC (Mediocentro)	4 puntos
DL (Delantero)	3 puntos

Cuadro 2: Puntos adicionales por gol según posición del jugador

1.2.5 El desafío de la información y el papel del análisis avanzado

Para contrarrestar los riesgos de subjetividad, opacidad algorítmica y eventos impredecibles detallados en la sección anterior, es imperativo que cualquier sistema de análisis que pretenda ser competitivo se base en la información más objetiva posible.

1.2.5.1 Estadísticas clásicas

El primer nivel de análisis se basa en las acciones cuantificables del partido: goles, asistencias, regates, pases clave, tiros a puerta, intercepciones y tarjetas. Estas estadísticas clásicas son fundamentales porque tienen un impacto directo y conocido en la puntuación fantasy. Son de fácil acceso y forman la base histórica de cualquier pronóstico.

Sin embargo, el uso exclusivo de estas métricas presenta limitaciones severas en la predicción del rendimiento futuro:

- **Pobreza de frecuencia e irregularidad:** muchas acciones clave, como los goles o las asistencias, son eventos de baja frecuencia. Un jugador puede haber tenido un excelente rendimiento en varios partidos sin acumular estas estadísticas, lo que invisibiliza su valor real y dificulta predecir su rendimiento semanal con precisión.
- **Falta de contexto:** una métrica simple (ej. un disparo a puerta) no informa sobre la calidad ni la posición del pase o del tiro. Esta falta de matiz impide una valoración objetiva del riesgo y la oportunidad.

1.2.5.2 El auge de la estadística avanzada

La limitación en la calidad predictiva de las estadísticas clásicas es lo que ha impulsado la revolución del Football Analytics a nivel global. Actualmente, la incorporación de especialistas en análisis de datos y ciencia de datos forma parte integral de prácticamente todos los cuerpos técnicos profesionales. Estos equipos están dedicados a transformar Big Data en información útil para la estrategia, lo que ha elevado el estándar de análisis en el deporte.

El Big Data en el fútbol se basa en datos de seguimiento y datos de eventos, lo que permite calcular métricas avanzadas que reflejan la calidad, la probabilidad y el impacto real de las acciones en el campo. Algunos ejemplos son:

- **Goles esperados (Expected Goals o xG):** mide la probabilidad de que un tiro

a puerta resulte en gol, basándose en miles de disparos históricos con características similares (distancia, ángulo, tipo de asistencia, etc.). El xG actúa como un indicador adelantado del potencial ofensivo, proporcionando una medida del rendimiento ofensivo subyacente de un jugador, lo que permite pronosticar un aumento en sus goles futuros si registra un alto xG con baja conversión real.

- **Asistencias esperadas (Expected Assists o xA):** mide la probabilidad de que un pase clave resulte en una asistencia. El xA valora la calidad del pase que precede a un disparo, independientemente de si el compañero consigue marcar o no. Si un jugador genera consistentemente un alto xA pero tiene pocas asistencias reales, sugiere que está creando oportunidades de calidad y que sus asistencias reales aumentarán.

1.2.6 Riesgos y desafíos

El proceso de pronosticar la puntuación fantasy se enfrenta a riesgos significativos y limitaciones inherentes a la naturaleza de los datos deportivos. La tarea de predicción es extremadamente difícil, y además, las dos alternativas principales de puntuación presentan fallos fundamentales:

- **Sesgo e inconsistencia humana en los sistemas subjetivos:** el sistema de puntuación por crónicas es altamente susceptible al sesgo humano. El periodista puede mostrar predilección personal por ciertos equipos o jugadores, puede dejarse influir por el resultado final del partido, o ser inconsistente en su criterio semana a semana. Modelar esta aleatoriedad es extremadamente difícil para cualquier algoritmo y constituye el principal riesgo de la predicción subjetiva.
- **Opacidad de los algoritmos de los sistemas objetivos:** aunque los sistemas objetivos como SofaScore prometen objetividad y exactitud, los pesos exactos que el algoritmo otorga a cada acción (pases, duelos ganados, etc.) y la fórmula final utilizada para combinar miles de datos son completamente opacos. Esta falta de acceso a la función de puntuación exacta obliga a tratar el algoritmo como una “caja negra”.
- **Eventos impredecibles:** ambos sistemas son igualmente vulnerables a los eventos impredecibles, una limitación intrínseca a la propia naturaleza de los datos y del deporte: cualquier pronóstico basado en datos históricos es vulnerable a eventos futuros no anticipados y poco probables, como lesiones, cambios tácticos, goles en propia puerta o rotaciones de la plantilla, lo cual introduce un ruido constante que

desafía la precisión de cualquier sistema predictivo.

1.3 Justificación del proyecto

El presente proyecto nace de la necesidad de profesionalizar y sistematizar la toma de decisiones en el ecosistema del Fantasy. En la actualidad, el mánager toma decisiones de forma puramente intuitiva o sesgada. Esto genera una oportunidad tecnológica: la creación de un sistema de información que transforme los datos históricos y forma reciente en inteligencia estratégica aplicable.

La propuesta no se limita a una mera visualización de estadísticas; se trata de una plataforma web diseñada como un centro de gestión de activos, donde el análisis predictivo y la administración de plantillas convergen para dotar al usuario de una ventaja competitiva basada en evidencias empíricas. Como factor diferencial y de vanguardia, el sistema no solo implementa modelos de aprendizaje supervisado para predecir puntuaciones, sino que introduce una capa de *XAI* (*Explainable Artificial Intelligence*).

En los sistemas predictivos tradicionales, el usuario recibe un dato (ej. “Este jugador obtendrá 7 puntos”) sin conocer la lógica subyacente, lo que genera desconfianza. Esta propuesta rompe esta “caja negra” permitiendo que el sistema explique qué variables han pesado más en la predicción. Este enfoque de XAI cumple un doble propósito:

- **Transparencia:** el usuario comprende los factores críticos de éxito de su plantilla.
- **Validación de hipótesis:** permite al mánager confrontar su conocimiento futbolístico con los pesos reales que el modelo asigna a cada métrica, facilitando un aprendizaje continuo y una toma de decisiones mucho más robusta.

2. Objetivos

2.1 Objetivo general

El objetivo general de este trabajo de fin de grado es el diseño e implementación de una plataforma web orientada al análisis predictivo y a la gestión de activos en el ámbito del Fantasy Football, con el fin de apoyar la toma de decisiones de los managers en un entorno caracterizado por la elevada complejidad y volumen de información.

Para ello, la plataforma se apoya en modelos de *machine learning* capaces de procesar datos históricos y actuales con el objetivo de generar predicciones sobre el rendimiento futuro de los jugadores y recomendaciones que contribuyan a optimizar la gestión de las plantillas, tanto en la selección de alineaciones como en la planificación de fichajes y ventas.

Un aspecto fundamental del objetivo general es la incorporación de técnicas de *Explainable Artificial Intelligence* (XAI), orientadas a dotar de transparencia a los modelos predictivos y a permitir que los usuarios comprendan los factores que influyen en cada recomendación, favoreciendo así la confianza y la validación de las decisiones propuestas por el sistema.

Asimismo, la plataforma se concibe con un enfoque de accesibilidad progresiva, combinando el acceso abierto a funcionalidades básicas sin necesidad de registro con un conjunto de funcionalidades avanzadas personalizadas para usuarios autenticados.

Finalmente, el sistema integra el análisis del contexto competitivo, incluyendo variables relacionadas con el próximo rival, con el objetivo de ofrecer predicciones y recomendaciones contextualizadas y alineadas con la realidad competitiva del *Fantasy Football*.

2.2 Objetivos funcionales

Para la consecución del objetivo general, se definen los siguientes objetivos que describen las capacidades del sistema, y serán detallados más adelante en el punto 4 Requistios:

OBJ-F1: Permitir la visualización de clasificación LaLiga y de los equipos de *Fantasy Football* sin necesidad de registro previo. Esta funcionalidad está orientada a ofrecer un acceso abierto a información relevante, facilitando que cualquier usuario pueda consultar datos básicos, estadísticas y análisis generales sin comprometer información personal. Esta decisión contribuye

a mejorar la accesibilidad y a aumentar el alcance potencial de la plataforma.

- OBJ-F2:** Generación de predicciones sobre el rendimiento de los jugadores. Estas predicciones se basan en el análisis de datos históricos, estadísticas actuales y otros factores contextuales relevantes. El sistema debe ser capaz de transformar grandes volúmenes de información en estimaciones comprensibles que ayuden a los usuarios a anticipar el comportamiento futuro de los jugadores en las próximas jornadas.
- OBJ-F3:** Generar recomendaciones automáticas para la gestión de la plantilla de los usuarios registrados. Dichas recomendaciones abarcan aspectos clave del *Fantasy Football*, como la selección de alineaciones óptimas, la identificación de oportunidades de fichaje, la venta de jugadores con bajo rendimiento esperado o la sustitución estratégica de determinados activos. Este enfoque permite apoyar la toma de decisiones del usuario sin sustituir su criterio personal.
- OBJ-F4:** Análisis del próximo rival constituye otro objetivo funcional relevante del sistema. El rendimiento de un jugador no depende únicamente de sus estadísticas individuales, sino también del contexto del enfrentamiento. Por ello, la plataforma incorpora factores como la dificultad del partido, el estado de forma del rival o el rendimiento defensivo del equipo contrario, enriqueciendo así la calidad de las recomendaciones ofrecidas.
- OBJ-F5:** Para los usuarios autenticados, la plataforma ofrece funcionalidades avanzadas orientadas a la personalización de la experiencia. Entre ellas se incluye la gestión individualizada de equipos, permitiendo al usuario adaptar el sistema a su situación concreta dentro de la liga Fantasy.
- OBJ-F6:** Presentación de explicaciones claras y comprensibles sobre las recomendaciones generadas. El sistema no solo debe indicar qué acción se recomienda, sino también por qué se propone dicha acción, destacando los factores más influyentes en la decisión. Este enfoque refuerza la confianza del usuario y favorece un uso crítico de las recomendaciones automáticas.
- OBJ-F7:** Asegurar una interfaz web intuitiva y accesible, diseñada para ser utilizada tanto por usuarios con experiencia en *Fantasy Football* como por usuarios principiantes. La claridad visual, la organización de la información y la

facilidad de navegación son aspectos clave para asegurar una experiencia de usuario satisfactoria.

OBJ-F8: Evaluar los modelos predictivos asegurando que superen un umbral mínimo de rendimiento según la métrica seleccionada (por ejemplo, MAE, RMSE o precisión). Esta evaluación garantiza que las predicciones sean confiables y útiles para la toma de decisiones, evitando incorporar modelos con resultados insuficientes o inconsistentes que puedan comprometer la validez de las recomendaciones. Se profundizará más en estos aspectos en apartados posteriores del proyecto.

OBJ-F9: Ofrecer vistas detalladas de equipos y jugadores, incluyendo estadísticas históricas, rendimiento reciente, predicciones futuras y contexto competitivo, permitiendo al usuario profundizar en el análisis de cada activo.

OBJ-F10:

Incorporar un apartado social que permita a los usuarios registrados añadir amigos, consultar sus plantillas y comparar estrategias, fomentando la interacción, la competitividad y el aprendizaje colaborativo entre managers.

2.3 Objetivos técnicos

El eje técnico fundamental del proyecto radica en la implementación de modelos de *machine learning* orientados a la predicción del rendimiento de los jugadores y a la generación de recomendaciones. Estos modelos deben ser capaces de procesar datos deportivos reales procedentes de distintas fuentes, adaptarse a la evolución temporal de la competición y ofrecer resultados consistentes que sirvan de apoyo a la toma de decisiones.

La obtención de los datos necesarios para alimentar estos modelos representa, por sí misma, un desafío tecnológico de primer orden. Para ello, el sistema incorpora mecanismos de web scraping sobre fuentes fiables, permitiendo la recopilación automática de estadísticas y contexto competitivo. Este proceso conlleva una complejidad añadida, ya que exige armonizar la heterogeneidad de estructuras y formatos de cada fuente externa.

En particular, uno de los principales retos asociados al uso de múltiples orígenes de información es la unificación de los datos relativos a los jugadores. Dado que diferen-

tes sitios web emplean nomenclaturas o variaciones ortográficas distintas, el proyecto contempla el diseño de un mecanismo de *matching* y normalización de entidades. Esta funcionalidad resulta imprescindible para garantizar la coherencia de la información y evitar inconsistencias que afecten a la precisión predictiva.

La integración de técnicas de *Explainable Artificial Intelligence* (XAI) constituye otro pilar del proyecto. Se busca incorporar métodos que permitan analizar la relevancia de las variables y ofrecer explicaciones interpretables para los usuarios. Esta ambición combina aspectos avanzados de IA con requisitos de usabilidad, suponiendo además un hito innovador al tratarse de una tecnología no explorada previamente por el autor.

Asimismo, se propone el desarrollo de un sistema de autenticación y autorización que permita diferenciar entre usuarios anónimos y registrados, garantizando un acceso controlado a las funciones avanzadas y protegiendo la información personal.

Por otro lado, la gestión y el procesamiento de datos deportivos reales se asumen como parte fundamental del trabajo. El sistema debe asegurar la limpieza y actualización constante de la base de datos, ya que la calidad de la información de entrada influye directamente en la validez de las recomendaciones ofrecidas.

Como fase esencial del desarrollo, se establece la validación y evaluación del sistema mediante métricas adecuadas que permitan medir la calidad de las predicciones en distintos escenarios. Este procedimiento permite justificar las decisiones técnicas adoptadas e identificar con rigor las fortalezas y del software.

3. Análisis de aplicaciones similares

Para contextualizar el desarrollo de la plataforma propuesta, resulta imprescindible realizar un estudio del estado del arte que identifique las soluciones actuales, sus fortalezas y sus carencias tecnológicas. En este apartado se lleva a cabo un análisis comparativo de las herramientas más representativas del ecosistema *Fantasy* en España, evaluando tanto plataformas comerciales consolidadas como proyectos de código abierto.

El objetivo de este examen es definir un marco de referencia funcional, detectar oportunidades de innovación y justificar las decisiones de diseño que permitirán dotar a este proyecto de un valor añadido diferencial. El estudio comienza con el análisis de AnalíticaFantasy.com, uno de los referentes actuales en la gestión de datos estadísticos aplicados al juego.

3.1 AnalíticaFantasy.com

3.1.1 Descripción general de la plataforma

La web analizada proporciona un entorno completo para el seguimiento del fútbol y el Fantasy Football. En su página principal, la aplicación muestra noticias destacadas y un componente de actualización en tiempo real sobre el estado de forma de los jugadores, indicando cambios relevantes como “duda leve” a “disponible”. Además, ofrece información sobre los partidos de las próximas dos jornadas, mostrando la alineación probable de cada equipo y estimaciones de titularidad para los jugadores. La plataforma también proporciona informes comparativos sobre goles, partidos jugados, penaltis, tarjetas y otras métricas, así como indicadores de la probabilidad de que cada equipo gane, ofreciendo así una visión integral del rendimiento y contexto competitivo.

3.1.2 Detalles de equipos

En la vista de cada equipo, se ofrece información completa sobre el próximo partido, incluyendo el rival y el horario del encuentro, así como la alineación probable con estimaciones de titularidad para cada jugador. Además, incorpora información contextual como las condiciones meteorológicas durante el partido, el registro de encuentros recientes del equipo y estadísticas de la temporada, incluyendo partidos jugados, ganados, empatados, perdidos, goles a favor y en contra, y las formaciones empleadas.

La plataforma también integra noticias relacionadas con el equipo y una sección interactiva de encuestas en la que los usuarios pueden votar sobre qué equipo creen que ganará el próximo partido. La información Fantasy del equipo se complementa mostrando los jugadores con más puntos totales, mejor media, mayor valor de mercado y aquellos que han incrementado su precio recientemente. Asimismo, se incluyen detalles sobre jugadores lesionados, sancionados o apercibidos, y estadísticas puramente deportivas, como minutos jugados, goles, asistencias, suplencias y titularidades.

Además, debajo de la zona de suplentes incluye un componente que a cada posición asigna una dificultad, pero en ningún momento se detalla en base a qué se toma esa decisión.



Figura 1: Figura de referencia de la dificultad de un partido en AnalíticaFantasy.com.

3.1.3 Análisis detallado de partidos

La aplicación permite la visualización de partidos pasados con un alto nivel de detalle. Para cada encuentro, se muestran el resultado final, los goleadores, las tarjetas recibidas y los minutos jugados por cada jugador. Sobre las alineaciones se representan gráficamente los puntos obtenidos en cada modelo Fantasy, pudiendo cambiar la referencia mediante botones interactivos.

Al hacer clic sobre un jugador, se despliega un modal con toda la información individual. La plataforma también muestra los resultados de las encuestas relacionadas con el partido y gráficas que resumen estadísticas del equipo, incluyendo partidos jugados, ganados, empatados, perdidos, goles a favor y en contra y puntos obtenidos. Además, se registra cronológicamente cada suceso del encuentro, como cambios, goles y tarjetas, proporcionando un análisis completo y visual de todos los eventos significativos.

De nuevo en esta vista se incluye información sobre los partidos pasados y próximos de cada equipo.

3.1.4 Apartado H2H

La aplicación incluye un módulo denominado “H2H” que permite analizar enfrentamientos previos entre dos equipos, incorporando datos históricos de varias temporadas. Este apartado facilita la comparación directa de resultados y tendencias de rendimiento entre rivales específicos, apoyando el análisis predictivo y estratégico dentro del Fantasy Football.

3.1.5 Detalles de jugadores

Cada jugador dispone de una vista individual en la que se presenta información personal, equipo y posición, junto con los puntos esperados en la próxima jornada diferenciando si con una estimación diferente en función de si es titular o no. Se incluyen la media de puntos obtenidos en casa y fuera, el valor actual del jugador, así como su valor máximo y mínimo y la estimación de puja ideal dentro del Fantasy.

La plataforma también ofrece un histórico de los partidos de la temporada, puntos obtenidos y la información sobre las posiciones en las que ha jugado, permitiendo un análisis completo del rendimiento y evolución del jugador.

Además del perfil se puede navegar a otros apartados, como “Puntos”, donde se clasifica cada partidos del jugador en un rango según los puntos obtenidos, acompañado de un histograma, con el eje X siendo los partidos y el Y los puntos obtenidos. De forma similar, hay apartados de *Minutos*, *Mercado*, *Calendario* y *Proyección*, mostrando diferentes datos y gráficas del jugador y su valor de mercado.

3.1.6 Seguimiento de jugadores

La aplicación incorpora un apartado específico para el seguimiento de “Jugadores”, en la barra lateral, que se identifican los jugadores lesionados junto con el tipo de lesión y el tiempo estimado de recuperación. También se muestran los jugadores sancionados, indicando el motivo y la duración de la sanción, así como los jugadores apercibidos de sanción, lo que permite a los usuarios anticipar riesgos y gestionar su equipo Fantasy de manera más estratégica.

3.1.7 Predicciones

Desde el menú lateral también se puede navegar hacia “Predicciones Fantasy”, contrariamente a lo que parece, esta vista tan solo permite al usuario tratar de predecir

resultados de partidos futuros y jugadores que destacarán, positiva y negativamente.

3.1.8 Gestión de plantilla

Incorpora una vista donde el usuario puede configurar su equipo sobrepresionado en un campo de fútbol, pudiendo elegir entre diferentes formaciones y todos los jugadores de la liga. Para cada jugador se muestra el próximo partido, la probabilidad de ser titular y el valor de mercado.

Adicionalmente ofrece información una vez se ha confeccionado la plantilla, muestra datos agregados de goles, puntos, asistencias y otros factores determinantes.

3.2 JornadaPerfecta.com

3.2.1 Descripción general de la plataforma

La aplicación web analizada ofrece un acceso integral a la información de LaLiga y su ecosistema Fantasy. En la página de inicio, los usuarios pueden visualizar todos los equipos de la liga y acceder a sus detalles individuales, así como consultar un listado completo de los partidos de la próxima jornada para acceder a sus respectivos informes. Gran parte de la página principal se dedica a noticias destacadas y promociones de la aplicación, incluyendo gran cantidad de publicidad de casas de apuestas, cuya visualización requiere desactivar bloqueadores de anuncios.

3.2.2 Próximos partidos

En la vista de próximos partidos, la plataforma presenta información detallada sobre fecha y hora de los encuentros, alineaciones probables y datos visuales que reflejan la participación de los jugadores en los últimos tres partidos mediante indicadores de color (rojo, naranja o verde) para titular o suplente. Además, se alerta sobre jugadores aparcados de sanción.

Se incluye además, información sobre los próximos partidos de cada uno de los equipos y detalles de la retransmisión deportiva.

Esta sección mantiene la presencia de noticias y publicidad, integrando la información de manera visualmente destacada para los usuarios.



Figura 2: Figura de referencia de la vista de la plantilla de un equipo en JornadaPerfecta.com.

3.2.3 Detalles de equipo

La vista de cada equipo proporciona la alineación probable con elementos visuales adicionales que destacan información relevante de cada jugador. La plataforma incluye datos sobre jugadores no disponibles por sanción o lesión, así información especialmente relevante como los lanzadores habituales de faltas, penaltis y córners.

Además, se presentan actuaciones recientes de los jugadores, incluyendo los puntos obtenidos en los partidos y su posición, así como el valor de mercado dentro de la aplicación de Biwenger.

3.2.4 Noticias y recomendaciones

El apartado de noticias combina información deportiva con recomendaciones de fichajes o traspasos, presentadas de manera visualmente clara y destacada. Esta sección también incluye un espacio de comentarios utilizado por los usuarios para compartir

dudas o solicitar consejos sobre decisiones a tomar en sus equipos Fantasy, fomentando la interacción y el intercambio de información entre la comunidad.



Figura 3: Figura de referencia del apartado de noticias.

3.2.5 Mercado

Se incluye un apartado donde el usuario puede ver el valor de mercado de cada jugador, cuenta con diferentes filtros donde se puede elegir entre otros: modelo de puntuación fantasy, equipo, posiciones... Adicionalmente muestra el listado de jugadores con información histórica del precio de cada jugador.

3.2.6 Seguimiento de jugadores lesionados

La plataforma incorpora una vista específica dedicada a los jugadores lesionados, organizada por equipos, que muestra información detallada sobre la lesión de cada jugador y el plazo estimado de regreso a la competición. Este apartado permite a los usuarios gestionar sus plantillas considerando las bajas y planificando estrategias alternativas de manera más informada.

3.2.7 Detalles de jugador

La vista individual de cada jugador ofrece información básica sobre su participación en los partidos recientes. Se muestran su posición, dorsal y valor de mercado tanto en Biwenger como en LaLiga Fantasy. La plataforma incluye un registro histórico de puntos de cada jornada, diferenciando entre partidos disputados en casa y fuera, y presenta información sobre los próximos encuentros del jugador. Complementariamente,

se incorpora un gráfico que representa la evolución del valor de mercado del jugador durante el último mes en ambos mercados, acompañado nuevamente de noticias y actualizaciones relevantes sobre su rendimiento y situación deportiva.

3.3 Repositorios

3.3.1 Repositorios analizados

Se han explorado proyectos como “Football-Prediction” y “LaLiga-Prediction-Analytics”, los cuales han permitido comprender el flujo de trabajo típico en este tipo de proyectos y las tecnologías que suelen utilizarse, como pandas, scikit-learn y Matplotlib. Además, estos proyectos han servido para evaluar si las fuentes de información planteadas inicialmente, FBref.com y futbolfantasy.com, son realmente fiables. Se comprobó que, en su mayoría, se utiliza FBref como fuente principal, lo que confirma la intuición inicial de que se trata de un sitio con datos confiables.

La principal diferencia y limitación respecto al proyecto que se va a realizar radica en el nivel de predicción. Mientras que estos repositorios se centran en predecir el resultado final de una temporada o los resultados de un partido con cierta probabilidad, por ejemplo:

Home: Barcelona Away: Real Madrid
Predicted: Home Win Confidence: 78 %

Figura 4: Salida de la API de predicción mencionada en el análisis.

no ofrecen información detallada a nivel de jugadores.

En el presente proyecto, será necesario realizar scraping de información más detallada, no solo de los partidos en sí, sino de los jugadores que participan en cada partido. Esto añade una capa de complejidad significativa, ya que la obtención de los datos será más extensa y requerirá un mayor preprocesamiento para unificar y limpiar la información.

3.3.2 Repositorios sin mantenimiento

Se han encontrado otros repositorios, entre ellos “laliga-fantasy-bot”, “marca-fantasy-api-scraper-updated” y “laligafantasy”, en busca de proyectos con características similares al presente trabajo. La mayoría de estos proyectos funcionan únicamente como scrapers, limitándose a generar un archivo.csv con información de jugadores que varía según el repositorio consultado (minutos jugados, valor de mercado, etc.).

El repositorio más parecido fue “laligafantasy”, que ofrecía recomendaciones de alineaciones consideradas óptimas según la suma de puntos estimada y sugería al usuario incluso la formación a utilizar. Otras funcionalidades del proyecto no pudieron ser verificadas debido a la falta de mantenimiento, lo que constituye la principal limitación al buscar repositorios de código similares.

Todos estos proyectos tienen entre 2 y 6 años de antigüedad, carecen de mantenimiento y presentan numerosas issues abiertas, muchas de las cuales indican que ya no funcionan correctamente.

Durante este análisis también se identificó una limitación importante para el desarrollo del presente trabajo: ya no existe una API pública de LaLiga Fantasy. Todos sus endpoints han sido migrados a Android, lo que hace imposible realizar scraping para obtener información actualizada sobre el valor de mercado de los jugadores. Aunque se podría trabajar con datos de la temporada actual, se decidió omitir la gestión del mercado para mantener coherencia con los datos históricos utilizados en el entrenamiento del modelo.

3.4 Conclusiones y valor añadido

El análisis detallado de plataformas como FutbolFantasy.com y AnalíticaFantasy.com, junto con los repositorios relacionados, se ha encontrado un conjunto de funcionalidades recurrentes. Todas estas soluciones proporcionan un análisis exhaustivo de los encuentros disputados, incluyendo resultado final, goleadores, alineaciones titulares y suplentes, puntuaciones Fantasy por jugador, minutos jugados y eventos relevantes como tarjetas y cambios, así como estadísticas generales del partido (tiros, córners, pases, etc.).

Cada web integra vistas detalladas de jugadores con información esencial: posición y dorsal, valor de mercado actual, histórico de puntuaciones por jornada, comparativas de rendimiento en casa versus fuera, y estado del jugador (disponible, lesionado o sancionado). Asimismo, todas implementan un seguimiento completo de la disponibilidad de jugadores, incluyendo lesiones con tiempo estimado de recuperación, sanciones y jugadores apercibidos de riesgo de sanción próxima.

A nivel de equipo, las plataformas ofrecen datos consolidados como clasificación, próximos partidos, calendario completo, alineación probable y estadísticas de temporada (partidos jugados, ganados, empatados, perdidos, goles a favor y en contra). También incorporan información de los mercados Fantasy más populares, mostrando valor de mercado, estimaciones de titularidad y cambios recientes de precio. Finalmente, se in-

cluyen datos contextuales como noticias, comparativas H2H y condiciones del partido (horario, lugar y clima).

3.4.1 Limitaciones identificadas del presente trabajo

Uno de los principales obstáculos técnicos radica en la inaccesibilidad de datos financieros en tiempo real. Debido a que, como se ha comentado, la API pública de LaLiga Fantasy ha restringido sus endpoints y los ha migrado a entornos cerrados de aplicaciones móviles, resulta imposible realizar un scraping automatizado y estable para obtener los valores de mercado de los jugadores. Esta carencia de una fuente de datos económica oficial y constante obliga al proyecto a prescindir de la gestión de presupuestos y fluctuaciones de precio, centrando todo el esfuerzo analítico exclusivamente en el rendimiento deportivo y las métricas de juego, donde los datos históricos son más consistentes.

En cuanto a la lógica predictiva, el proyecto reconoce una limitación en el modelado de las alineaciones probables. Desarrollar un algoritmo propio capaz de predecir la titularidad con alta precisión requeriría procesar variables cualitativas muy complejas, como el estado anímico del vestuario o noticias. Por ello, se ha optado únicamente por predecir rendimiento, dejando de lado la titularidad del jugador, este dato estará presente a la hora de entrenar modelos de machine learning pero no se explicitará en la interfaz.

Finalmente, es necesario considerar la diferencia de los datos disponibles frente a plataformas ya consolidadas como FutbolFantasy o JornadaPerfecta. Estos proyectos cuentan con años de madurez, infraestructuras robustas y, sobre todo, un volumen de datos históricos masivo que permite entrenar modelos con una profundidad que un proyecto individual no puede igualar a corto plazo y sin APIs que en su mayoría son de pago.

3.4.2 Valor añadido de la aplicación propuesta

A pesar de las limitaciones inherentes a un desarrollo independiente, esta plataforma se diferencia de las soluciones comerciales mediante una propuesta de valor centrada en la calidad del dato y la utilidad directa para el usuario.

- **Explicabilidad de las predicciones (XAI):** a diferencia de las aplicaciones existentes, la plataforma propuesta implementa técnicas de Explainable AI para ofrecer análisis causal sobre los factores que influyen en las predicciones. Por ejemplo, permite responder preguntas como: ¿Qué impacta más en la predicción:

minutos jugados, tiros del rival o forma reciente del portero? Este enfoque convierte las predicciones en información interpretable y confiable, evitando la “caja negra” de los modelos tradicionales y aumentando la confianza del usuario en decisiones complejas o contraintuitivas.

- **Recomendaciones de jugadores por posición:** la aplicación ofrecerá un sistema de recomendaciones, identificando los tres mejores jugadores por posición para cada jornada.
- **Entorno puramente deportivo:** a diferencia de las plataformas existentes, la aplicación se centra únicamente en información deportiva: partidos, rendimiento y estadísticas de jugadores y equipos. No incluye publicidad ni noticias externas, ofreciendo un entorno limpio y enfocado para la toma de decisiones en Fantasy Football.

En conjunto, estas innovaciones permiten ofrecer un producto que, si bien es más acotado en volumen de datos, resulta mucho más avanzado en su capacidad analítica. Al centrarse en la interpretación del dato y en un entorno de usuario optimizado, la aplicación supera las barreras de las soluciones actuales, aportando una ventaja estratégica real y una experiencia de uso profesional para el mánager de Fantasy Football.

4. Metodología

Para el desarrollo de este proyecto se evaluaron diferentes metodologías ágiles, siendo las principales candidatas Scrum y Kanban. A continuación se presenta el análisis comparativo que justifica la elección final.

4.1 Evaluación de Scrum

Ventajas consideradas:

- Metodología ágil más utilizada en la industria del desarrollo software.
- Sprints de tiempo fijo con entregas incrementales, muy útil para forzar la constancia y el trabajo continuo.
- Ceremonias estructuradas (Sprint Planning, Daily Scrum, Sprint Review, Retrospective) que permiten la correcta gestión del proyecto.
- Estimaciones de tiempo que facilitan la planificación.

Inconvenientes para este proyecto:

- Scrum está diseñado para equipos, no para desarrollo individual. Los roles (Product Owner, Scrum Master, Developer) resultan artificiales cuando el desarrollador es una única persona.
- Las ceremonias diarias (por ejemplo, el Daily Scrum) carecen de sentido práctico en contexto individual.
- Rigidez de los sprints: una vez iniciado el sprint, no se pueden modificar los objetivos ni las tareas seleccionadas hasta su finalización.

Motivos de descarte:

La naturaleza individual del proyecto y la ausencia de entregas incrementales intermedias hacen que Scrum introduzca complejidad innecesaria sin aportar beneficios reales. Adaptar artificialmente los roles y ceremonias no representa fielmente cómo se desarrolló el proyecto.

4.2 Metodología seleccionada: Kanban

Kanban es una metodología ágil basada en flujo continuo de trabajo que visualiza el proceso mediante un tablero dividido en columnas que representan estados de las tareas. No utiliza iteraciones de tiempo fijo ni roles predefinidos.

Estructura del tablero Kanban implementado:

- **Backlog:** tareas pendientes de iniciar
- **To Do:** tareas priorizadas para desarrollo inmediato
- **In Progress:** tareas en desarrollo activo (límite Work In Progress: 3 tareas)
- **Done:** tareas completadas y validadas

Ventajas para este proyecto:

- **Flexibilidad:** permite reordenar prioridades según descubrimientos técnicos o cambios en requisitos sin violar reglas metodológicas.
- **Flujo natural:** se adapta al ritmo real de trabajo individual sin imposiciones de tiempo o plazos de entrega intermedios.
- **Visualización clara:** el tablero proporciona instantáneamente el estado del proyecto.

Desventajas asumidas:

- Menor estructura formal para métricas de velocidad.
- Requiere disciplina personal para mantener límites WIP y actualizar el tablero.

Kanban resulta más apropiado para el contexto de este TFG al tratarse de un proyecto de desarrollo individual sin entregas intermedias formales. La flexibilidad de Kanban permite adaptarse a las necesidades reales del proyecto manteniendo trazabilidad completa del proceso. Además, al no estar sujeto a sprints de tiempo fijo, se facilita la gestión temporal del proyecto en coordinación con otras asignaturas del grado, permitiendo ajustar la carga de trabajo según períodos de exámenes, entregas de otras materias y disponibilidad personal, algo que la rigidez de los sprints de Scrum dificultaría considerablemente.

Implementación de Kanban

Herramienta utilizada: GitHub Projects

Herramienta de monitorización: Clockify

Límites WIP aplicados:

- Máximo 3 tareas en “In Progress” simultáneamente

Priorización: Las tareas del Backlog se priorizaron y planifican según dependencias técnicas y criticidad funcional.

4.3 Planificación Temporal

Herramienta de planificación: Microsoft Project

Herramienta de monitorización: Clockify

Parámetros de planificación:

- Rol único: Developer
- Jornada laboral: 4 horas diarias
- Duración total estimada: 17 semanas

El diagrama de Gantt generado (ver Figura 5) muestra la distribución temporal de las diferentes fases del proyecto: análisis y diseño inicial, desarrollo de módulos principales, integración, pruebas y documentación final.

A pesar de que el estándar habitual de una jornada de trabajo se establece en 8 horas diarias, en la planificación de este proyecto se ha considerado cada día como equivalente a 4 horas de trabajo efectivo. Esta decisión se debe a que no resulta realista asumir una dedicación diaria de 8 horas completas al TFG, dado que este se desarrolla en paralelo con otras asignaturas del grado que también requieren una inversión significativa de tiempo y esfuerzo.

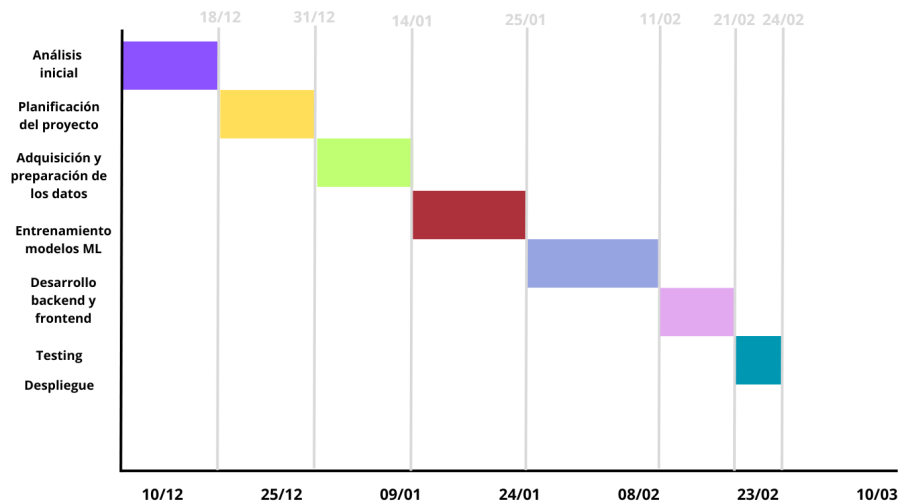


Figura 5: Diagrama de Gantt de la línea base del cronograma.

4.4 EDT (Estructura Desglose de Trabajo)

A continuación, se presenta la Estructura de Desglose de Trabajo (EDT), que organiza el trabajo en paquetes manejables y define la base lógica sobre la que se asientan las actividades y recursos detallados en el diccionario de la EDT posterior.



Figura 6: Estructura de Desglose de Trabajo (EDT) del proyecto.

Diccionario de la EDT

Id: 1

Nombre paquete: Análisis e investigación inicial

Descripción: Investigación y análisis de proyectos existentes relacionados con predicción deportiva y aplicación de machine learning en fútbol, con el objetivo de identificar buenas prácticas, limitaciones y oportunidades de mejora.

Actividades

Actividad	Duración	Recurso
A1 - Estudio proyectos similares	4 días	Google Chrome, Github
A2 - Investigación fuentes de datos	2 días	Google Chrome
A3 - Evaluar viabilidad web scraping	2 días	Google Chrome, documentación técnica
A4 - Análisis de competidores	4 días	Google Chrome

Id: 2

Nombre paquete: Planificación del proyecto

Descripción: Definición formal del alcance del proyecto, especificación de requisitos funcionales y no funcionales, historias de usuario y planificación detallada de recursos y cronograma.

Actividades

Actividad	Duración	Recurso
A5 - Definición del alcance del proyecto	2 días	Google Docs
A6 - Especificación Requisitos Funcionales y no Funcionales	2 días	Google Docs
A7 - Definición historias de usuario y mockups	3 días	Google Docs
A8 - Elección stack tecnológico	2 días	Documentación técnica
A9 - Planificación temporal y de recursos	2 días	MS Project

Id: 3

Nombre paquete: Adquisición y preparación de los datos

Descripción: Obtención de datos mediante web scraping de las fuentes identificadas, almacenamiento en formato CSV y preprocesamiento para su posterior uso en modelos de machine learning. Esta fase incluye limpieza de datos, tratamiento de valores nulos, normalización.

Actividades

Actividad	Duración	Recurso
A10 - Desarrollo scripts de scrapping	6 días	VS Code, Python, pandas
A11 - Matching de datos	2 días	VS Code, Python, pandas
A12 - Almacenamiento en formato CSV	1 día	VS Code, Python, pandas
A13 - Preprocesamiento y limpieza de datos	6 días	VS Code, Python, pandas, numpy

Id: 4

Nombre paquete: Entrenamiento modelos de ML

Descripción: Selección, entrenamiento y optimización de modelos predictivos para la evaluación del rendimiento de jugadores. Se implementan técnicas de validación cruzada, ajuste de hiperparámetros mediante Grid Search y técnicas de explicabilidad XAI para garantizar predicciones precisas y comprensibles.

Actividades

Actividad	Duración	Recurso
A14 - Selección algoritmos candidatos	1 día	VS Code, documentación de scikit-learn
A15 - Entrenamiento inicial de los modelos	2 días	VS Code, Python, scikit-learn
A16 - Ajuste de hiperparámetros, validación cruzada y evaluación	12 días	VS Code, Python, scikit-learn
A17 - Investigación e implementación XAI	6 días	VS Code, Python, Chrome

Id: 5

Nombre paquete: Desarrollo Backend y Frontend

Descripción: Selección, entrenamiento y optimización de modelos predictivos para la evaluación del rendimiento de jugadores. Se implementan técnicas de validación cruzada, ajuste de hiperparámetros mediante Grid Search y técnicas de explicabilidad XAI para garantizar predicciones precisas y comprensibles.

Actividades

Actividad	Duración	Recurso
A18 - Configuración inicial del proyecto	1 día	VS Code, Django
A19 - Diseño de la arquitectura final	2 días	Mermaid
A20 - Modelado de la BD	4 días	Mermaid
A21 - Desarrollo vistas y templates	10 días	VS Code, Python, Django

Id: 6

Nombre paquete: Testing

Descripción: Verificación del correcto funcionamiento del sistema mediante pruebas unitarias, pruebas sobre la lógica de obtención y combinación de datos, y pruebas sobre las vistas de la aplicación web, con el objetivo de garantizar la calidad del software antes del despliegue.

Actividades

Actividad	Duración	Recurso
A22 - Pruebas unitarias de funcionalidades clave	6 días	VS Code, pytest
A23 - Pruebas de consistencia de los datos	2 días	VS Code, Python
A24 - Corrección de errores	4 días	VS Code, Python

Id: 7

Nombre paquete: Despliegue

Descripción: Configuración del entorno de producción y despliegue de la aplicación en una plataforma cloud accesible a través de contenedores de Docker.

Actividades

Actividad	Duración	Recurso
A25 - Configuración del entorno de producción	2 días	Docker, DockerHub
A26 - Despliegue	1 día	Microsoft Azure

4.5 Planificación de riesgos

Para asegurar el éxito del proyecto, se ha llevado a cabo una identificación y evaluación de los posibles riesgos que podrían impactar en el desarrollo, estimando su probabilidad

de ocurrencia y su impacto en el cronograma o alcance.

Identificación y Matriz de Riesgos

Probabilidad/Impacto	Bajo	Medio	Alto	Muy Alto
Muy Alta	R4	R2	R1	R7
Alta		R3, R6	R5	
Media	R9		R8	
Baja				

Figura 7: Matriz de riesgos del proyecto.

ID	Nombre	Probabilidad	Impacto
R1	Restricciones en la obtención de datos	Muy alta	Alta
Descripción: Imposibilidad de extraer ciertos datos debido a bloqueos de IP, captchas, APIs limitadas o cambios estructurales en el DOM de las páginas fuente.			
Plan de mitigación: Se implementarán buenas prácticas de scraping (tiempos de espera aleatorios, cabeceras User-Agents). Como contingencia, si el bloqueo es total, se buscarán alternativas web o descargarán y almacenarán los HTML manualmente.			

ID	Nombre	Probabilidad	Impacto
R2	Rendimiento predictivo deficiente	Media	Muy Alto
Descripción: Existe la incertidumbre de que los modelos de Machine Learning no logren una precisión o capacidad predictiva aceptable con los datos proporcionados. Plan de mitigación y contingencia: Si las métricas iniciales son deficientes, se realizará una iteración extra de Feature Engineering (creación de nuevas variables derivadas). Si el límite predictivo de los datos se mantiene bajo, la contingencia es cambiar el enfoque del TFG hacia el análisis y la investigación: en lugar de prometer una herramienta infalible, el valor del proyecto residirá en la comparativa exhaustiva de los algoritmos y en documentar científicamente por qué ciertas variables no son determinantes para predecir el rendimiento deportivo.			

ID	Nombre	Probabilidad	Impacto
R3	Solapamiento de calendario académico	Alta	Medio
Descripción: Picos de trabajo por entregas o exámenes de otras asignaturas que reduzcan la disponibilidad de 4 horas diarias previstas. Plan de mitigación: La elección metodológica de Kanban absorbe este riesgo al carecer de sprints rígidos. Permite pausar y reanudar tareas según la carga lectiva sin romper la planificación metodológica.			

ID	Nombre	Probabilidad	Impacto
R4	Manejo de dependencias cruzadas	Muy Alta	Bajo
Descripción: Conflictos de compatibilidad entre las diferentes versiones de librerías de Machine Learning (scikit-learn, pandas, numpy) y el entorno del framework web (Django). Plan de mitigación: Mitigación mediante el uso estricto de entornos virtuales desde el primer día, aislando el entorno de datos del entorno web hasta su integración, y congelando las versiones estables en un archivo requirements.txt.			

ID	Nombre	Probabilidad	Impacto
R5	Integración de modelos ML en el backend	Alta	Alto
Descripción: Desconocimiento inicial sobre la arquitectura óptima para guardar los modelos entrenados, cargarlos y realizar inferencias desde Django de manera eficiente.			
Plan de mitigación: Realizar una fase inicial de investigación sobre las prácticas recomendadas para almacenar y cargar modelos de aprendizaje automático en aplicaciones web con Django. Para ello, se revisará la documentación oficial de frameworks utilizados y ejemplos de integración con Django.			

ID	Nombre	Probabilidad	Impacto
R6	Curva de aprendizaje de XAI	Alta	Medio
Descripción: La implementación de técnicas de XAI puede presentar una curva de aprendizaje más pronunciada de lo estimado inicialmente.			
Plan de mitigación: Se comenzará integrando las herramientas de XAI más documentadas en la industria (como SHAP). Si el tiempo apremia, se reducirá el alcance de la explicabilidad.			

ID	Nombre	Probabilidad	Impacto
R7	Coste temporal en el matching y procesamiento de datos	Muy Alta	Muy alto
Descripción: Dificultad para cruzar datos de diferentes fuentes (por ejemplo, discrepancias en la nomenclatura de los nombres de los jugadores o equipos), lo que puede consumir gran parte de la fase de preparación.			
Plan de mitigación: En lugar de hacer cruces manuales o exactos, se programarán mecanismos utilizando librerías específicas como FuzzyWuzzy o TheFuzz, complementados con diccionarios manuales únicamente para los casos extremos.			

ID	Nombre	Probabilidad	Impacto
R8	Rendimiento deficiente en producción	Media	Alto
Descripción: Tiempos de respuesta lentos o caídas por falta de memoria al intentar cargar y ejecutar los modelos de ML una vez la aplicación esté desplegada en un servidor en la nube.			
Plan de mitigación: Para evitar que la web sea lenta, la arquitectura se diseñará para cargar el modelo de Machine Learning en la memoria del servidor una sola vez durante el arranque de la aplicación (ej. instanciándolo en el archivo apps.py de Django) y no en cada petición HTTP del usuario. Si los recursos del servidor gratuito son insuficientes, se optará por desplegar versiones más ligeras de los modelos (menos profundidad de árboles o menos parámetros, sacrificando algo de precisión).			

ID	Nombre	Probabilidad	Impacto
R9	Desactualización del dataset	Media	Bajo
Descripción: Al estar analizando datos de una competición deportiva en curso, cada fin de semana se generan nuevos datos. Existe el riesgo de que, para la fecha de la defensa, el modelo no incluya los últimos partidos o fichajes recientes.			
Plan de mitigación: Definir en el alcance del proyecto una “fecha de corte” clara (por ejemplo, “datos extraídos hasta la jornada 17”). Se trabajará con un dataset cerrado.			

4.6 Gestión de la Calidad

Para garantizar que el software desarrollado sea mantenible, escalable y libre de errores críticos, se han establecido las siguientes métricas y herramientas de calidad:

- **Análisis estático de código:** Se integrará SonarQube para evaluar continuamente la calidad del código. Esta herramienta detectará vulnerabilidades (security hotspots), code smells y código duplicado, asegurando que se cumplen los estándares de la industria.

4.7 Análisis de tiempos, costes y desviaciones

Tal y como se estableció en la planificación temporal, la jornada de trabajo estándar para este proyecto se fijó en 4 horas diarias, adaptándose a la realidad de la carga lectiva grado.

Durante el desarrollo, esta premisa se ha cumplido de forma general, con la excepción

del periodo vacacional de Navidad (del 24 de diciembre al 30 de enero). La mayor disponibilidad de tiempo en esas fechas permitió acelerar el ritmo de trabajo, logrando completar tareas en una menor cantidad de días calendario, aunque el volumen de horas de trabajo invertidas se mantuvo acorde a las estimaciones.

Para garantizar el éxito del proyecto y mitigar posibles solapamientos con entregas o exámenes de otras asignaturas, la estrategia principal de planificación consistió en concentrar el desarrollo para finalizar a finales de febrero. Esto proporcionó un amplio margen de holgura temporal en caso de no poder cumplir el primer plazo establecido.

Resumen temporal y desviaciones globales

A continuación, se presenta la comparativa entre la línea base del proyecto planificada y la ejecución real:

Parámetro	Planificado	Fecha real	Desviación
Fecha de inicio	10/12/2025	10/12/2025	–
Fecha de fin	12/03/2026	07/04/2026	+ 26 días naturales
Horas de trabajo total	364 h	384 h	+ 20 h
Días de trabajo (4h/día)	91 días	96 días	+ 5 días efectivos

Como se observa en los datos, el esfuerzo total del proyecto solo sufrió un incremento de 20 horas (equivalentes a 5 jornadas de trabajo de 4 horas). A nivel de calendario, la finalización se desplazó hasta el 7 de abril. Gracias al gran margen de holgura planificado desde el principio, este desplazamiento temporal fue absorbido íntegramente sin poner en riesgo la entrega ni la defensa del proyecto.

Comparativa de esfuerzo por paquetes de trabajo

En la siguiente tabla se desglosa el esfuerzo por cada paquete de la Estructura de Desglose de Trabajo (EDT), evidenciando dónde se produjeron las desviaciones:

ID	Paquete de trabajo	Horas previstas	Horas reales	Desviación
1	Análisis e investigación inicial	48 h	48 h	0 h
2	Planificación del proyecto	80 h	48 h	-32 h
3	Adquisición y preparación de datos	60 h	76 h	+16 h
4	Entrenamiento modelos de ML	48 h	68 h	+20 h
5	Desarrollo Backend y Frontend	68 h	88 h	+20 h
6	Testing	48 h	28 h	-20 h
7	Despliegue	12 h	28 h	+16 h
TOTAL		364 h	384 h	+20 h

Todas las entradas de la herramienta Clockify se pueden consultar en el Anexo II-Clockify.

Análisis de desviaciones

Al comparar la planificación inicial con el tiempo real invertido, se observan variaciones inherentes a la naturaleza de un proyecto que combina ingeniería de software y machine learning. Las diferencias más significativas se concentraron en las siguientes áreas:

- **Fase de entrenamiento y optimización:** las tareas relacionadas con el ajuste de features, hiperparámetros, validación cruzada y evaluación requirieron 48 horas frente a las 32 estimadas. Esto refleja la dificultad de acotar temporalmente las fases de experimentación, donde los resultados obligan a realizar múltiples iteraciones, pruebas y refinamientos hasta alcanzar la precisión deseada en el modelo.
- **Desarrollo de vistas y templates (Frontend/Backend):** su implementación alcanzó un total de 60 horas (frente a las 40 iniciales) debido al desarrollo iterativo necesario para integrar correctamente la lógica de los modelos predictivos en la interfaz visual.
- **Compensación de tiempos:** por el contrario, fases como la planificación del proyecto (estimada de forma muy conservadora en 80 h) o el testing requirieron menos dedicación de la prevista inicialmente, lo que ayudó a equilibrar el balance global de horas.

Estimación económica y desglose de costes

Para calcular el coste del desarrollo, se ha tomado como referencia la tarifa de un perfil profesional de Analista programador J2EE / Web, representativo del estándar de la industria de software. Los datos provienen del "Informe final sobre la consulta preliminar del mercado" de la Junta de Andalucía y disponible en la web de TFG de la ETSII US. En dicho documento, la media acotada para este perfil oscila entre los 28,37 €/hora para perfiles Junior y los 33,43 €/hora para perfiles Senior, por lo que se ha establecido un valor medio de 31 €/hora para este proyecto.

A diferencia de las estimaciones basadas únicamente en salarios brutos, este valor de 31 €/hora representa el precio/hora sin IVA que las empresas del sector facturan habitualmente a la Administración Pública. Al tratarse de una tarifa de licitación para la prestación de servicios profesionales, este importe ya integra de forma intrínseca todos los costes laborales asociados, incluyendo la Seguridad Social a cargo de la empresa, así como los gastos de estructura, herramientas, licencias y el beneficio industrial.

Por este motivo, la cifra empleada constituye una aproximación realista del coste total de mercado. Al cubrir ya todas las cargas sociales y costes indirectos dentro de la propia tarifa horaria de facturación, resulta innecesario aplicar factores correctores o porcentajes adicionales sobre el cálculo final.

Además del coste de personal, se han contemplado los siguientes gastos operativos y de capital:

CAPEX (Capital Expenditure):

- **Amortización de equipo informático:** según las tablas de la AEAT 2026, los equipos para tratamiento de información tienen un coeficiente de amortización lineal máximo del 26 % anual. Para un ordenador de 1.200 €, la amortización imputada por las 17 semanas de duración del proyecto se calcula como:

$1,200 \times 0,26 \times (17/52) = 102,00 \text{ €}$. Se establece un coste fijo de **102 €**.

OPEX (Operational Expenditure):

- Uso de herramientas externas y servicios cloud. Esto incluye el control de versiones mediante GitHub (plan Team, 4 €/usuario/mes), herramientas de Inteligencia Artificial como Gemini Pro (aprox. 20 €/mes), y el despliegue de la aplicación mediante contenedores Docker sobre Microsoft Azure (App Service y Container Registry), con un coste estimado de 30 €/mes.
- No obstante, gracias a la disponibilidad de licencias educativas y el programa Azure

for Students, el coste real aplicado al proyecto ha sido de 0 €, lo que supone un ahorro aproximado de 216 € durante los cuatro meses de desarrollo.

Cálculo de costes

Aplicando las premisas anteriores a las horas reales de trabajo:

Concepto de gasto	Coste planificado (364 h)	Coste Real (384 h)
Coste personal (Horas × 31) €	11.284,00 €	11.904,00 €
CAPEX (Amortización de equipo)	102,00 €	102,00 €
OPEX (Licencias e Infraestructura Cloud)	0,00 €*	0,00 €*
COSTE TOTAL DEL PROYECTO	11.386,00 €	12.006,00 €

(Nota: Sin la aplicación de licencias educativas, el OPEX ascendería a 216 € en total).

El coste final de desarrollo del Trabajo de Fin de Grado asciende a 12.006,00 €, lo que supone un incremento de 620,00 € respecto a lo estimado inicialmente, derivado principalmente del aumento de horas en las fases de entrenamiento de modelos y desarrollo.

Reserva de contingencia

Atendiendo a las buenas prácticas en la dirección y gestión de proyectos, no es recomendable establecer un presupuesto cerrado sin margen de maniobra. Por ello, se ha establecido una reserva de contingencia del 10 % sobre el coste total final:

Reserva de contingencia: 1.200,60 €

Por tanto, el presupuesto final estimado con el que se debería dotar al proyecto para garantizar su viabilidad técnica y económica asciende a:

13.206,60 €

5. Justificación y resumen del stack tecnológico

El stack tecnológico de este TFG se ha seleccionado para resolver un problema híbrido: construir una aplicación web completa e integrar un motor de análisis de datos y *Machine Learning* con capacidad de explicación (XAI). Para ello se adopta una arquitectura cliente-servidor desacoplada y un stack mayoritariamente basado en Python, lo que permite unificar el ciclo de vida del dato y su exposición a través de una API web.

A nivel de principios, las decisiones de tecnología se han guiado por: mantenibilidad y modularidad del código, reproducibilidad del entorno, viabilidad en el marco temporal de un TFG y escalabilidad razonable. En conjunto, el stack se entiende como el conjunto de lenguajes, frameworks, bases de datos y herramientas que hacen posible el desarrollo, ejecución y operación del sistema.

5.1 Visión general de la arquitectura del sistema

El sistema se basa en una arquitectura cliente-servidor desacoplada, estableciendo una separación estricta entre *frontend*, *backend/API* y motor de procesamiento de datos. Esta separación mejora la organización del código y favorece la escalabilidad y mantenibilidad: cada capa expone contratos claros y puede evolucionar con menor riesgo de efectos colaterales.

Capas principales:

- **Capa de presentación (*Frontend*):** desarrollada con *React* y *Vite*. Se encarga de la interfaz de usuario, la visualización de datos y la interacción con el usuario. *Vite* se utiliza como servidor de desarrollo y herramienta de build; proporciona un flujo moderno con recarga rápida y empaquetado para producción.
- **Capa de aplicación / *API* (*Backend*):** implementada con *Django* 5.1.4 + *Django REST Framework* (DRF). Esta capa encapsula la lógica de negocio, la autenticación y autorización, la validación de datos, y actúa como puente entre la UI y la base de datos/motor de ML. DRF se adopta por su enfoque orientado a construir APIs, reduciendo boilerplate y estandarizando el diseño de endpoints.
- **Capa de procesamiento:** motor especializado en *Python* que integra *Pandas/NumPy*, librerías de ML (*scikit-learn*, *XGBoost*, *LightGBM*) y de explicabilidad (*SHAP*). Esta capa puede ejecutarse como módulo interno del backend o

como componente independiente invocado por tareas programadas, dependiendo del caso de uso (entrenamiento offline vs predicción online).

- **Capa de persistencia:** gestión de datos relacionales mediante *PostgreSQL* (entorno de producción) y *SQLite* (entorno de desarrollo). Para producción se prioriza *PostgreSQL* por su robustez y su soporte de transacciones (con propiedades ACID), importante para consistencia bajo concurrencia y operaciones críticas. Además, se integra Redis como soporte de baja latencia y capa de caché.

Comunicación entre capas

- *Frontend* ↔ *Backend/API*: comunicación *HTTP* usando *JSON* como formato principal.
- *Backend* ↔ Persistencia: acceso mediante el ORM de *Django*, que traduce modelos del dominio a tablas relacionales, y permite migraciones y control de integridad de forma ordenada.
- *Backend/API* ↔ Motor ML: integración directa en *Python* para minimizar fricción. En ejecución, esto permite que endpoints de predicción llamen a un pipeline previamente entrenado y serializado, y devuelvan tanto la predicción como elementos de explicación (por ejemplo, top-features relevantes) cuando proceda.

Alternativas arquitectónicas consideradas

Monolito (UI con templates servidor + lógica + procesamiento en una única capa): se descartó por el acoplamiento y por dificultar una UI rica y dinámica (dashboards, estados complejos) sin mezclar responsabilidades. En un proyecto donde el motor de datos/ML evoluciona de forma distinta a la UI, la separación por capas reduce deuda técnica y permite trabajar mejor.

Microservicios: aunque es un patrón habitual en sistemas a gran escala, se descartó por introducir complejidad operativa no necesaria para el alcance del TFG (orquestración, observabilidad distribuida, latencias de red, despliegues coordinados). Para los objetivos académicos, la arquitectura desacoplada por capas aporta muchas ventajas sin el coste de operación de múltiples servicios.

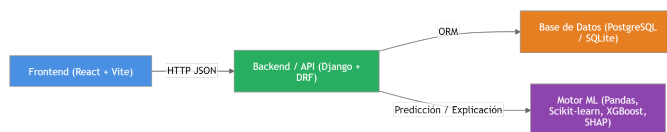


Figura 8: Diagrama de resumen del stack tecnológico.

5.2 Elección del lenguaje de programación: Python

La decisión de emplear *Python* como lenguaje principal se fundamenta en razones técnicas y de eficiencia en el desarrollo: el proyecto combina desarrollo web y un componente significativo de análisis de datos y *Machine Learning*. En este contexto, *Python* destaca por su madurez en el ecosistema de ciencia de datos y por permitir construir el motor predictivo y su integración con la API dentro del mismo entorno.

Además, la elección se apoya en la formación previa: haber trabajado extensamente con Python en el grado reduce la curva de aprendizaje y el riesgo técnico, lo cual es relevante en un TFG con restricciones de tiempo. Esta continuidad facilita también la reproducibilidad (mismo gestor de dependencias y mismas convenciones) y reduce la fragmentación que aparece cuando se mezclan lenguajes para piezas íntimamente conectadas.

Alternativas consideradas y justificación

Se analizaron alternativas como Java y JavaScript. Java, especialmente mediante el uso del framework Spring Boot, ofrece un entorno empresarial robusto y alto rendimiento, pero presenta mayores barreras para integrar flujos completos de Machine Learning, ya que la mayoría de herramientas punteras del sector están diseñadas prioritariamente para Python.

Node.js habría permitido unificar el lenguaje en *frontend* y *backend*, pero carece de un ecosistema maduro en ciencia de datos comparable al de Python. En este proyecto, donde el modelo predictivo es un componente central y no accesorio, elegir Python reduce la fricción técnica y permite desarrollar todo el ciclo de vida del dato dentro del mismo entorno tecnológico, lo que constituye una ventaja estructural significativa.

5.3 Backend: Django 5.1.4 + Django REST Framework

El *backend* se implementa con *Django* 5.1.4 por una combinación de productividad, estructura y seguridad. *Django* proporciona una base sólida para organizar el proyecto (apps, modelos, vistas, etc.), y DRF (*Django REST Framework*) se incorpora para exponer la funcionalidad mediante una *API REST*. Esto encaja con un *frontend* desacoplado que consume endpoints, evitando dependencia de templates del lado servidor.

Un elemento determinante es el ORM. Modelar entidades del dominio (jugadores, equipos, temporadas, partidos, etc.) como modelos declarativos mantiene coherencia con el modelo relacional y facilita la evolución del esquema mediante migraciones. Esto

reduce el uso de SQL manual en consultas repetitivas, disminuye la probabilidad de errores y mantiene un lenguaje común entre lógica de negocio y persistencia.

En cuanto a seguridad, *Django* incorpora protecciones y buenas prácticas documentadas para mitigar riesgos comunes. Por ejemplo, incluye mecanismos integrados para protección contra CSRF (Cross-Site Request Forgery) mediante middleware y herramientas asociadas, especialmente relevante cuando hay sesiones o peticiones que modifican estado. Esta base reduce el riesgo de implementar seguridad “a medida” en un contexto donde lo diferencial del proyecto no es construir un framework de autenticación.

La elección de *Django* 5.1.4 junto a *Django REST Framework* responde a la necesidad de construir un *backend* robusto, seguro y altamente modular. Mientras *Django* proporciona la base estructural (gestión de base de datos, seguridad y administración), DRF se incorpora para transformar estas capacidades en una *API RESTful* que sirve como el núcleo del sistema, promoviendo las buenas prácticas en el desarrollo software.

Además, el sistema de autenticación y autorización integrado de Django (usuarios, permisos, grupos) permite gestionar control de acceso con componentes maduros. En un proyecto académico, esto aporta valor porque focaliza el esfuerzo en el dominio deportivo y el componente predictivo.

Alternativas consideradas y justificación

Flask: ofrece flexibilidad, pero obliga a ensamblar manualmente múltiples piezas (auth, estructura, admin, convenciones), lo que aumenta el tiempo de desarrollo y la probabilidad de inconsistencias.

FastAPI: destaca por rendimiento, tipado y documentación automática; aun así, *Django* aporta una solución más integral cuando hay fuerte componente relacional, gestión de usuarios y necesidad de herramientas integradas (admin/ORM/migraciones) con alta madurez.

Spring Boot / Express: la principal desventaja en este proyecto sería la pérdida de integración directa con el stack de ML en Python o la necesidad de introducir servicios adicionales para el motor predictivo.

5.4 Persistencia y base de datos: SQLite y PostgreSQL

Durante la fase de desarrollo se ha utilizado SQLite por su simplicidad operativa: permite comenzar a trabajar con rapidez, no requiere desplegar servicios adicionales y se integra de forma inmediata con Django, lo que resulta especialmente útil en etapas

tempranas de prototipado y pruebas. No obstante, para el entorno de producción se realizará la migración a PostgreSQL, con el objetivo de disponer de una solución más robusta y preparada para crecer en volumen de datos y número de usuarios.

PostgreSQL es un sistema gestor de bases de datos relacional ampliamente adoptado en entornos profesionales y considerado una tecnología muy estandarizada, lo que facilita su integración con herramientas del ecosistema y reduce riesgos de mantenimiento a largo plazo. Además, ofrece garantías transaccionales ACID, aportando fiabilidad en operaciones críticas y consistencia del dato incluso ante fallos o ejecuciones concurrentes.

PostgreSQL también destaca por su buen comportamiento en escenarios con acceso simultáneo a datos, algo habitual cuando el sistema empieza a recibir varias peticiones a la vez (consultas desde la UI, escrituras de nuevas predicciones, etc.). Esta capacidad de gestionar concurrencia de forma eficiente permite mantener una experiencia de uso más estable y reduce problemas derivados de bloqueos y esperas en la base de datos conforme aumenta la carga. Por ello, la migración a PostgreSQL se plantea como una elección orientada a producción: una base de datos relacional madura y estandarizada, preparada para soportar crecimiento y operaciones críticas con garantías consistentes.

Como complemento indispensable, se incorpora **Redis** como almacén de datos clave-valor en memoria. Su elección se justifica por dos necesidades estructurales:

- **Soporte para Django Channels:** Redis funciona como la *channel layer* necesaria para gestionar la comunicación asíncrona de los *WebSockets*, permitiendo el envío de notificaciones en tiempo real (eventos de partidos o interacciones sociales).
- **Capa de caché:** se emplea para almacenar temporalmente los resultados de cálculos estadísticos complejos (percentiles y rankings). Al servir estos datos desde la memoria RAM, se evita la recomputación constante y se optimiza el rendimiento de la API bajo concurrencia.

Alternativas consideradas y justificación

Se consideraron alternativas como MySQL o bases de datos NoSQL. MySQL ofrece prestaciones similares; sin embargo, PostgreSQL destaca por su mayor cumplimiento de los estándares SQL y por sus capacidades avanzadas de indexación y gestión de consultas complejas. Por su parte, soluciones NoSQL como MongoDB podrían haber ofrecido un mayor rendimiento en determinados escenarios de alta escalabilidad o grandes volúmenes de datos no estructurados. No obstante, su adopción implicaba una curva de aprendizaje adicional y un cambio en el modelo de diseño orientado a documentos que no resultaba óptimo para un sistema con fuerte componente relacional

y consultas estructuradas complejas.

En este contexto, la elección de PostgreSQL permitió mantener coherencia con el modelo de datos planteado, reducir la complejidad técnica y optimizar el tiempo de desarrollo sin comprometer el rendimiento necesario para el proyecto.

5.5 Procesamiento de datos y Machine Learning

El proyecto incorpora un *pipeline* de datos completo que no se limita a entrenar un modelo: incluye ingesta, limpieza, construcción de features, validación, evaluación e interpretabilidad.

5.5.1 Preprocesado y ETL (Extract, Transform and Load)

Se emplean Pandas y NumPy para:

- Limpieza y normalización (tipos, formatos de fecha, estandarización de nombres).
- Tratamiento de nulos y outliers.
- Codificación de variables categóricas y construcción de datasets finales.
- Generación de *features* a partir de histórico (medias móviles, acumulados, forma reciente, local/visitante, etc.), cuidando que las *features* respeten la causalidad temporal (evitar *data leakage*).

El resultado del ETL es un dataset donde filas representan un hecho (p. ej., equipo en jornada, jugador en partido) y columnas representan variables. Esto se conoce como enfoque tabular y es coherente con la mayoría de datos deportivos agregados, donde la señal predictiva suele estar en combinaciones de variables independientes.

5.5.2 Modelado (scikit-learn, XGBoost, LightGBM)

Se utiliza *scikit-learn* para evaluación y validación (splits, métricas, pipelines y comparación de enfoques). Además, se exploran modelos basados en árboles y gradient boosting (XGBoost y LightGBM), especialmente competitivos en dominios tabulares por su capacidad de capturar no linealidades e interacciones con buen coste computacional.

La elección de boosting se justifica por:

- Buen rendimiento en tabular con variables heterogéneas.
- Estabilidad razonable con datasets medianos.
- Menor coste de ingeniería comparado con deep learning para este tipo de dato.

5.5.3 Interpretabilidad (XAI con *SHAP*)

La interpretabilidad se aborda con *SHAP* para generar explicaciones:

- **Globales:** identificar qué variables influyen más de forma general en el modelo.
- **Locales:** justificar una predicción individual descomponiéndola en contribuciones por *feature*.

Esto añade valor académico y práctico: el sistema no se limita a una salida numérica, sino que ofrece una narrativa técnica de “por qué” se produce la predicción, reduciendo el carácter de caja negra y facilitando el análisis de errores.

Alternativas consideradas y justificación

Se evalúa *deep learning* (TensorFlow/PyTorch), pero para datos tabulares suele requerir más ajuste, más datos y mayor coste de entrenamiento para obtener mejoras no garantizadas frente a boosting. Dado el alcance del TFG, se prioriza un enfoque más eficiente y explicable.

5.5.4 Serialización e integración del modelo predictivo

Para materializar la conexión entre la fase de entrenamiento y el entorno de operación, se hace uso del módulo pickle de Python. Una vez que el *pipeline* de preprocesamiento y los modelos predictivos han sido entrenados y validados, el objeto resultante se serializa y se exporta como un archivo binario (.pkl), con la tecnología Pickle.

En tiempo de ejecución, el backend de Django carga este artefacto en memoria al inicializar la aplicación o al instanciar la clase predictiva. Esta estrategia permite que los endpoints de la API realicen predicciones invocando al modelo ya ajustado, sin necesidad de retener en el servidor los datos de entrenamiento ni recalcular pesos. De esta forma, se garantiza una latencia baja en la respuesta HTTP y se mantiene una separación estricta entre el ciclo de entrenamiento (offline) y el de inferencia (online).

5.6 Obtención de datos y scraping

El proyecto requiere recopilar información deportiva desde fuentes heterogéneas, por lo que se adopta una estrategia híbrida de ingesta. En la práctica, muchas fuentes presentan limitaciones: APIs incompletas, restricciones de uso, cambios de condiciones o modelos de pago, por lo que combinar técnicas permite maximizar cobertura manteniendo viabilidad.

5.6.1 Descarga de CSVs con cuotas

Una fuente clave de cuotas de apuestas son los ficheros CSV públicos de *Football-Data.co.uk* (SP1.csv), que ofrece enlaces de descarga histórica para España. Estos CSVs se integran en el pipeline como datasets de entrada: se descargan y se normalizan para cruzarlos con el resto de tablas del dominio.

La inclusión de datos externos de contexto se consideraba algo fundamental en el proyecto y su obtención en fases iniciales fue escasa. En esa etapa se observó una diferencia aproximada de 0.5 en el MAE, en función de si se tenían o no datos relacionados con las apuestas.

5.6.2 Consumo de APIs y evaluación de librerías

A lo largo del desarrollo se exploran distintas APIs/librerías de datos deportivos (por ejemplo *statsbomb*, *soccerdata*), finalmente descartadas por limitaciones prácticas (cobertura, estabilidad, compatibilidad con el dataset objetivo). Este trabajo forma parte de la ingeniería de datos: evaluar fuentes, comparar consistencia temporal y decidir qué fuentes cumplen requisitos mínimos (completitud, actualización, integrabilidad).

5.6.3 Scraping: requests + parsing HTML

Dadas las limitaciones con las APIs, en la mayoría de casos se recurre a *scraping* con requests y parsing con BeautifulSoup4 con lxml. Esta decisión permite mantener viabilidad económica y cobertura de datos, asumiendo el coste técnico de:

- Normalizar formatos heterogéneos.
- Diseñar extractores robustos.
- Mitigar roturas ante cambios de HTML.

5.7 *Frontend con React y Vite*

La capa de presentación se desarrolla con *React* por su enfoque basado en componentes, idóneo para dashboards y vistas con alta reutilización (tarjetas de jugadores, tablas, filtros, gráficos, paneles de explicación). Se utiliza *Vite* como herramienta de build y servidor de desarrollo, mejorando significativamente la experiencia de desarrollo en comparación con herramientas tradicionales más pesadas.

5.7.1 Desacoplamiento respecto al servidor

Inicialmente se valora usar templates del lado servidor (*Django templates*), lo cual habría simplificado la arquitectura a corto plazo. Sin embargo, se opta por un *frontend* desacoplado para mantener separación de responsabilidades y permitir una evolución más flexible: rutas del cliente, estados complejos y consumo de API de forma limpia.

5.7.2 Base estructural: adaptación de *React Petclinic*

El proyecto toma como base estructural *React Petclinic*, del cual se elimina toda la interfaz y entidades, manteniendo únicamente el “esqueleto funcional” para adaptarlo al dominio deportivo. Esta decisión acelera la construcción inicial (estructura de componentes, navegación, etc.) y permite centrar más esfuerzo en las partes diferenciales (datos, predicción, explicaciones y UX de análisis).

Alternativas consideradas y justificación

Angular: muy completo y estructurado, pero con mayor curva de aprendizaje y sobrecarga conceptual para el alcance del TFG.

Vue: productivo y simple, pero *React* ofrece adopción industrial muy extendida y un ecosistema enorme para visualización y gestión de estado.

Templates del servidor: más sencillo al principio, pero limita escalabilidad de *UI* y mezcla responsabilidades, complicando la evolución hacia un cliente rico.

5.8 *APIs y otras herramientas de soporte*

Además de la *API* principal (DRF), se incorporan herramientas complementarias para resolver necesidades de producto y de explotación del dato:

Fuzzy matching: se incorpora para resolver pequeñas variaciones textuales (nombre real vs apodos, diferencias ortográficas) al conciliar entidades entre fuentes. Esto es habitual cuando se combinan datasets externos y *scraping*.

Meilisearch: se emplea como motor de búsqueda para habilitar consultas rápidas sobre campos de texto (nombres de jugadores/equipos). Se valoró también el uso de ElasticSearch y OpenSearch, pero fueron descartados por motivos técnicos.

Este conjunto de herramientas se justifica por un objetivo práctico: mejorar la experiencia de usuario (búsquedas rápidas, navegación fluida) y reducir fricción en la integración de datos (matching y normalización).

5.9 Entorno de desarrollo, despliegue y calidad de software

El proyecto adopta herramientas estándar para garantizar reproducibilidad, facilitar el desarrollo y sostener calidad de software. Aunque el alcance sea académico, estas decisiones aproximan el trabajo a prácticas profesionales y reducen riesgos técnicos.

5.9.1 Gestión de entornos virtuales y dependencias

Dado que el proyecto utiliza una gran cantidad de librerías con versiones específicas (especialmente en el stack de *Machine Learning*), se ha optado por el uso de entornos virtuales mediante venv (Python Virtual Environments). Lo que resulta fundamental en varios aspectos:

Aislamiento: esta técnica permite que las dependencias del proyecto residan en un directorio local, evitando conflictos con las librerías globales del sistema operativo.

Pip y requirements.txt: la gestión de paquetes se realiza a través de pip. Para asegurar que el motor de ML y el *backend* de *Django* funcionen correctamente en cualquier máquina, se mantiene un archivo requirements.txt actualizado. Este archivo actúa como el “contrato de dependencias” del sistema, permitiendo la instalación masiva de las versiones exactas necesarias mediante un solo comando. Puede ser generado automáticamente con diferentes herramientas, lo que disminuye en gran medida la complejidad de su elaboración.

5.9.2 Contenedores

Se ha implementado una arquitectura basada en contenedores utilizando *Docker*, lo que permite aislar cada componente del sistema en un entorno controlado. Para la

gestión de estos servicios, se emplea *Docker Compose*, actuando como el orquestador que define y arranca los cuatro nodos principales que conforman la aplicación:

- **Backend (*API*):** Contenedor basado en *Python* 3.12 que aloja la lógica de *Django* y el motor de *Machine Learning*.
- **Frontend (*UI*):** Contenedor que sirve la aplicación *React*, optimizada tras el proceso de *build* de *Vite*.
- **Capa de mensajería y caché (*Redis*):** Nodo independiente basado en *redis:7-alpine* para el soporte de tiempo real y optimización de lecturas de la *API*.
- **Persistencia (*PostgreSQL*):** Servicio dedicado para la base de datos relacional, asegurando el almacenamiento persistente mediante volúmenes de Docker.
- **Motor de búsqueda (*Meilisearch*):** Nodo independiente para la indexación y búsqueda avanzada de datos, garantizando que los procesos de búsqueda intensiva no penalicen el rendimiento de la base de datos principal o no afecte al no estar disponible.

Ventajas de este enfoque:

- **Consistencia entre entornos:** se elimina el problema de “funciona en mi máquina”, ya que el entorno de desarrollo es idéntico al de producción.
- **Aislamiento de dependencias:** cada servicio tiene sus propias librerías y configuraciones (por ejemplo, el stack de ML en el backend no interfiere con el motor de búsqueda).
- **Escalabilidad y portabilidad:** la arquitectura está preparada para ser desplegada en cualquier proveedor de nube (AWS, Azure, Google Cloud, etc.) mediante un único comando, lo que reduce drásticamente la complejidad operativa.

5.9.3 Control de versiones, planificación y control de calidad

Control de versiones: GitHub.

Planificación temporal: Microsoft Project.

Registro de horas: Clockify.

Calidad: SonarQube para análisis estático.

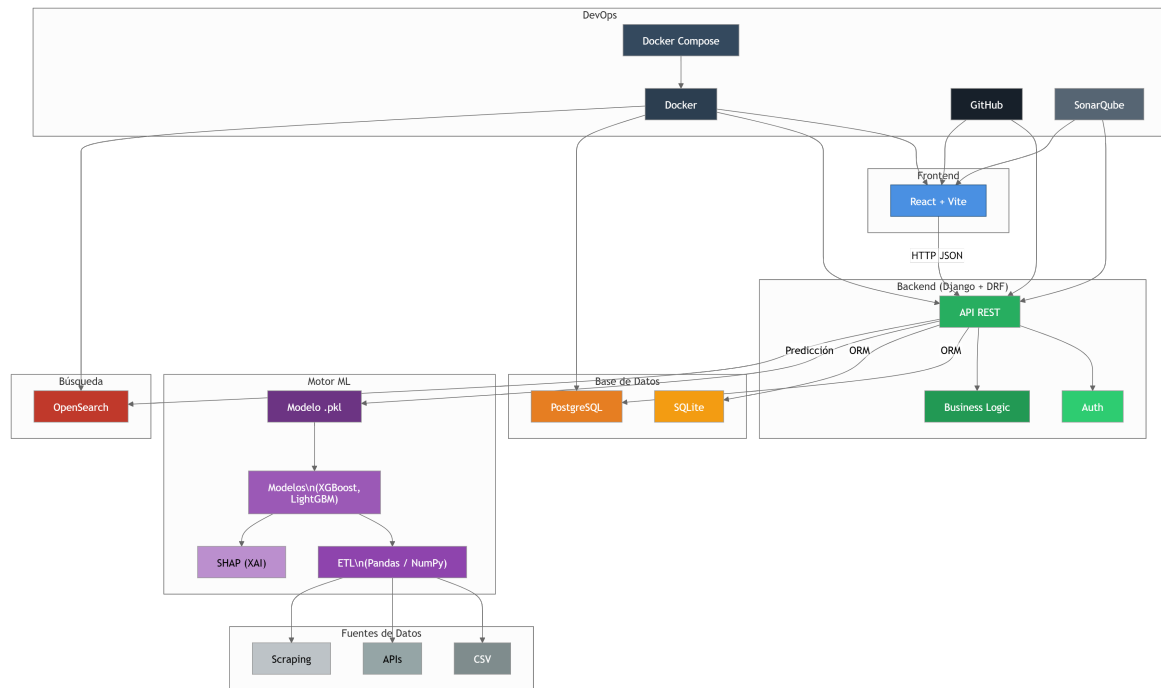


Figura 9: Estructura del sistema y comunicación entre sus capas.

6. Requisitos

6.1 Alcance detallado

El sistema a desarrollar es una plataforma web para Fantasy Football orientada al análisis predictivo del rendimiento de jugadores y a la recomendación de decisiones de gestión, accesible desde navegador y con un modelo de acceso progresivo (funcionalidades públicas y funcionalidades avanzadas para usuarios autenticados). En concreto, el alcance incluye:

- Visualización de información pública de competición: clasificación, equipos, plantillas y vistas detalladas de equipos y jugadores (estadísticas históricas y rendimiento reciente).
- Predicción de puntuación de jugadores para la próxima jornada, incluyendo presentación de un margen de error y explicación de los factores más influyentes (XAI).
- Recomendaciones automáticas para apoyar decisiones de mánager: recomendaciones por posición (“Top 3”), asistente de gestión (vender/fichar) y panel de análisis del próximo rival.
- Funcionalidades para usuarios registrados: creación de cuenta, autenticación, gestión de perfil y persistencia de una plantilla personalizada con formación, proyección de puntuación y alertas básicas.
- Acceso a histórico y simulación/retro-evaluación (backtesting funcional) para comparar predicción vs resultado en jornadas pasadas.

6.2 Fuera de alcance

Para acotar la complejidad, se consideran fuera de alcance en esta versión del proyecto:

- Integración oficial con LaLiga Fantasy (login, mercado real, transacciones reales o sincronización automática) ya que no existe API pública estable.
- Predicción de evolución del valor de mercado (ya identificado como limitación por ausencia de histórico consistente).
- Modelado explícito de lesiones/rotaciones/rumores en tiempo real (se tratará como limitación del modelo y del dominio).

- Notificaciones push móviles y apps móvil dedicada.

6.3 Suposiciones, dependencias y restricciones

- Dependencia de disponibilidad y estabilidad de fuentes externas (APIs/HTML/CSV); cambios de formato pueden requerir adaptación del pipeline.
- Restricción de idioma: interfaz y mensajes en castellano (RNF1).
- Restricción de rendimiento: objetivo de tiempos de respuesta (RNF3) sujeto a condiciones de prueba controladas.
- Se considerará que todos los jugadores están disponibles y en forma durante toda la temporada.

6.4 Actores

En el modelo de casos de uso de la plataforma se consideran únicamente actores humanos, diferenciando entre actores primarios (persiguen un objetivo directo con el sistema) y actores de soporte (interactúan para administración/supervisión).

6.4.1 Usuario anónimo

Objetivo: Consultar información pública y explorar jugadores y equipos sin crear una cuenta.

Interacciones típicas:

- Ver clasificación y páginas públicas de equipos/jugadores (RF1, RF2, RF3).
- Buscar por nombre y aplicar filtros básicos (RF4).
- Consultar fichas y paneles informativos accesibles sin autenticación (RF5).

6.4.2 Usuario registrado (Mánager)

Objetivo: Gestionar su plantilla y tomar decisiones apoyadas por predicciones, recomendaciones y explicaciones.

Interacciones típicas:

- Registrarse, iniciar/cerrar sesión y gestionar su cuenta (RF6).
- Crear/editar su plantilla y formación; ver proyección agregada (RF7, RF8).
- Consultar predicciones con explicación (XAI) (RF8, RF9).
- Consultar recomendaciones por posición y asistente de gestión (RF10, RF11).
- Acceder a histórico y simulación en jornadas pasadas (RF12).
- Configurar visibilidad de plantilla y usar funcionalidades sociales (RF13, RF14).
- Recibir notificaciones/alertas (RF15).

6.5 Requisitos funcionales

Para la priorización de los requisitos del sistema se utilizará el método *MoSCoW*, una técnica ampliamente empleada en la gestión de proyectos que permite clasificar los requisitos en función de su importancia y necesidad.

ID	Descripción	Historia de usuario
RF1	Visualización de clasificación	Como: Usuario anónimo Quiero: Ver la clasificación de la liga y su evolución por jornada Para: Entender el estado actual de la competición sin salir de la aplicación
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Must Have	CA1.1. La clasificación muestra posición, nombre, puntos, racha reciente, goles a favor, en contra y diferencia de goles. CA1.2. Al menos el 95 % de los datos están disponibles y correctos.

ID	Descripción	Historia de usuario
RF2	Ficha de equipo	Como: Usuario anónimo Quiero: Ver la plantilla y detalles de un equipo Para: Conocer su estado actual
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Must Have	CA2.1. Acceso desde clasificación o buscador. CA2.2. Muestra plantilla con enlaces a fichas de jugadores. CA2.3. Muestra próximo rival y panel de análisis.

ID	Descripción	Historia de usuario
RF3	Ficha de jugador	Como: Usuario anónimo Quiero: Ver información detallada de un jugador Para: Analizar su rendimiento
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Must Have	CA3.1. Muestra estadísticas históricas del jugador. CA3.2. Incluye datos de temporadas anteriores. CA3.3. Incluye puntos acumulados de la temporada actual.

ID	Descripción	Historia de usuario
RF4	Buscador de equipos y jugadores	Como: Usuario anónimo Quiero: Buscar jugadores o equipos rápidamente Para: Encontrarlos sin navegar manualmente
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Must Have	CA4.1. Permite búsqueda por texto parcial. CA4.2. Ofrece autocompletado a partir de 3 caracteres.

ID	Descripción	Historia de usuario
RF5	Panel próximo rival	Como: Usuario anónimo Quiero: Ver el análisis del próximo rival Para: Entender el contexto táctico del partido
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Should Have	CA5.1. Incluye métricas de últimas 5 jornadas (GF, GC, racha). CA5.2. Incluye histórico de enfrentamientos previos (H2H).

ID	Descripción	Historia de usuario
RF6	Registro e inicio/cierre sesión	Como: Usuario anónimo Quiero: Crear una cuenta e iniciar sesión Para: Acceder a funcionalidades personalizadas
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Must Have	CA6.1. Valida correctamente los campos. CA6.2. Muestra mensajes de error informativos. CA6.3. Permite eliminar cuenta de usuario.

ID	Descripción	Historia de usuario
RF7	Gestión de fantasy	Como: Usuario registrado Quiero: Gestionar mi plantilla de Fantasy Para: Hacer seguimiento estratégico de mis jugadores
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Must Have	CA7.1. Permite añadir y eliminar jugadores de plantilla. CA7.2. Mantiene persistencia de cambios. CA7.3. Ofrece interfaz familiar y accesible. CA7.4. Soporta múltiples formaciones.

ID	Descripción	Historia de usuario
RF8	Gestión de plantillas	Como: Usuario registrado Quiero: Gestionar múltiples plantillas Para: Organizar escenarios y estrategias
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Must Have	CA8.1. Permite crear nuevas plantillas. CA8.2. Permite renombrar plantillas existentes. CA8.3. Permite definir plantilla predeterminada.

ID	Descripción	Historia de usuario
RF9	Predicción de puntos de cada jugador	Como: Usuario registrado Quiero: Ver los puntos predichos de mi plantilla Para: Decidir mi alineación óptima
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Must Have	CA9.1. Muestra puntos predichos en Mi Plantilla. CA9.2. Permite seleccionar jornada desde la interfaz. CA9.3. Informa motivo si no hay predicción. CA9.4. Muestra proyección total de plantilla.

ID	Descripción	Historia de usuario
RF10	Explicabilidad de las predicciones	Como: Usuario registrado Quiero: Entender la predicción de mis jugadores Para: Aumentar mi confianza en el sistema
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Must Have	CA10.1. Muestra top 3 <i>features</i> más influyentes. CA10.2. Describe contribución de cada <i>feature</i> . CA10.3. Muestra aviso si la explicación no está disponible.

ID	Descripción	Historia de usuario
RF11	Recomendaciones por posición (Top 3)	Como: Usuario registrado Quiero: Ver candidatos destacados por posición Para: Tomar mejores decisiones de fichaje
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Should Have	CA11.1. Muestra top 3 jugadores por posición. CA11.2. Incluye explicación de la recomendación. CA11.3. Incluye acceso directo a ficha del jugador.

ID	Descripción	Historia de usuario
RF12	Asistente de gestión	Como: Usuario registrado Quiero: Recibir consejo sobre decisiones de plantilla Para: Optimizar rendimiento en la liga
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA12.1. Permite seleccionar jugador para análisis. CA12.2. Recomienda acción (vender/fichar). CA12.3. Incluye comparativa y explicación completa.

ID	Descripción	Historia de usuario
RF13	Histórico de predicciones	Como: Usuario registrado Quiero: Revisar predicciones pasadas Para: Medir la fiabilidad del sistema
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA13.1. Incluye tabla predicho vs real.

ID	Descripción	Historia de usuario
RF14	Sistema de amistad	Como: Usuario registrado Quiero: Enviar, recibir y rechazar solicitudes de amistad Para: Conectar y comparar equipos
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA14.1. Permite envío por nombre o apodo. CA14.2. Permite aceptar o rechazar solicitudes. CA14.3. Evita solicitudes duplicadas activas.

ID	Descripción	Historia de usuario
RF15	Visibilidad de plantilla	Como: Usuario registrado Quiero: Configurar la visibilidad de mi plantilla Para: Proteger mi estrategia
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA15.1. Permite marcar plantilla como pública o privada. CA15.2. Privada: solo propietario accede. CA15.3. Pública: solo amigos acceden.

ID	Descripción	Historia de usuario
RF16	Sistema de notificaciones	Como: Usuario registrado Quiero: Recibir notificaciones de eventos relevantes Para: Estar siempre al día
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA16.1. Muestra notificaciones en bandeja. CA16.2. Permite marcar notificaciones como leídas.

ID	Descripción	Historia de usuario
RF17	Partidos de la jornada	Como: Usuario anónimo Quiero: Ver los partidos de cada jornada Para: Consultar fácilmente el calendario
	Información del Requisito	Criterios de aceptación
	Actor: Usuario anónimo Auth: No Prioridad: Should Have	CA17.1. Muestra partidos de la jornada seleccionada. CA17.2. Muestra escudos, horarios y días. CA17.3. Muestra sucesos del partido si ya se jugó.

ID	Descripción	Historia de usuario
RF18	Personalización de perfil	Como: Usuario registrado Quiero: Personalizar mi perfil Para: Reflejar mi identidad y preferencias
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA18.1. Permite foto de perfil personalizada. CA18.2. Permite seleccionar equipos favoritos. CA18.3. Permite estado personalizado visible.

ID	Descripción	Historia de usuario
RF19	Comparativa de jugadores	Como: Usuario registrado Quiero: Comparar jugadores en diferentes temporadas Para: Tomar mejores decisiones de fichaje
	Información del Requisito	Criterios de aceptación
	Actor: Usuario registrado Auth: Sí Prioridad: Could Have	CA19.1. Permite seleccionar múltiples jugadores. CA19.2. Permite elegir temporadas a comparar. CA19.3. Muestra estadísticas normalizadas por 90 minutos.

6.6 Requisitos no funcionales

RNF1. Rendimiento y tiempos de respuesta

RNF1.1: Consultas estándar en < 500 ms.

RNF1.2: Predicción/XAI en < 2 s con modelo precargado.

RNF2. Usabilidad e idioma

RNF2.1: Interfaz íntegramente en castellano, incluyendo errores.

RNF3. Calidad del dato y anti data leakage

RNF3.1: Features calculadas solo con información anterior al evento predicho.

RNF4. Soporte y explicabilidad para el usuario

RNF4.1: Glosario accesible de features/estadísticas.

RNF4.2: Nombres legibles en explicaciones con indicación positiva/negativa.

RNF4.3: Acceso visible a ayuda e tooltips en pantallas clave.

RNF5. Seguridad y privacidad

RNF5.1: Contraseñas con hash seguro + salt (mecanismo del framework).

RNF5.2: Sesión cerrada sin acceso a recursos privados sin reautenticación.

RNF5.3: Minimización de datos personales (solo username como identificador público).

6.7 Casos de uso

Los casos de uso se modelan considerando dos perfiles: usuario anónimo y usuario registrado. Se presentan mediante diagramas que ilustran las interacciones entre actores y sistema.

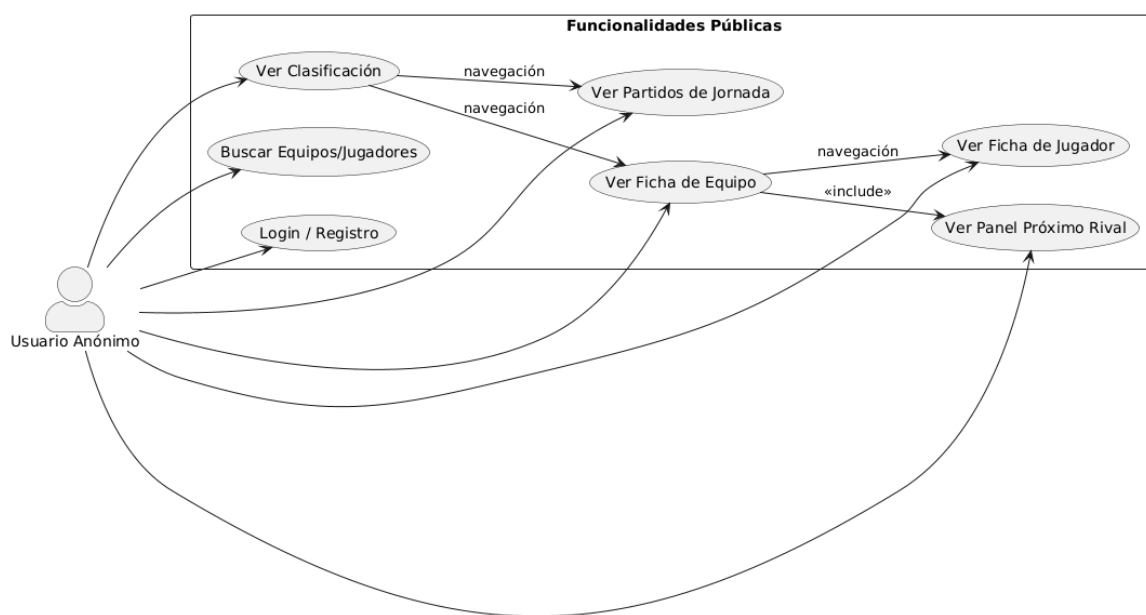


Figura 10: Casos de uso para Usuario Anónimo.

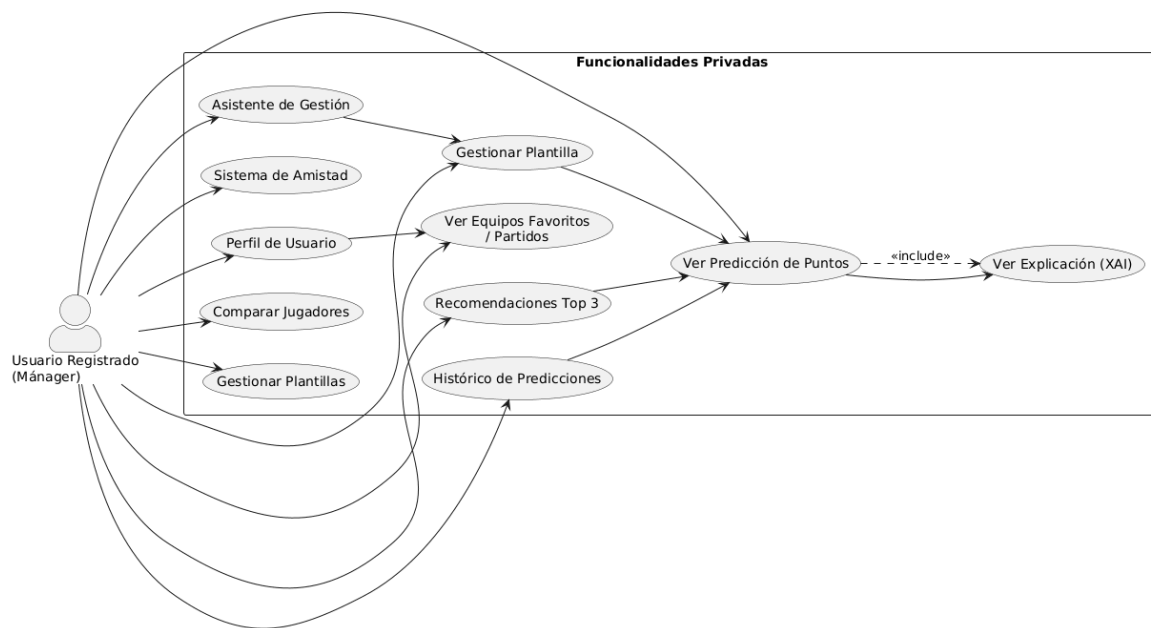


Figura 11: CCasos de uso para Usuario Registrado.

ID: CU1 - Consultar clasificación de LaLiga	
Actores: Usuario anónimo	Precondiciones: Ninguna
RF Asociados: RF1	Postcondiciones: Se muestra la clasificación.
Flujo principal 1. El usuario accede a “Liga”. 2. El sistema carga la jornada por defecto (actual). 3. El sistema muestra equipos, posición, puntos, racha, GF, GC... 4. (Opcional) El usuario cambia la jornada. 5. El sistema refresca la tabla.	Flujo alternativo/excepciones A1. El usuario navega a una jornada en el futuro, en dicho caso, se le informa que no hay clasificación disponible dado que es el futuro.

ID: CU2 - Consultar ficha de equipo	
Actores: Usuario anónimo	Precondiciones: Ninguna
RF Asociados: RF2, RF5	Postcondiciones: Se muestra la ficha del equipo y el panel de su próximo rival.
Flujo principal 1. El usuario selecciona un equipo desde clasificación o buscador. 2. El sistema abre la ficha del equipo. 3. El sistema muestra plantilla (jugadores + posiciones) y enlaces a fichas individuales. 4. El sistema muestra próximo rival. 5. El usuario abre el análisis del rival. 6. El sistema muestra métricas del rival y H2H.	Flujo alternativo/excepciones - A1. No hay datos de próximo rival: se muestra “Próximo rival no disponible”. - A2. No hay datos H2H: se muestra “Histórico no disponible” sin error.

ID: CU3 - Consultar ficha de jugador	
Actores: Usuario anónimo	Precondiciones: Ninguna
RF Asociados: RF3	Postcondiciones: Se muestra ficha del jugador o estado “No disponible”.
Flujo principal 1. El usuario entra a la ficha del jugador (desde equipo o buscador). 2. El sistema muestra estadísticas históricas y últimas 8 jornadas. 3. El usuario consulta temporadas anteriores (si existen). 4. El sistema muestra los datos adicionales.	Flujo alternativo/excepciones - A1. Sin datos de temporadas anteriores: se oculta sección o muestra “Sin registros”. - A2. Jugador no encontrado: se muestra 404 funcional (“Jugador no encontrado”).

ID: CU4 - Gestionar plantilla y ver predicción con XAI	
Actores: Usuario registrado	Precondiciones: Usuario autenticado.
RF Asociados: RF7, RF8, RF9	Postcondiciones: Plantilla guardada y XAI y predicciones mostradas.
Flujo principal 1. El usuario accede a “Mi plantilla”. 2. El sistema carga la plantilla guardada. 3. El usuario añade/elimina jugadores y elige formación. 4. El sistema guarda cambios. 5. El sistema muestra puntos predichos + incertidumbre y proyección total. 6. El usuario abre el detalle de un jugador. 7. El sistema muestra top 3 features por su contribución.	Flujo alternativo/excepciones - A1. Sin predicción para un jugador: se muestra motivo en ese jugador. - A2. Explicación no disponible: se muestra aviso “Explicación no disponible” manteniendo la predicción. - A3. Si no existen suficientes datos previos simplemente se mostrará la media previa al partido.



Figura 12: Flujo de usuario en el caso de uso 4.

6.8 Prototipado de UI

Para la elaboración de los mockups y la versión inicial de las vistas de la aplicación se utilizó la herramienta de inteligencia artificial Stitch (<https://stitch.withgoogle.com/>). Esta herramienta permite generar interfaces a partir de descripciones textuales detalladas, lo que facilitó la creación de diseños alineados con los requisitos funcionales y la idea conceptual del proyecto.

Se optó por esta solución debido a que, además de generar una imagen en formato PNG, también proporciona automáticamente el código HTML junto con su CSS asociado, lo que supone una ventaja significativa en términos de productividad.

Esta funcionalidad permite avanzar directamente hacia una implementación real de la interfaz, reduciendo considerablemente el esfuerzo necesario en comparación con otras alternativas, como el uso de herramientas de diseño estático (por ejemplo, Canva) o el desarrollo manual desde cero del HTML y CSS. Gracias a este enfoque, se ha conseguido acelerar la fase de prototipado y reducir el tiempo de desarrollo, manteniendo

al mismo tiempo una coherencia entre el diseño inicial y la implementación final de la interfaz.

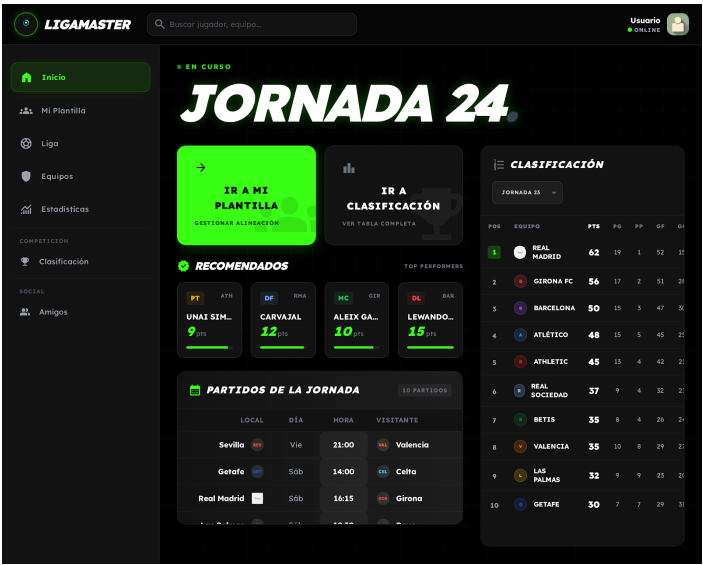


Figura 13: Prototipo del menú principal de la aplicación, con algunas funcionalidades y donde el usuario puede navegar entre las distintas secciones desde la barra lateral.

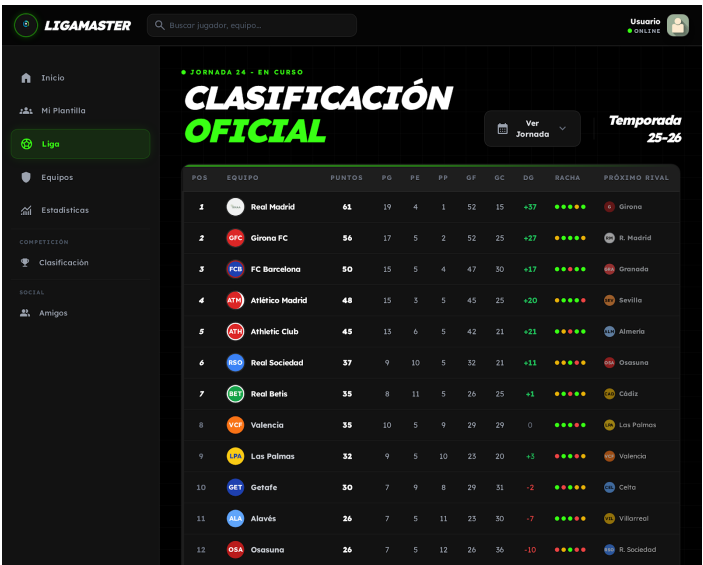


Figura 14: Prototipo de clasificación de LaLiga, mostrando equipos, posiciones y puntos.

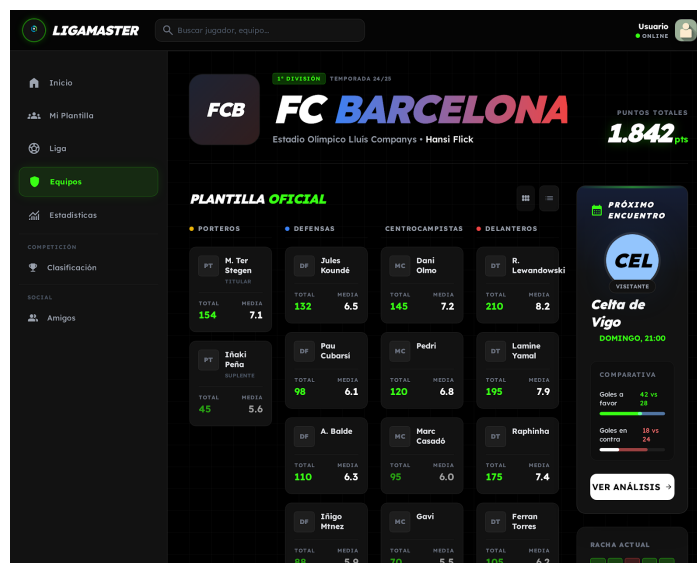


Figura 15: Prototipo de la ficha de equipo, con plantilla, posiciones, próximo partido y acceso a detalles individuales de los jugadores.

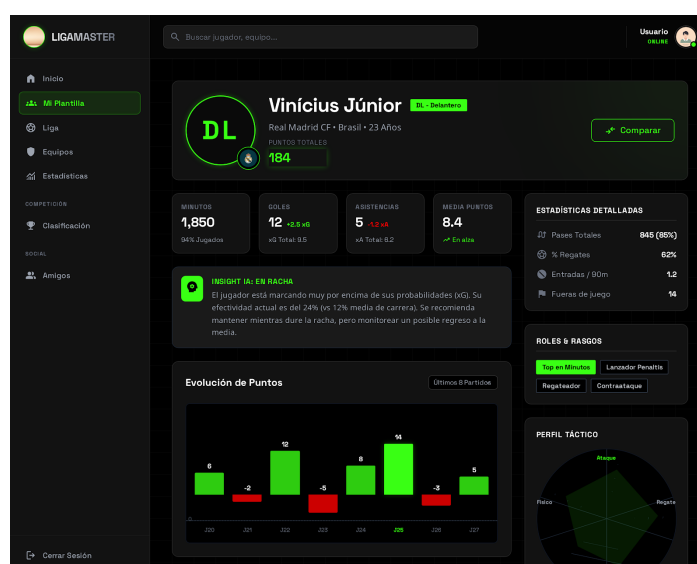


Figura 16: Prototipo de la vista de jugador, mostrando estadísticas históricas y rendimiento reciente.

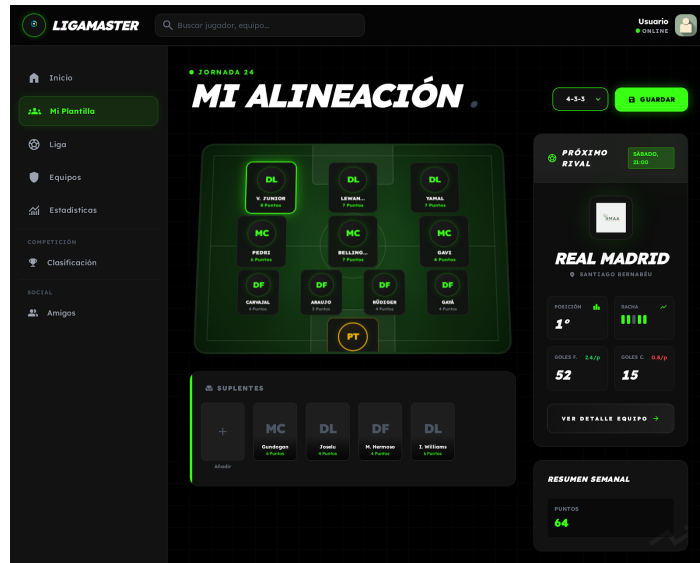


Figura 17: Prototipo de la vista de plantilla de usuario, con predicción de puntos y explicación XAI.

7. Diseño del sistema

El presente capítulo describe el diseño integral del sistema desarrollado, abordando de forma estructurada tanto la arquitectura software como la organización de los datos y la experiencia de usuario. Se parte de una visión global del sistema para, posteriormente, profundizar en los distintos niveles de abstracción, incluyendo la arquitectura, el modelado de datos, los patrones de diseño y la interfaz de usuario.

El objetivo principal de este diseño es garantizar un sistema robusto, escalable y mantenible, capaz de integrar de forma eficiente componentes heterogéneos como servicios web, procesamiento de datos y modelos de Machine Learning. Para ello, se ha seguido un enfoque basado en buenas prácticas de ingeniería del software, priorizando la separación de responsabilidades, la modularidad y la reutilización de componentes.

7.1 Arquitectura global del sistema

La arquitectura del sistema define la organización de sus componentes principales, así como las relaciones e interacciones entre ellos, estableciendo las bases para su correcto funcionamiento y evolución futura. En este proyecto, se ha optado por una arquitectura distribuida que separa claramente las capas de frontend, backend y sistemas de datos, favoreciendo la escalabilidad, flexibilidad y mantenibilidad del sistema.

Esta separación permite desacoplar responsabilidades y facilitar la evolución independiente de cada componente. En particular, la capa de datos queda especializada en tres subsistemas complementarios: persistencia transaccional en PostgreSQL, búsqueda y recuperación documental en MeiliSearch, y almacenamiento en memoria de baja latencia mediante Redis. Esta distribución funcional reduce cuellos de botella en lectura, mejora la capacidad de respuesta del sistema y habilita comunicación asíncrona eficiente para funcionalidades en tiempo real.

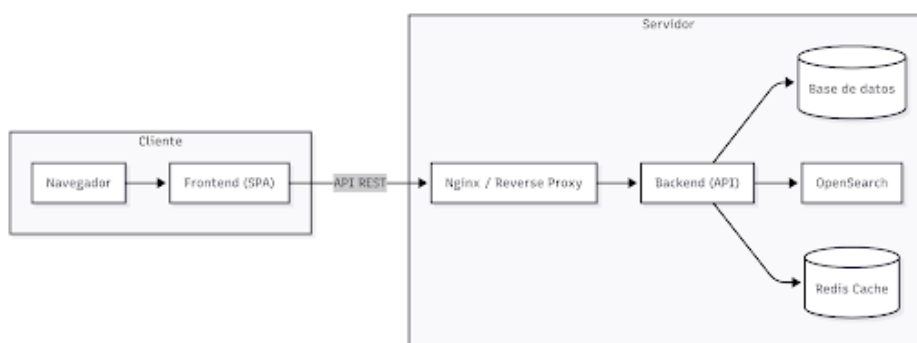


Figura 18: Resumen de alto nivel de la arquitectura.

Este diagrama representa la arquitectura física y lógica del sistema, detallando cómo interactúan los componentes del lado del cliente y del servidor. A continuación, se presenta una explicación técnica y formal de cada bloque, adecuada para la memoria de un TFG.

7.1.1 Bloque de cliente: *frontend*

A diferencia de una arquitectura monolítica tradicional, el cliente actúa como un nodo independiente que se ejecuta en el navegador del usuario.

El *frontend* es la capa de presentación encargada de renderizar la interfaz de usuario. Aunque consume recursos de forma dinámica, su función principal es capturar las interacciones del usuario y presentar los datos procesados (estadísticas, predicciones y gráficos XAI) de manera comprensible. El navegador descarga los activos del frontend y establece una conexión de red con el servidor a través de protocolos estándar.

7.1.2 Punto de entrada: *Caddy*

Caddy se configura como el punto de entrada principal al servidor, actuando como intermediario entre el cliente y los servicios internos. Su incorporación responde a decisiones de diseño centradas tanto en la mejora del rendimiento como en el refuerzo de la seguridad, ya que permite gestionar de forma eficiente el flujo de peticiones que llegan desde el frontend.

En este sentido, *Caddy* funciona como un proxy inverso, recibiendo las solicitudes y redirigiéndolas al backend correspondiente de manera transparente. Además, aporta una capa de aislamiento que protege la infraestructura interna, ocultando la topología de la red y centralizando la gestión de políticas de seguridad, como la configuración de HTTPS/SSL, en un único punto.

7.1.3 Núcleo del sistema: *backend*

El *backend* constituye la capa de aplicación desarrollada en *Django* y actúa como el núcleo que conecta y coordina el resto de servicios del sistema. En él se implementa la lógica de negocio, encargándose del procesamiento de las peticiones, la gestión de la autenticación y la comunicación con el motor de *Machine Learning* para la obtención de predicciones y recomendaciones.

Asimismo, desempeña un papel de intermediación, ya que traduce las solicitudes del usuario en consultas específicas dirigidas a los distintos sistemas de almacenamiento y búsqueda. De este modo, se abstrae la complejidad de los servicios subyacentes y se ofrece una interfaz unificada y coherente al frontend.

La solución propuesta se fundamenta en una arquitectura distribuida, en la que las responsabilidades de presentación, orquestación, lógica de negocio y persistencia se encuentran claramente separadas. Este enfoque permite garantizar propiedades clave como la escalabilidad, mantenibilidad y extensibilidad, aspectos críticos en sistemas que integran componentes web y modelos de Machine Learning.

7.1.4 Patrones empleados

Con el objetivo de estructurar el sistema de forma clara, mantenible y escalable, se han aplicado diversos patrones de diseño arquitectónicos ampliamente utilizados en la ingeniería del software. Estos patrones permiten organizar el código de manera eficiente, reducir el acoplamiento entre componentes y facilitar tanto la evolución como la reutilización del sistema. A continuación, se describen los principales patrones adoptados, junto con su aplicación concreta dentro del proyecto y la justificación de su uso desde un punto de vista técnico.

Modelo MVC / MTV

El sistema adopta el patrón clásico Model-View-Controller (MVC), adaptado al framework *Django* bajo la variante Model-Template-View (MTV). Este patrón establece una separación clara entre la representación de los datos, la lógica de control y la presentación.

Aplicación en el sistema:

- `models.py`: define el modelo de dominio, incluyendo las entidades persistentes y sus relaciones.
- `main/api/*` y `drf_views.py`: implementan la capa de control, gestionando las peticiones HTTP y exponiendo la API REST.
- `frontend-web/src/*`: constituye la capa de presentación, desarrollada en *React*.

Este patrón permite desacoplar la lógica de negocio de la interfaz de usuario y del protocolo HTTP, facilitando la testabilidad, el mantenimiento y la evolución del sistema a lo largo del desarrollo del proyecto.

7.1.5 Patrón cliente-servidor

El sistema se fundamenta en un modelo de desacoplamiento integral entre la interfaz de usuario y la lógica de servidor. Esta separación permite que el *frontend* (desarrollado en *React*) y el *backend* (*Django*) operen como entidades independientes.

Ventajas:

- Facilita el despliegue independiente de ambas partes.
- Permite el escalado horizontal específico (por ejemplo, aumentar instancias de la API sin afectar al servidor de archivos estáticos).
- Asegura que el backend pueda dar servicio a otros posibles clientes en el futuro (como una aplicación móvil nativa) sin modificaciones estructurales.

7.1.6 Arquitectura en capas

El sistema se estructura siguiendo una arquitectura en capas, donde cada nivel tiene una responsabilidad bien definida y se comunica únicamente con capas adyacentes.

Capas principales:

- **Capa de presentación:** *frontend-web* (*React*).
- **Capa de servicios/API:** *api/views* (*Django REST Framework*).
- **Capa de lógica de negocio:** módulos de dominio y entrenamiento Modelos (ML).
- **Capa de persistencia:** *models.py* y base de datos relacional.

La separación por capas reduce el acoplamiento entre componentes y mejora la modularidad del sistema, siendo especialmente relevante en un entorno híbrido que combina procesamiento de datos, modelos predictivos y servicios web.

7.1.7 Patrones de comportamiento

Además de los patrones arquitectónicos, el sistema incorpora patrones de comportamiento que permiten gestionar de forma eficiente la interacción entre componentes y la ejecución de procesos complejos. Estos patrones resultan especialmente relevantes en contextos donde existen eventos, flujos dinámicos o múltiples estrategias de ejecución.

En este apartado se detallan los patrones de comportamiento implementados, poniendo especial énfasis en su utilidad dentro del sistema y en cómo contribuyen a mejorar la extensibilidad y desacoplamiento.

7.1.8 Observer

El patrón *Observer* define una relación uno-a-muchos donde múltiples objetos (observadores) son notificados automáticamente ante cambios de estado en un objeto sujeto.

Aplicación en el sistema:

Se implementa mediante el sistema de *signals* de *Django* (`signals.py`):

- Creación automática de `UserProfile` tras la creación de un `User`.
- Indexación de entidades (Jugador, Equipo) en *MeiliSearch*.
- Emisión de notificaciones en tiempo real mediante *WebSockets* (Django Channels), utilizando *Redis* como backend.

Este patrón desacopla la lógica de persistencia de las acciones derivadas (indexación y notificaciones), mejorando la posible escalada del sistema.

7.1.9 Template method

Define el esqueleto de un algoritmo en una clase base, delegando ciertos pasos en subclases que pueden redefinir comportamientos específicos.

Aplicación en el sistema:

- `common_trainer.py`: define la clase base `BaseTrainer` con la estructura del proceso de entrenamiento.
- Scripts como `entrenar-modelo-*.py` y `predecir.py` extienden dicha clase, sobrescribiendo etapas concretas del *pipeline*.

Permite centralizar la lógica común del entrenamiento de modelos, reduciendo duplicidad de código y asegurando la consistencia en el pipeline de Machine Learning.

7.1.10 Strategy

Permite seleccionar dinámicamente un algoritmo o comportamiento en tiempo de ejecución.

Aplicación en el sistema:

- `predecir.py`: el diccionario `MODELO_POR_POSICION` determina el modelo a utilizar según el contexto.
- La función `cargar_modelo(...)` selecciona la implementación concreta.
- `predicciones.py`: permite especificar el tipo de modelo mediante parámetros en la petición.

Este patrón facilita la experimentación y comparación entre distintos algoritmos (Random Forest, Ridge, Baseline), permitiendo cambiar la estrategia sin modificar el flujo principal de ejecución.

7.1.11 Patrones en el *frontend*

El diseño del *frontend* no solo responde a criterios visuales, sino también a principios estructurales que facilitan la gestión del estado y la reutilización de lógica. En este sentido, se han adoptado patrones propios del ecosistema *React* que permiten construir interfaces modernas, escalables y mantenibles. A continuación, se describen los principales mecanismos utilizados en el *frontend*, centrados en la gestión del estado global y la encapsulación de lógica reutilizable.

Hooks

Los Hooks son funciones especializadas que permiten guardar el estado y otras funcionalidades del ciclo de vida del framework dentro de componentes funcionales. A continuación, se describen los tipos de Hooks integrados en el sistema y su propósito específico:

- `AuthContext.jsx` (`useState`, `useEffect`, `useCallback`, `useContext`).
- `JornadaContext.jsx`.
- `useTourWithManualExit.js`.

Más allá de los hooks nativos, el sistema implementa Custom Hooks, como `useTour-WithManualExit.js`. Estos son ganchos personalizados que encapsulan lógica compleja y específica del dominio para ser reutilizada en diferentes pantallas de la aplicación. En lugar de duplicar el código del tour guiado en cada página, el Custom Hook extrae esa lógica, manteniendo los componentes de la interfaz limpios y centrados exclusivamente en la representación visual.

Esta arquitectura basada en Hooks no solo mejora la legibilidad del código, sino que asegura que el *frontend* sea reactivo y eficiente, alineándose con los estándares actuales de la industria y garantizando una experiencia de usuario fluida.

7.1.12 Context API

Context API es un sistema nativo de *React* diseñado para compartir datos de forma global entre componentes, eliminando la necesidad de pasar propiedades manualmente a través de niveles intermedios. Este enfoque resuelve eficazmente el prop drilling, práctica ineficiente en la que los datos deben atravesar componentes que no los requieren funcionalmente solo para alcanzar a un nodo inferior en la jerarquía.

Aplicación en el sistema:

- **AuthContext:** autenticación.
- **JornadaContext:** estado de jornada.
- **TourContext:** gestión del tour interactivo.

Permite compartir información global entre componentes de forma eficiente, mejorando la escalabilidad y mantenibilidad del *frontend*.

7.2 Modelado predictivo

Una vez procesados los datos, el sistema aborda la fase de modelado mediante técnicas de Machine Learning supervisado. El objetivo es predecir el rendimiento futuro de los jugadores a partir de su histórico y de variables contextuales. En este apartado se describen los algoritmos utilizados, las estrategias de optimización y las métricas empleadas para evaluar el rendimiento de los modelos, justificando cada decisión desde un punto de vista técnico y práctico.

- **Variable objetivo (Y):** `target_pf_next` (Puntos Fantasy que el jugador obtendrá en la jornada siguiente).

- **Variables independientes (X):** Más de 50 variables que incluyen medias móviles, roles de élite, dificultad del rival, localía y cuotas de mercado.

7.2.1 Algoritmos y optimización

No se utiliza un único modelo, sino que se compite entre varias familias de algoritmos para cada posición:

- **Random Forest (RF):** Excelente para capturar interacciones no lineales y reducir la varianza.
- **XGBoost (XGB):** Un algoritmo de Gradient Boosting extremadamente potente que optimiza el error de forma iterativa.
- **Ridge y ElasticNet:** Modelos lineales con regularización $L1$ y $L2$ para evitar el sobreajuste cuando hay muchas variables correlacionadas.

El proceso de GridSearch permite encontrar la combinación óptima de hiperparámetros (profundidad del árbol, tasa de aprendizaje, número de estimadores) mediante validación cruzada, seleccionando el modelo con el menor Error Absoluto Medio (MAE).

7.2.2 Métricas de evaluación del modelo

Con el propósito de garantizar la validez científica y la utilidad práctica del sistema, la evaluación del rendimiento de los modelos se ha articulado en torno a dos métricas fundamentales: el Error Absoluto Medio (MAE) y el coeficiente de correlación de Spearman. La elección del MAE responde a su alta interpretabilidad en el dominio del Fantasy Football, ya que permite cuantificar la desviación media de las predicciones en la misma unidad de medida que la variable objetivo (puntos).

Complementariamente, el coeficiente de Spearman resulta crítico para validar la capacidad de ordenación del modelo, asegurando que la jerarquía de recomendaciones de jugadores sea consistente con su rendimiento real. Para contextualizar estos valores, se han realizado análisis estadísticos preliminares y procesos de benchmarking destinados a establecer umbrales de aceptación, los cuales definen el nivel de precisión mínimo para considerar un resultado como satisfactorio y apto para el apoyo a la decisión.

7.2.2.1 Justificación técnica del coeficiente de Spearman

A diferencia de la correlación de Pearson, que evalúa la relación lineal entre dos variables, el coeficiente de correlación de Spearman (r_s o ρ) es una métrica adimensional que mide la fuerza y la dirección de la asociación monótona entre dos variables. En el contexto de este proyecto, su uso es preferible por las siguientes razones:

1. Evaluación del ranking sobre el valor absoluto

En un sistema de recomendación para managers, a menudo es más valioso saber qué jugador rendirá mejor que otro (orden), que conocer la cifra exacta de puntos (magnitud). Si el modelo predice que el Jugador A sacará 8 puntos y el Jugador B sacará 6, y la realidad es que el A saca 5 y el B saca 3, el MAE registrará un error alto, pero la correlación de Spearman será perfecta (+1), ya que el orden de recomendación fue correcto para la toma de decisiones (fichar al Jugador A).

2. Robustez ante relaciones no lineales

Los puntos no siempre siguen una distribución lineal respecto a las variables independientes (por ejemplo, el número de entradas o pases clave). Spearman captura relaciones monótonas donde las variables tienden a cambiar juntas, pero no necesariamente a un ritmo constante, lo que se ajusta mejor a la naturaleza estocástica del deporte.

3. Formulación matemática

El coeficiente se calcula basándose en los rangos (posiciones en la lista) de los datos en lugar de sus valores brutos. La fórmula utilizada es:

$$r_s = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Donde:

- d_i : Representa la diferencia entre los rangos de la predicción y el resultado real para cada observación i .
- n : Es el número total de observaciones (jugadores evaluados).

4. Interpretación de resultados

El valor de r_s oscila entre -1 y +1:

- +1: existe una coincidencia perfecta en el orden de los rangos (el modelo ordena a los jugadores exactamente igual que su rendimiento real).

- 0: no existe asociación entre los rangos (las predicciones son equivalentes al azar).
- -1: existe una oposición perfecta (el modelo recomienda como “mejores” a los jugadores que acaban rindiendo peor).

En el ámbito de la analítica deportiva, debido a la alta variabilidad del juego, se han establecido como umbrales de éxito valores de $r_s \geq X$ para considerar que el modelo aporta un valor estadístico significativo para la recomendación estratégica.

7.2.3 Criterios de validez y benchmarking de modelos

Para determinar si un modelo predictivo es útil en un entorno tan volátil como el fútbol, no es suficiente con obtener un error bajo; es imperativo establecer qué constituye un “buen resultado” para cada posición. Antes de la fase de entrenamiento, se ha realizado un estudio de referencia basado en la distribución real de puntos.

Un error absoluto medio (MAE) de 3 puntos no significa lo mismo para un Portero que para un Centrocampista. Como se observa en el estudio previo, la naturaleza de la puntuación varía drásticamente:

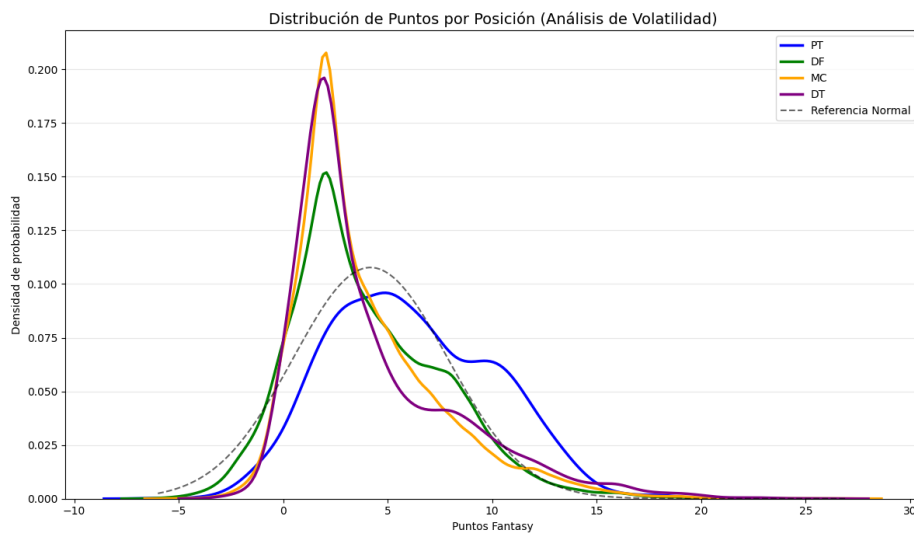


Figura 19: Distribución de puntos por posición comparada con distribución normal.

- **Porteros (PT):** Tienen una media de puntos más alta ($\approx 6,1$), lo que permite un MAE absoluto mayor ($\approx 3,25$) manteniendo una fiabilidad relativa alta (53.3 % de error).

- **Centrocampistas y Defensas (MC/DF):** Tienen medias más bajas ($\approx 3,9$), lo que implica que un error de 3 puntos supone casi un 71 % de desviación sobre su media, siendo mucho más crítico reducir el error en estas posiciones.

Para evitar los sesgos de los valores atípicos (jugadores con puntuaciones extremas de 35 o 40 puntos), se ha descartado el uso de la desviación estándar en favor de los percentiles de error.

Este enfoque garantiza que los umbrales de validez se ajusten a la forma real de la distribución de cada posición. A continuación, se presentan los objetivos de rendimiento que los modelos de Machine Learning deben alcanzar para ser considerados válidos:

Posición	Media Puntos	MAE Medio	Error %	Excelente (P25)	Pobre (P75)
PT	6.09	3.25	53.3	2.93	3.63
DF	3.98	2.80	70.3	2.15	3.18
MC	3.94	2.81	71.2	2.10	3.25
DT	4.25	2.94	69.2	2.25	3.25

Cuadro 3: Distribución de desempeño esperado por posición.

Por lo que los objetivos marcados para la fase de entrenamiento son:

Posición	Élite (P25)	Aceptable (Mediana)	Alerta (P75)
Porteros (PT)	$MAE \leq 2,93$	$MAE \leq 3,18$	$MAE > 3,63$
Defensas (DF)	$MAE \leq 2,15$	$MAE \leq 2,79$	$MAE > 3,18$
Mediocentros (MC)	$MAE \leq 2,10$	$MAE \leq 2,60$	$MAE > 3,25$
Delanteros (DT)	$MAE \leq 2,25$	$MAE \leq 2,75$	$MAE > 3,25$

Cuadro 4: Objetivos de rendimiento para modelos de ML por posición.

7.3 Decisiones de diseño estratégicas

Más allá de la implementación técnica, el sistema incorpora una serie de decisiones de diseño clave que afectan directamente a la validez, fiabilidad y utilidad del modelo

predictivo. Estas decisiones han sido tomadas considerando tanto aspectos teóricos como prácticos del dominio. En esta sección se exponen las principales elecciones estratégicas realizadas, junto con su justificación y su impacto en el comportamiento del sistema.

7.3.1 Estrategia de extracción y congelación de datos

Se ha optado por congelar la extracción de datos en la jornada 17 de la temporada 2025/26. Esta decisión se fundamenta en los siguientes pilares:

- **Volumen de datos significativo:** el histórico abarca 2 temporadas y media de competición. Considerando una media de 25 jugadores con minutos por cada uno de los 10 partidos de cada jornada, el volumen de registros gestionado es superior a las 23.000 entradas. Este volumen garantiza una base estadística sólida para el entrenamiento de los modelos.
- **Factor de predicción real:** al limitar el entrenamiento hasta la jornada 17, el sistema se enfrenta a datos que “desconoce” (las jornadas posteriores). Esto permite realizar un backtesting funcional genuino, donde el modelo debe predecir resultados que, aunque ya han ocurrido en la realidad, no han sido utilizados para su ajuste.
- **Integridad del modelo:** aunque se han implementado técnicas de ingeniería de variables para evitar que información del futuro se filtre en el pasado, la decisión de realizar un corte temporal estricto actúa como una salvaguarda física. De este modo, se asegura que el modelo solo toma decisiones basadas en la información que un mánager real tendría disponible en el momento de realizar su alineación.

7.3.2 Especialización de modelos por posición

Una de las innovaciones clave en la arquitectura es la segmentación del aprendizaje según el rol del jugador. Se ha descartado el uso de un modelo único global en favor de modelos específicos para cada demarcación (Porteros, Defensas, Centrocampistas y Delanteros).

Esta decisión se fundamenta en la heterogeneidad de las métricas de rendimiento:

- **Divergencia de variables:** las variables que determinan el éxito de un Portero (PT), como paradas, despejes o goles encajados, son irrelevantes o inexistentes para un Mediocentro (MC), quien se evalúa por pases clave, recuperaciones o asistencias.

- **Distribución de puntuaciones:** los sistemas de puntuación ponderan de forma distinta las mismas acciones según la posición. Un gol de un defensa suele tener un impacto estadístico y de puntuación mayor que el de un delantero.
- **Optimización del aprendizaje:** al entrenar modelos independientes, el algoritmo de boosting puede identificar patrones específicos de cada posición sin el “ruido” que generarían los perfiles estadísticos de otros roles. Esto reduce el error absoluto medio (MAE) y mejora la precisión en jugadores con roles muy específicos.

Se observan diferencias significativas en la distribución de puntos según la posición, por lo que se considera esta opción más adecuada para aportar valor y abordar el problema de la forma más realista posible.

7.4 Modelado de datos

El diseño de los datos constituye uno de los pilares fundamentales del sistema, ya que determina tanto la integridad de la información como la eficiencia en su acceso y procesamiento. Para ello, se ha seguido un enfoque progresivo que abarca desde el modelo conceptual hasta su implementación en base de datos.

En este apartado se describe la evolución del modelo de datos a través de sus distintos niveles de abstracción, garantizando la coherencia entre el diseño teórico y la implementación final.

El diseño de la base de datos se ha desarrollado de forma progresiva a través de tres niveles de abstracción: modelo conceptual, modelo entidad-relación (ER) y modelo lógico. Esta evolución permite partir de una visión general del dominio hasta llegar a una implementación concreta en base de datos, alineada finalmente con los modelos definidos en Django (`models.py`).

7.4.1 Modelado conceptual

El modelo conceptual define las entidades principales del dominio y las relaciones entre ellas a alto nivel, sin entrar aún en detalles de implementación. En este caso, el dominio modelado corresponde a una competición de fútbol con información tanto estructural (equipos, jornadas, partidos) como analítica (estadísticas y rendimiento de jugadores).

Las entidades principales identificadas son:

- **Temporada:** representa una competición concreta (por ejemplo, 2023/2024). Es el eje central del modelo.
- **Equipo:** entidad independiente que participa en distintas temporadas.
- **Jugador:** entidad que representa a cada futbolista.
- **Jornada:** subdivisión temporal de una temporada.
- **Partido:** evento en el que participan dos equipos dentro de una jornada.

A partir de estas entidades principales, se introducen entidades intermedias y de análisis:

- **EquipoTemporada:** resuelve la relación muchos-a-muchos entre equipo y temporada.
- **EquipoJugadorTemporada:** representa la plantilla de un equipo en una temporada concreta, incluyendo atributos como dorsal, edad o percentiles.
- **ClasificacionJornada:** almacena la clasificación de los equipos en cada jornada.
- **EstadisticasPartidoJugador:** recoge el rendimiento individual de cada jugador en un partido.
- **Calendario:** estructura auxiliar para consultas rápidas de partidos programados.
- **RendimientoHistoricoJugador:** agrega estadísticas acumuladas por jugador, equipo y temporada.

Relaciones clave del modelo conceptual:

- Una Temporada contiene múltiples Jornadas.
- Una Temporada tiene múltiples Equipos (a través de EquipoTemporada).
- Un Equipo participa en múltiples Partidos.
- Un Jugador juega en múltiples equipos y temporadas (a través de EquipoJugadorTemporada).
- Un Partido genera estadísticas de múltiples jugadores.
- Un Jugador tiene historial y estadísticas acumuladas.

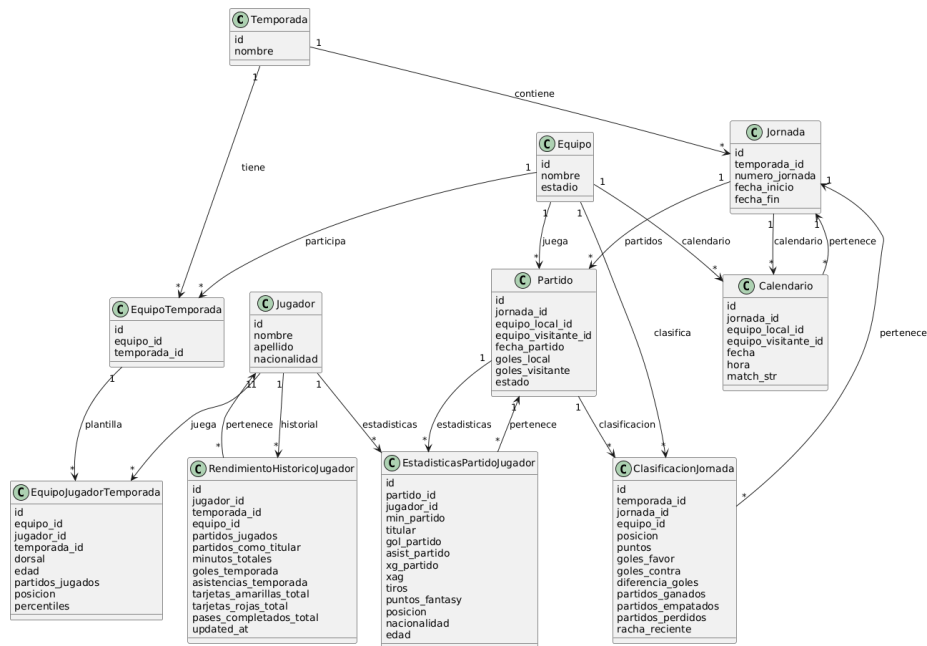


Figura 20: Modelo conceptual del sistema.

7.4.2 Diagrama entidad-relación

Dando el salto natural a la arquitectura relacional, el segundo diagrama elimina el ruido visual de los atributos para centrarse exclusivamente en cómo las entidades interactúan entre sí, observándose claramente las cardinalidades.

Relaciones principales:

- Una temporada contiene múltiples jornadas, mientras que cada jornada pertenece a una única temporada.
- Una temporada se asocia con múltiples registros de equipo-temporada, lo que permite modelar la participación de distintos equipos en cada edición de la competición. Esta relación resuelve el caso muchos-a-muchos entre equipos y temporadas.
- Un equipo participa en múltiples partidos, ya sea como equipo local o visitante.
- Una jornada agrupa múltiples partidos.
- Un partido genera múltiples registros en estadísticas de partido por jugador.
- Un jugador puede tener múltiples registros de estadísticas de partido, correspondientes a los distintos eventos.
- La entidad equipo-temporada se relaciona con equipo-jugador-temporada, lo que

permite definir la plantilla de cada equipo en una temporada concreta, incluyendo información adicional como dorsal, edad o métricas avanzadas.

- Un jugador se relaciona con rendimiento histórico, lo que permite almacenar estadísticas agregadas por temporada y equipo, facilitando el análisis longitudinal de su rendimiento.

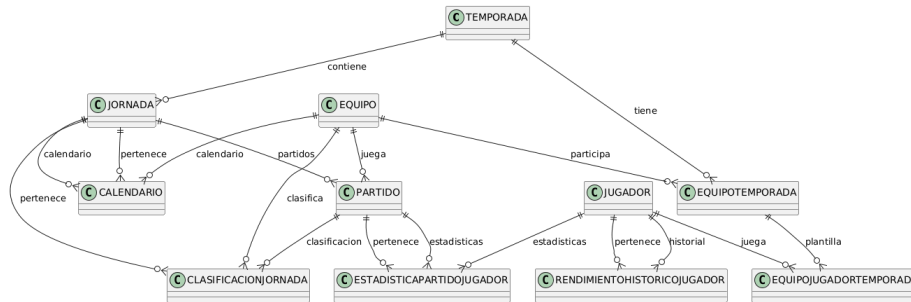


Figura 21: Diagrama Entidad-Relación.

7.4.3 Modelo lógico

El modelo lógico representa la implementación concreta en base de datos relacional, y en este caso se ha desarrollado directamente a partir de Django (models.py), lo cual garantiza coherencia entre diseño e implementación.

Cada entidad del modelo ER se transforma en una tabla (entity) con:

- Clave primaria (id)
- Claves foráneas (FK)
- Restricciones (UNIQUE, validators)

El modelo lógico no es solo teórico, sino que está implementado directamente en Django:

- Cada entidad → models.Model
- Relaciones → ForeignKey
- Restricciones → unique_together, validators
- Enumeraciones → TextChoices (ej: EstadoPartido, Posicion)

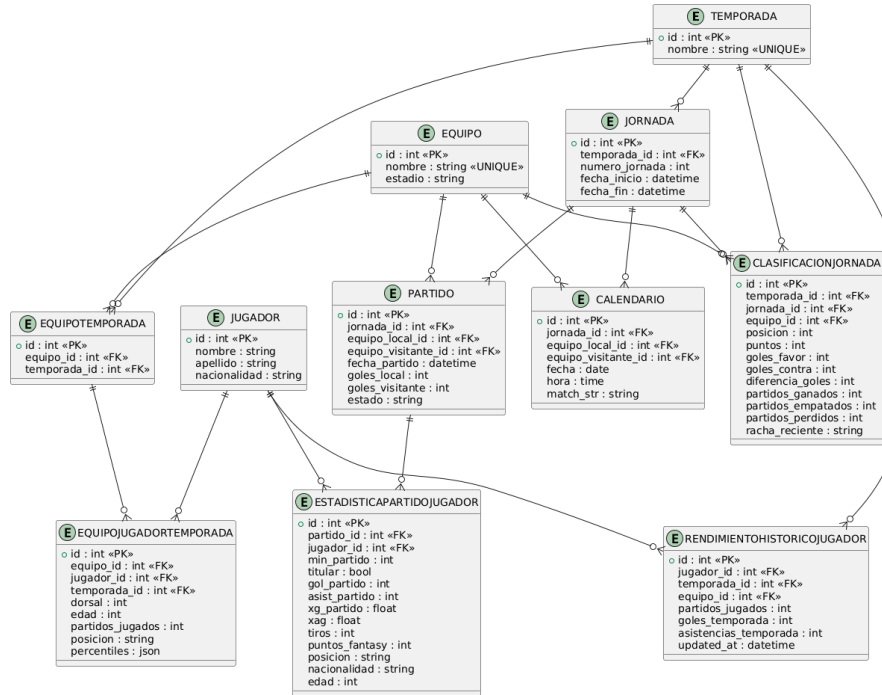


Figura 22: Esquema lógico del sistema.

7.5 Infraestructura de datos y búsqueda

Con el fin de cumplir los requisitos de rendimiento y escalabilidad, el sistema se apoya en una infraestructura de datos compuesta por múltiples tecnologías especializadas, ya explicada en el punto de Stack Tecnológico. Cada una de ellas desempeña un papel fundamental en la gestión, consulta y optimización de la información.

En esta sección se describen los distintos sistemas utilizados, así como su integración dentro de la arquitectura global y la justificación de su elección.

- **Base de datos (PostgreSQL)**: Es el sistema de persistencia relacional primario. Almacena la información estructural y crítica: datos de jugadores, histórico de plantillas, usuarios y configuraciones de seguridad. Garantiza la integridad referencial y el cumplimiento de las propiedades ACID.
- **MeiliSearch**: Se utiliza como motor de indexación y búsqueda avanzada. Su función es permitir que el Buscador (RF4) y el Matching de entidades funcionen con latencias mínimas, incluso sobre grandes volúmenes de texto, mediante el uso de índices invertidos.
- **Redis**: Actúa como capa de memoria distribuida de alta velocidad y como infraestructura de mensajería interna. En el sistema se utiliza para cachear respuestas

frecuentes de la API y para soportar el canal de comunicación de Django Channels en funcionalidades en tiempo real. Esta doble función reduce la carga de PostgreSQL y estabiliza la latencia en operaciones repetitivas, a la vez que garantiza coordinación entre instancias backend.

Inicialmente, se consideró el uso de Elasticsearch como motor de búsqueda. Sin embargo, debido a las limitaciones de licencia y a la necesidad de utilizar una solución completamente abierta y sin costes asociados, se decidió sustituirlo por OpenSearch. Esta alternativa, desarrollada como un fork open source de Elasticsearch, mantiene una alta compatibilidad a nivel conceptual y funcional, permitiendo implementar las mismas estrategias de indexación y consulta sin dependencia de licencias propietarias. Finalmente, esta opción se descartó debido a motivos de rendimiento y se sustituyó por MeiliSearch.

7.6 Diseño de la interfaz

La interfaz de usuario constituye el punto de interacción entre el sistema y el usuario final, por lo que su diseño debe garantizar tanto la usabilidad como la claridad en la presentación de la información. En esta sección se describe la estructura de navegación, los componentes principales y las decisiones de diseño visual adoptadas, con el objetivo de ofrecer una experiencia de usuario coherente e intuitiva.

7.6.1 Nivel de navegación

El nodo raíz o vista de aterrizaje es el Menú (Home). Desde este punto, o a través del Sidebar, el sistema bifurca la navegación hacia los diferentes dominios de la aplicación:

- **Clasificación:** vista general del estado de la liga y resultados de la jornada seleccionada.
- **Vista Jugador:** acceso directo a perfiles individuales.
- **Mi Plantilla:** módulo de gestión del equipo del usuario.
- **Amigos:** módulo social para interactuar con otros usuarios.
- **Equipos:** directorio o listado global de las entidades deportivas.

7.6.2 Estructura base y componentes persistentes

La interfaz se construye sobre un layout principal que encapsula dos componentes de renderizado persistente, garantizando que el usuario nunca pierda el contexto de navegación:

- **Navbar (barra de navegación superior):** Actúa como el controlador del estado de autenticación de la sesión. Es visible en todas las rutas de la aplicación y renderiza condicionalmente el acceso al inicio de sesión (Login) o la gestión de la cuenta (Perfil), dependiendo de la sesión activa del usuario.
- **Sidebar (menú lateral):** Funciona como el enrutador principal. Está permanentemente accesible y proporciona saltos directos a los módulos core del sistema: Menú, Mi Plantilla, Liga, Amigos, Equipos y Estadísticas.

7.6.3 Flujos de profundización

La aplicación implementa un sistema de navegación jerárquica donde las vistas generales actúan como pasarelas hacia vistas de detalle, pasando los identificadores correspondientes (por ejemplo, vía parámetros en la URL o props en el enrutador):

- **Flujo de entidades deportivas:** tanto la vista de Clasificación como el directorio general de Equipos convergen en un nodo común: el Detalle de Equipo. A su vez, el Detalle de Equipo actúa como contenedor de una plantilla, permitiendo navegar hacia la Vista Jugador específica al seleccionar un integrante del club.
- **Flujo analítico:** la vista de Estadísticas permite el salto directo hacia la Vista Jugador para consultar métricas detalladas de un perfil concreto.
- **Flujo social:** la vista de Amigos permite ramificar la navegación hacia el Detalle de Amigo. Por otro lado, la vista de Mi Plantilla no contiene navegación específica más allá del navbar y el menú lateral.

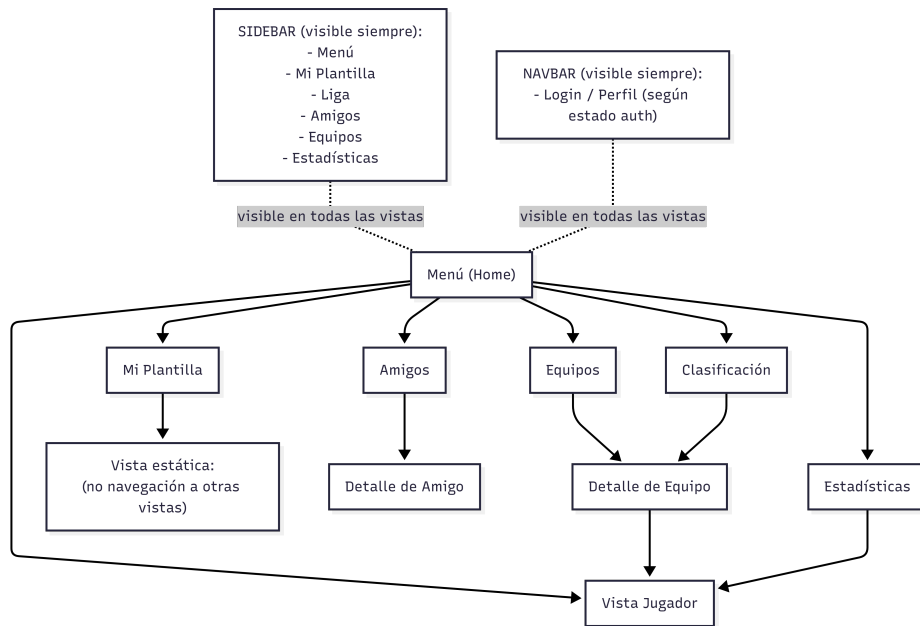


Figura 23: Diagrama de navegación de UI.

7.6.4 Diseño de la interfaz

El diseño de la paleta de colores responde a criterios de simplicidad, contraste y coherencia visual. La combinación de verde vibrante (*Primary*) con tonalidades más oscuras busca transmitir una sensación de tecnología, modernidad e innovación, valores que se alinean con la naturaleza inteligente del sistema y su integración con modelos de Machine Learning e IA.

Primary	Primary dark	Negro puro	Gris oscuro	Gris dark
#50c878	#2d8659	#050505	#1e1e1e	#262626

Cuadro 5: Paleta de colores empleada en la interfaz.

Familia tipográfica

- **Display:** Lexend, sans-serif (títulos y elementos destacados)
- **Body:** Noto Sans, sans-serif (textos corridos y descripciones)

7.7 Diseño de seguridad y autenticación

El sistema implementa la seguridad utilizando las capacidades nativas de Django, lo que incluye autenticación basada en sesiones, protección frente a ataques CSRF, gestión segura de cookies y almacenamiento de contraseñas mediante algoritmos de hashing robustos. Esta aproximación permite delegar en el framework gran parte de la gestión de la seguridad, reduciendo la complejidad del desarrollo y minimizando errores comunes en la implementación manual de estos mecanismos.

7.8 Diseño del despliegue

Para garantizar la portabilidad, escalabilidad y consistencia del sistema entre los diferentes entornos de ejecución, se ha adoptado una estrategia de containerización basada en Docker y Docker Compose. Esta decisión permite aislar las dependencias de cada componente, asegurando que el software se comporte de manera idéntica en desarrollo, preproducción y producción.

7.8.1 Ecosistema de contenedores y responsabilidades

Durante el desarrollo, el sistema se descompone en cinco servicios principales orquestados mediante una red virtual privada:

- **Capa de persistencia (db):** basada en PostgreSQL 16-alpine. Se ha seleccionado la variante alpine para minimizar el peso de la imagen. La persistencia se garantiza mediante volúmenes de Docker (`postgres_data`).
- **Capa de caché y mensajería (redis):** basada en Redis 7 alpine. Proporciona caché distribuida para respuestas de API y backend de channel layer para Django Channels. Su incorporación reduce latencia en lecturas repetidas y permite comunicación en tiempo real entre múltiples instancias del backend.
- **Motor de búsqueda (meilisearch):** nodo de MeiliSearch encargado de la indexación y consultas de alto rendimiento.
- **Núcleo de aplicación (backend):** ejecuta la lógica de Django sobre un servidor Daphne (ASGI), permitiendo la gestión híbrida de peticiones REST y comunicaciones bidireccionales mediante WebSockets.
- **Servidor web y frontend (frontend):** contenedor dual que contiene los assets de React y un servidor Nginx configurado como punto de entrada único al sistema.

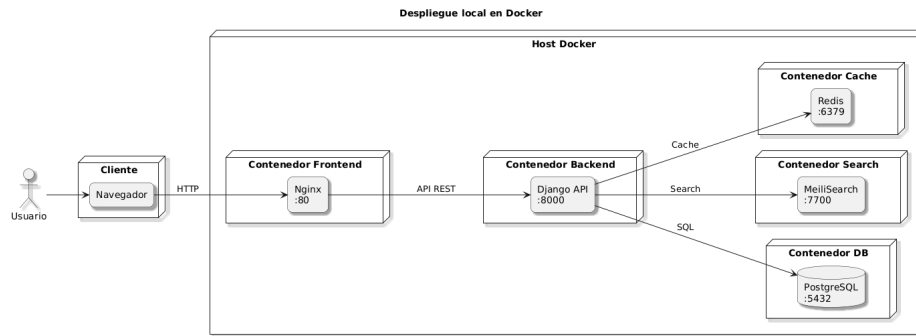


Figura 24: Diagrama de despliegue del sistema en local.

Para la fase de producción, se ha optado por una estrategia diferente: utilizar servicios gestionados en la nube para alojar la base de datos, la capa de caché y el frontend. Esta decisión ha sido motivada por el uso del crédito de Azure proporcionado por la Universidad de Sevilla; si se desplegaba todo en conjunto, el rendimiento era deficiente y se consumiría todo el crédito disponible en poco tiempo.

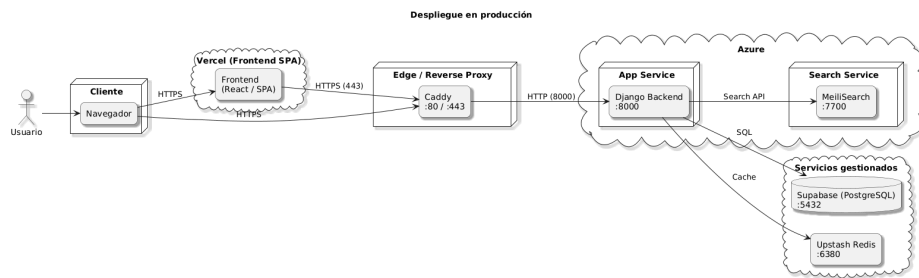


Figura 25: Diagrama de despliegue del sistema en fase de producción.

7.8.2 Estrategia de construcción

Para optimizar el rendimiento y la seguridad, se han diseñado Dockerfiles multi-etapa:

- **Backend:** se utiliza una primera etapa (builder) con las herramientas de compilación necesarias para instalar las dependencias de Python y las librerías de Machine Learning. En la etapa de runtime, solo se copian los binarios instalados a una imagen limpia de Python 3.11-slim, reduciendo drásticamente el tamaño final y eliminando herramientas de compilación que podrían ser explotadas.
- **Frontend:** se emplea un entorno Node.js 20-slim para la construcción con React. Posteriormente, el resultado se transfiere a una imagen de Nginx Alpine, que actúa como servidor de alto rendimiento para archivos estáticos.

Se han definido dos entornos diferenciados para cubrir el ciclo de desarrollo:

- **Entorno de desarrollo (Local):** ejecución mediante docker-compose up en la máquina local del desarrollador. Permite una depuración rápida manteniendo la paridad con el entorno final.
- **Entorno de producción (Azure):** el despliegue final se realiza en Azure Web App for Containers. Esta elección permite escalar vertical y horizontalmente el sistema según la carga de usuarios. Se aprovecha el crédito ofrecido por la Universidad de Sevilla a través del programa Azure for Students, garantizando un entorno profesional, seguro y monitorizado sin coste adicional para el proyecto.

En definitiva, la estrategia de despliegue se ha regido por un principio de optimización crítica de recursos sin sacrificio de funcionalidad. La selección sistemática de versiones slim y alpine para las imágenes base, junto con una configuración de contenedores de baja demanda, permite ofrecer la experiencia de usuario completa y el rendimiento analítico requerido manteniendo una huella computacional mínima. Este enfoque garantiza la viabilidad del sistema dentro de los límites de computación del programa Azure for Students, evitando el agotamiento prematuro de las cuotas de crédito de la Universidad de Sevilla.

8. Recolección y análisis de datos

Este capítulo detalla el proceso crítico de transformación de los datos en bruto obtenidos mediante técnicas de extracción web hasta la consolidación de un dataset estructurado apto para el entrenamiento de modelos de Machine Learning. Se describe la procedencia de los datos, el proceso de integración entre fuentes heterogéneas, la limpieza, y la ingeniería de variables que permite al sistema capturar la naturaleza estocástica del fútbol.

8.1 Fuentes de datos y proceso de scraping

La construcción del dataset se fundamenta en la extracción automatizada de datos desde tres fuentes especializadas y complementarias:

- **FBRef.com:** Fuente principal de métricas avanzadas de rendimiento individual (xG, PSxG, conducciones progresivas, intercepciones, estadísticas defensivas y de portería). Los datos se obtienen procesando los archivos HTML de cada partido de las últimas 2,5 temporadas (2023/24, 2024/25 y la actual 2025/26).
- **FutbolFantasy.com:** Proporciona el contexto específico del juego Fantasy, incluyendo los puntos obtenidos por cada jugador en cada jornada. Constituye la fuente de la variable objetivo (`target_pf_next`).
- **Transfermarkt:** Fuente de contexto competitivo, empleada para extraer las rachas de los equipos, la clasificación por jornada, datos de estadios y nacionalidades de los jugadores.

8.1.1 Barreras técnicas y estrategias de mitigación

Durante el proceso de extracción se detectaron dos barreras críticas que obligaron a diseñar estrategias específicas:

- **Bloqueos de IP en FBRef:** Este portal implementa políticas extremadamente estrictas de *rate limiting*. Ante la imposibilidad de realizar un scraping masivo automatizado sin ser bloqueado, se optó por la descarga controlada y local de los archivos HTML de cada partido, procesándolos de forma diferida. Esta decisión garantizó la integridad de los datos sin comprometer la disponibilidad del sistema.

- **Evasión de Cloudflare en FutbolFantasy:** El portal bloquea agentes no humanos. Se implementó el módulo `descargar_puntos.py` utilizando la librería `cloudscraper`, que simula una huella de navegador real gestionando automáticamente los desafíos de JavaScript y cabeceras *User-Agent*.

Para garantizar la robustez de las descargas, se implementó un algoritmo de *backoff* exponencial con varianza aleatoria, que evita que el servidor identifique patrones de automatización y permite recuperarse de errores transitorios de red:

Listing 1: Algoritmo de reintentos con backoff exponencial

```
delay_base = min(backoff_inicial * (2 ** (intento - 1)),
                 backoff_max)
delay = delay_base + random.uniform(0, 1.5)
time.sleep(delay)
```

8.1.2 Módulos de scraping

La lógica de extracción se estructura en módulos especializados:

- `fbref.py`: Parsea las tablas HTML de cada partido (summary, passing, defense, etc.) mediante BeautifulSoup. Combina múltiples tablas en memoria usando el nombre normalizado del jugador como clave de unión. Incluye lógica especializada para porteros, cuyas métricas (PSxG, Save%, GA90) se extraen de tablas parcialmente distintas.
- `roles.py`: Extrae los rankings de eficiencia de FBRef (goles por disparo, pases al último tercio, intercepciones, etc.) desde nodos HTML ocultos en comentarios del DOM, usando una técnica de extracción específica:

Listing 2: Extracción de datos ocultos en comentarios HTML

```
comments = soup.find_all(string=lambda text:
                          isinstance(text, Comment))
for c in comments:
    if 'id="div_leaders"' in c:
        frag_soup = BeautifulSoup(c, "lxml")
```

- `transfermarkt.py`: Extrae la clasificación por jornada (PJ, PG, PE, PP, GF, GC, puntos) y datos de contexto competitivo.

- `commons.py`: Motor de normalización de texto compartido entre todos los módulos. Aplica normalización NFD para eliminar acentos y diacríticos, expresiones regulares para limpiar caracteres especiales, y detección de minutos concatenados al nombre del jugador (ej.: 90+4').
- `popularDB.py`: Orquestador final del pipeline. Valida la disponibilidad local de los HTMLs, crea las entidades base en la base de datos (temporadas, equipos, jornadas), ensambla las estadísticas por partido y jugador, aplica la normalización final y persiste los datos en PostgreSQL.

8.2 Matching de entidades entre fuentes

Uno de los mayores retos técnicos del proyecto fue la desambiguación de nombres entre fuentes. Un mismo jugador puede aparecer como “Lamine Yamal”, “L. Yamal” o “Yamal” según el portal. El proceso de *matching* se articula en tres niveles.

8.2.1 Normalización de texto

Antes de cualquier comparación, el sistema aplica una normalización agresiva: conversión a minúsculas, eliminación de tildes mediante NFD, eliminación de caracteres especiales (guiones, puntos) y limpieza de minutos concatenados por FBRef al nombre del jugador.

8.2.2 Diccionario de alias

Para casos donde la normalización es insuficiente (ej.: “Pepelu” vs. “José Luis García Vaya”), el sistema utiliza un diccionario de alias por temporada y equipo que intercepta estos casos antes de la lógica probabilística.

8.2.3 Fuzzy Matching contextual a tres niveles

El *matching* no compara contra toda la base de datos, sino que se acota al contexto del partido (los jugadores de los dos equipos enfrentados), eliminando falsos positivos masivos. La función `obten_coincidencia_nombre` aplica una estrategia en cascada:

1. **Regla de apellidos críticos:** Para apellidos muy comunes en el fútbol (García, González, Williams...) se exige coincidencia del nombre completo. Este nivel resuelve casos como los hermanos Iñaki y Nico Williams del Athletic Club.

2. **Coincidencia por prefijo/sufijo:** Para nombres de un único token sin apellido crítico, el sistema detecta si coincide con el inicio o fin de algún candidato. Resuelve apodos y abreviaciones (“Araújo” → “Ronald Araújo”).
3. **Fallback a WRatio:** Si los niveles anteriores no producen un resultado confiable, se recurre al algoritmo **WRatio** de la librería **rapidfuzz**, que combina internamente *Partial Ratio*, *Token Sort Ratio* y escalado ponderado de pesos. Esto supera las limitaciones de la distancia de Levenshtein clásica, que rechazaría como válida la coincidencia “Isaac Palazon” frente a “Isaac Palazon Camacho” por la diferencia de caracteres.

En caso de conflicto (el mismo jugador Fantasy propuesto para múltiples registros de FBRef), el sistema aplica criterios de desempate por minutos jugados, puntuación Fantasy y consistencia del apellido.

8.3 Limpieza y saneamiento del dataset

Una vez consolidado el histórico, el pipeline aplica las siguientes transformaciones de saneamiento:

- **Tratamiento de outliers:** Se identifican registros con puntuaciones anómalas superiores a 30 puntos en una jornada (habitualmente errores de las fuentes) y se sustituyen por la mediana o moda estadística del jugador, evitando que el modelo aprenda de eventos estadísticamente irrelevantes.
- **Filtrado de actividad:** Se eliminan registros de jugadores con menos de 10 minutos disputados, ya que su aporte estadístico no es representativo y genera métricas por 90 minutos distorsionadas.
- **Filtrado por historial mínimo:** Se exige un mínimo de 5 partidos por jugador para que las variables de ventana deslizante tengan una base histórica estadísticamente fiable.
- **Sanitización de valores no válidos:** Los valores NaN e infinitos se corrigen mediante imputación por mediana o relleno controlado en función de la fase del procesamiento.

8.4 Ingeniería de características

El sistema no utiliza datos brutos para el entrenamiento, sino variables elaboradas que capturan tendencias temporales y contexto competitivo. Es importante destacar que estas variables derivadas no residen de forma persistente en la base de datos; se generan dinámicamente durante el pipeline de entrenamiento a partir de los datos brutos, garantizando que los cálculos se realicen siempre sobre el estado más actualizado y sin redundancia de almacenamiento.

8.4.1 Variables de serie temporal

Para capturar el estado de forma de cada jugador, se generan métricas sobre ventanas deslizantes con desplazamiento temporal (`shift(1)`), garantizando que cada fila utilice únicamente información anterior al partido a predecir:

- **Rolling Mean (media móvil simple)**: Promedio de las últimas N jornadas (ventanas de 3 y 5). Captura la estabilidad y el rendimiento sostenido.
- **EWMA (Exponential Weighted Moving Average)**: Media ponderada que prioriza los encuentros más recientes, permitiendo al modelo reaccionar con mayor sensibilidad ante cambios de forma, rol o minutaje.

$$EWMA_t = \alpha \cdot X_t + (1 - \alpha) \cdot EWMA_{t-1}$$

Durante el desarrollo se evaluó también una ventana de 7 partidos, que fue descartada por suavizar en exceso la señal y enmascarar cambios relevantes en el rendimiento.

8.4.2 Enriquecimiento con roles de élite

Los rankings de eficiencia extraídos de FBRef (jugadores en el top-10 de una métrica en la liga) se convierten en variables numéricas mediante un factor de posición exponencial. Un jugador en el puesto 1 de un ranking tiene un peso significativamente mayor que uno en el puesto 10:

$$Score = \sum \left(Valor \times Peso \times \frac{1}{\sqrt{Posicion}} \right)$$

El módulo `role_enricher.py` crea adicionalmente variables de interacción entre la condición de élite y la intensidad del rol específico del jugador (ej.: `elite_paradas_interact`

para porteros).

8.4.3 Integración de cuotas de mercado y contexto de partido

Las cuotas de las casas de apuestas, transformadas en probabilidades implícitas, se incorporan como señal del “sentimiento del mercado” previo al partido. Esta fue una decisión de diseño determinante: en las fases iniciales del proyecto sin estas variables, los resultados predictivos eran significativamente peores. Los datos se obtienen de football-data.co.uk.

Variables derivadas de este bloque: probabilidad implícita de victoria (`p_home`, `p_away`), probabilidad de más de 2.5 goles (`p_over25_ewma5`), dificultad del rival (`fixture_difficulty_home`, `fixture_difficulty_away`), y goles encajados esperados por el rival (`opp_gc_ewma5`).

8.4.4 Variables específicas por posición

Para maximizar la precisión predictiva, se han diseñado conjuntos de variables específicos según el rol del jugador. Las variables de ruido (estadísticas propias de otra posición) se eliminan explícitamente para evitar que el modelo aprenda de señales sin valor predictivo real para cada demarcación.

Porteros (PT/GK) Las variables de creación ofensiva (goles, xG, regates, conducciones, duelos) se eliminan por ser ruido puro para esta posición. Las variables específicas generadas son:

- `cs_probability`: Probabilidad compuesta de portería a cero, calculada como media ponderada de cuatro señales: probabilidad implícita de las apuestas (peso 0.4), ratio inverso de goles encajados del rival (0.2), ratio inverso de tiros del rival (0.2) y dificultad del fixture (0.2).
- `cs_expected_points`: Puntos esperados por *clean sheet* ($4 \times cs_probability$).
- `save_per_90_ewma5`: Media exponencial de paradas por cada 90 minutos, capturando la actividad y eficacia del portero.
- `psxg_per_90_ewma5`: Goles esperados tras el tiro (PSxG) por 90 minutos, indicador de la presión recibida.

- **expected_gk_core_points**: Variable sintética que combina CS esperado, volumen de paradas y PSxG en un único indicador: $cs_expected_points + 0,4 \cdot save_per_90_ewma5 - 0,3 \cdot psxg_per_90_ewma5$.

Defensas (DF) Las métricas de portería (paradas, PSxG, goles en contra y sus derivadas temporales) se eliminan como ruido. Las variables específicas generadas son:

- **tackles_per_90_ewma5**: Intensidad defensiva en entradas normalizada a 90 minutos, con suavizado exponencial.
- **int_per_90_ewma5**: Capacidad de interceptación por 90 minutos, con tendencia exponencial.
- **clearances_per_90**: Despejes normalizados a 90 minutos.
- **defensive_actions_total**: Agregado ponderado de acciones defensivas ($entradas + intercepciones + despejes \times 0,5$).
- **def_actions_per_90** y **def_actions_ewma5**: Normalización e historial exponencial de las acciones defensivas totales.
- **cs_activity_alignment**: Correlación entre la probabilidad de portería a cero del equipo y la actividad defensiva propia del jugador.
- **int_momentum**: Tendencia en el número de intercepciones (diferencia EWMA3 – EWMA5).
- **defensive_context**: Interacción entre la condición de local y la dificultad del fixture.

Mediocentros (MC) Las métricas exclusivas de portería (paradas, PSxG, *clean sheet* esperado) se eliminan como ruido. Las variables específicas generadas son:

- **pass_attempts_per_90**: Volumen de distribución de juego normalizado a 90 minutos.
- **pass_accuracy_consistency**: Estabilidad en el acierto de pase en las últimas jornadas (EWMA5 del porcentaje de pases completados).
- **dribbles_per_90**: Frecuencia de regates por 90 minutos.
- **dribble_intensity**: Media móvil de conducciones totales.

- **progressive_action_rate**: Tasa de conducciones progresivas por 90 minutos.
- **defensive_contribution**: EWMA5 de las acciones defensivas totales (entradas más intercepciones), capturando el rol recuperador del centrocampista.
- **creative_vs_defensive**: Ratio entre el promedio móvil de pases y el de entradas, que caracteriza el perfil ofensivo o defensivo del jugador.
- **dribble_confidence**: EWMA5 del porcentaje de regates completados, indicador de confianza en la conducción.
- **overall_activity**: Suma de pases, entradas, regates y conducciones, como índice global de implicación en el juego.
- **playing_time_form_combo**: Producto del porcentaje de minutos jugados (EWMA5) por los puntos Fantasy (EWMA5), penalizando a jugadores con buen rendimiento pero escasa participación.

Delanteros (DT) Las métricas de portería y las acciones defensivas (despejes, bloqueos, duelos aéreos, *clearances* temporales) se eliminan como ruido. Las variables específicas generadas son:

- **xg_overperformance**: Diferencia entre goles marcados y xG esperado en los últimos 5 partidos, detectando estados de gracia o mala racha ante el gol.
- **shot_conversion_ewma5**: Eficiencia de cara a puerta (goles por tiro), con suavizado exponencial para capturar la tendencia reciente.
- **offensive_pressure_score**: Variable compuesta que pondera el volumen de tiros y el xG generado ($shots_roll5 \times 0,4 + xg_roll5 \times 0,6$).
- **scoring_stability**: Inversa de la diferencia absoluta entre la media móvil de goles (roll5 y ewma5), midiendo la consistencia goleadora.
- **xg_per_minute_ewma**: xG generado por minuto disponible, que penaliza a jugadores con buenos números pero escasa participación.
- **goals_per_minute_ewma**: Goles por minuto disponible.
- **scoring_momentum**: Tendencia de racha goleadora calculada como la diferencia EWMA3 – EWMA5 de goles.
- **xg_momentum**: Tendencia del xG generado (EWMA3 – EWMA5).

- **pf_consistency_fw**: Inversa de la volatilidad de los puntos Fantasy en los últimos 5 partidos, midiendo la fiabilidad del delantero.

8.4.5 Variables comunes eliminadas por ruido

Con independencia de la posición, se eliminan sistemáticamente aquellas variables que los modelos penalizan con importancia nula o que introducen colinealidad sin valor predictivo adicional: `duels_won_pct_ewma5`, `duels_won_roll5`, `duels_won_ewma5`, `elite_entradas_interact` y `ratio_roles_criticos`.

8.5 Prevención del data leakage

Para garantizar que el sistema sea aplicable en el mundo real, se implementan dos salvaguardas fundamentales:

- **Desplazamiento temporal (Shift)**: Todas las estadísticas de rendimiento se desplazan una jornada hacia atrás (`shift(1)`) por jugador. Para predecir la jornada N , el modelo solo tiene acceso a variables calculadas hasta la jornada $N - 1$.
- **Validación temporal estricta**: En lugar de una validación aleatoria (K-Fold), se utiliza un corte temporal: el 80 % de los datos más antiguos se usan para entrenamiento y el 20 % más reciente para test. Esto garantiza que el modelo nunca aprende del futuro.

8.6 Estructura del dataset final

Tras el proceso de ETL y enriquecimiento, el dataset final presenta una estructura robusta con un total de **29.222 filas** (observaciones jugador/partido) y **68 columnas**. Las variables se agrupan en los siguientes bloques:

Bloque	Variables representativas
Identificación y Bio	player, nacionalidad, edad, posicion, dorsal
Contexto de Encuentro	equipo_propio, equipo_rival, is_home, temporada, jornada, fecha_partido
Rendimiento ofensivo	gol_partido, asist_partido, xg_partido, xag, tiros, tiro_puerta_partido, regates_completados, conducciones_progresivas, pases_totales, pases_completados_pct
Métricas defensivas y portería	entradas, intercepciones, despejes, bloqueos, duelos_ganados, porcentaje_paradas, psxg, goles_en_contra
Contexto competitivo	pos_clasificacion, pj, pg, pe, pp, gf, gc, pts, racha5partidos, opp_gc_ewma5, opp_shots_ewma5, fixture_difficulty_home/away
Cuotas de mercado	p_home, p_away, p_over25, p_over25_ewma5
Disponibilidad y forma	min_partido, minutes_pct_roll13/5, minutes_pct_ewma3/5, pf_roll13/5, pf_ewma3/5
Variable objetivo	target_pf_next

Cuadro 6: Bloques de variables del dataset final.

Las variables de serie temporal (rolling y EWMA) y las features específicas por posición descritas en el apartado 8.4.4 se generan dinámicamente sobre este conjunto base durante la fase de entrenamiento, y no se almacenan de forma persistente en la base de datos.

9. Implementación y desarrollo

9.1 Implementación del backend con Django

Durante la implementación del backend con *Django*, uno de los principales retos fue organizar la lógica de negocio de forma que el sistema fuese escalable, eficiente y capaz de integrar múltiples componentes como el scraping de datos, los modelos de Machine Learning, la caché y la comunicación en tiempo real.

Modelado de datos

El diseño de los modelos se planteó para representar de forma fiel el dominio del fútbol profesional y permitir tanto análisis estadístico como predicción. Para ello, se construyó una estructura relacional donde entidades como Jugador, Equipo, Partido y Jornada se interconectan mediante claves foráneas.

El núcleo del sistema es el modelo EstadísticasPartidoJugador, que almacena un conjunto muy amplio de variables (goles, asistencias, xG, duelos, métricas defensivas, etc.). Este modelo actúa como fuente principal de datos tanto para el análisis como para los modelos de Machine Learning.

Se incorporaron restricciones como `unique_together` para garantizar la integridad (evitar duplicar estadísticas de un jugador en un mismo partido), así como validadores en múltiples campos. Además, se utilizaron campos JSON en varios modelos (roles, percentiles, etc.) para permitir mayor flexibilidad sin necesidad de modificar el esquema constantemente.

Un aspecto especialmente relevante en el modelado de los datos fue la gestión de las posiciones de los jugadores. Las fuentes externas utilizadas durante el proceso de scraping presentan una gran heterogeneidad en la nomenclatura de posiciones y roles (por ejemplo, “MCF” para mediocentro defensivo), lo que dificultaba su uso directo tanto en el análisis como en los modelos predictivos.

Para solucionar este problema, se optó por normalizar todas las posiciones en cuatro categorías principales: portero, defensa, centrocampista y delantero. Esta decisión simplifica el tratamiento de los datos y permite unificar criterios en todo el sistema.

Sin embargo, surgía un inconveniente adicional: muchos jugadores son polivalentes y pueden desempeñar diferentes roles a lo largo de la temporada. Para abordar esta

situación, se implementó un método que calcula la posición más frecuente de cada jugador en función de sus apariciones reales en partidos:

Listing 3: Función para calcular posición más frecuente de un jugador

```
def get_posicion_mas_frecuente(self):
    posiciones = (
        self.estadisticas_partidos
        .filter(posicion__isnull=False)
        .values('posicion')
        .annotate(count=Count('id'))
        .order_by('-count')
        .first()
    )

    if posiciones:
        return posiciones['posicion']
    return None
```

Este enfoque permite asignar de forma dinámica una posición representativa basada en datos históricos, mitigando los problemas derivados de la polivalencia y mejorando la coherencia tanto en la visualización como en el entrenamiento de los modelos de Machine Learning.

Implementación de la API

La construcción de una API REST desde cero es una tarea compleja. Sin un framework especializado, habría que implementar manualmente:

- **Serialización:** Convertir objetos Python/ORM a JSON y validar datos entrantes.
- **Autenticación y permisos:** Verificar quién es el usuario y qué puede hacer.
- **Negociación de contenido:** Servir JSON, XML, etc. según lo que pida el cliente.
- **Paginación:** Fragmentar resultados de 10.000 registros en bloques.
- **Filtrado y búsqueda:** Aplicar criterios a consultas sin duplicar SQL.
- **Estándares HTTP:** Retornar códigos de estado correctos (200, 400, 404, 500, etc).

Django REST Framework (DRF) abstrae toda esta complejidad mediante abstracciones de alto nivel que permiten construir APIs robustas en pocas líneas de código, siguiendo patrones estándar de la industria.

La API del proyecto se divide en dos versiones paralelas:

- **API v1:** Endpoints desarrollados incrementalmente durante las primeras fases del proyecto. Conviven endpoints dispares como `/api/predecir-portero/`, `/api/cambiar-jornada/`, `/api/plantilla/`, cada uno con su propia lógica de validación y respuesta. Aunque heterogénea, esta versión sigue activa porque ciertas funcionalidades dependen de ella.
- **API v2 (RESTful):** Una arquitectura más sistemática usando ViewSets de DRF, con endpoints jerárquicos como `/api/v2/jugadores/`, `/api/v2/equipos/`, `/api/v2/clasificacion/`. Sigue principios REST: operaciones CRUD sobre recursos claramente definidos, códigos HTTP semánticos, y paginación estándar.

Decisión de diseño: mantener ambas en paralelo tiene un coste de mantenimiento, pero refleja más fielmente el proceso iterativo de desarrollo de un proyecto software.

9.1.1 Serializers

Un serializer en DRF es una clase que define cómo se transforma un objeto de *Python* (en este caso, un modelo Django) en una representación serializable (JSON) y viceversa. Además de esta transformación, define explícitamente la estructura de los datos que se intercambian entre cliente y servidor, incluyendo sus tipos y las reglas de validación que deben cumplirse. De este modo, el serializer actúa como el punto donde se formaliza el contrato de datos de la API.

A continuación se detalla la organización y funcionalidad de algunos serializadores del proyecto:

Gestión de autenticación y validación de datos La seguridad y la integridad de los datos de usuario comienzan en el RegisterSerializer. A diferencia de un ModelSerializer estándar, esta clase permite un control total sobre el proceso de validación. Implementa métodos específicos como `validate_username` y `validate_email` para garantizar la unicidad de las credenciales antes de la creación del registro. Además, el método `validate` actúa como un filtro de seguridad final, verificando la coincidencia de contraseñas y aplicando las políticas de validación nativas de Django.

Listing 4: Validación de registro con verificación de contraseñas en Django REST Framework

```
def validate(self, data):
    if data['password1'] != data['password2']:
        raise serializers.ValidationError({'password2': 'Las
            contraseñas no coinciden.'})
    try:
        validate_password(data['password1'])
    except Exception as e:
        raise serializers.ValidationError({'password1':
            list(e.messages)})
    return data
```

Perfiles de usuario y campos dinámicos Para ofrecer una respuesta rica en información sin múltiples consultas al servidor, el `UserSerializer` utiliza la capacidad de anidamiento de DRF para integrar el perfil extendido del usuario (`UserProfile`). Un aspecto avanzado es el uso de `SerializerMethodField`, que permite generar atributos de solo lectura de forma programática. En este caso, se emplea para resolver la `foto_url`, gestionando la lógica necesaria para extraer la ruta de la imagen del perfil o manejar excepciones si el recurso no existe.

Listing 5: Serializer anidado con campos dinámicos usando `SerializerMethodField`

```
class UserSerializer(serializers.ModelSerializer):
    profile = UserProfileSerializer(read_only=True)
    foto_url = serializers.SerializerMethodField()

    def get_foto_url(self, obj):
        try:
            if obj.profile.foto:
                return obj.profile.foto.url
        except Exception:
            pass
        return None
```

Relaciones sociales La gestión de amistades requiere una lógica que identifique correctamente a los participantes de la relación según quién realice la consulta. El `AmistadSerializer` resuelve esta ambigüedad accediendo al contexto de la petición para determinar cuál de los dos usuarios en el modelo `Amistad` es el “amigo” del usuario autenticado. Por otro lado, serializadores como `SolicitudAmistadSerializer` utilizan el

parámetro `source` para crear alias legibles (como `emisor_username`) a partir de relaciones de clave foránea, simplificando la estructura del JSON para el frontend.

Listing 6: Resolución ambigua de relaciones sociales en serializers mediante context

```
def _amigo(self, obj):
    req = self.context.get('request')
    return obj.usuario2 if obj.usuario1 == req.user else
        obj.usuario1

def get_amigo_username(self, obj):
    return self._amigo(obj).username
```

Serialización de recursos genéricos Finalmente, el módulo incluye serializadores de solo lectura para recursos básicos como Equipos, Jugadores y Plantillas. Estos están diseñados para ser eficientes en operaciones de listado masivo, devolviendo únicamente los campos esenciales requeridos por la interfaz, como el nombre, la nacionalidad o la formación táctica, garantizando un rendimiento óptimo de la API al minimizar la carga de datos transferidos.

9.1.2 Implementación de la lógica de negocio

La arquitectura del backend se ha diseñado siguiendo un principio de modularidad estricta y desacoplamiento. Para gestionar la alta complejidad de las reglas de negocio deportivo y la integración de modelos predictivos, el directorio `api/` se fragmentó en módulos especializados. Esta estructura no solo facilita el mantenimiento preventivo, sino que permite optimizar de forma aislada componentes críticos como la inferencia de Machine Learning, la búsqueda avanzada o el procesamiento estadístico masivo.

A continuación, se detallan las soluciones técnicas implementadas, agrupadas por su funcionalidad dentro del ecosistema de la aplicación:

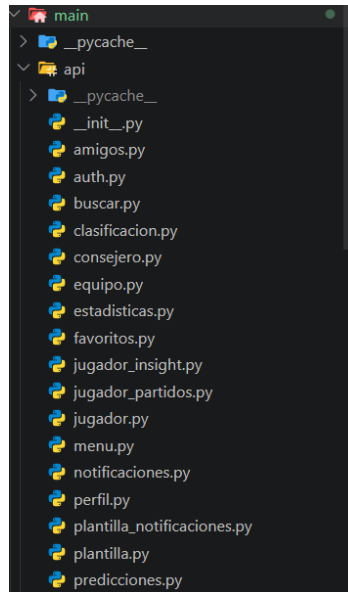


Figura 26: Captura real de la organización del módulo de api/.

9.1.2.1 Gestión de identidad y entorno social

Este bloque gestiona el ciclo de vida del usuario y su interacción social. `auth.py` centraliza la identidad asegurando la integridad mediante un helper centralizado (`_user_info`) que inyecta datos del perfil y resuelve la seguridad CSRF. `perfil.py` extiende esto con la lógica de privacidad y la gestión de aquellos sucesos que lanzan una notificación.

La capa social reside en `amigos.py`, que implementa un sistema reactivo de solicitudes que dispara avisos mediante `notificaciones.py`. Para personalizar la experiencia, `favoritos.py` persiste las preferencias de clubes, permitiendo que el resto de la API filtre rankings y calendarios de forma automática.

9.1.2.2 Motor de búsqueda, clasificación y agregación de estadísticas

Este bloque constituye el núcleo analítico del sistema, encargándose de transformar volúmenes masivos de métricas en bruto en información estructurada, rankings comparativos y narrativas de partido. El módulo `buscar.py` actúa como la interfaz de abstracción sobre el motor MeiliSearch, implementando búsquedas de texto con lógica difusa sobre jugadores y equipos, aplicando pesos dinámicos para priorizar coincidencias exactas. Complementando esta capa, `jugador_insight.py` utiliza un motor heurístico que pondera estadísticas clave según la demarcación del futbolista para generar conclusiones semánticas dinámicas, transformando datos numéricos en información de

valor deportivo.

Por su parte, `estadisticas.py` implementa un motor de búsqueda global con filtrado avanzado por jornadas. Su mayor valor técnico reside en el tratamiento de la integridad de datos mediante la detección de outliers: el sistema identifica puntuaciones anómalas superiores a 40 puntos (frecuentes por errores en fuentes externas) y las sustituye por la moda estadística del jugador para no sesgar las medias. Asimismo, resuelve el reto de la comparativa justa mediante el cálculo dinámico de percentiles relativos por posición, permitiendo determinar el rango real de un jugador dentro de su rol específico.

Listing 7: Cálculo dinámico de percentiles relativos por posición

```
for pos, jugadores_pos in jugadores_por_posicion.items():
    for stat_key in stats_to_calc:
        values = [r.get(stat_key, 0) for r in jugadores_pos]
        if values and len(values) > 1:
            for row in jugadores_pos:
                val = row.get(stat_key, 0)
                count_lower = sum(1 for v in values if v < val)
                count_equal = sum(1 for v in values if v == val)
                percentil = int(((count_lower + count_equal *
                                0.5) / len(values)) * 100)
                row[f'{stat_key}_percentil'] = max(0, min(100,
                                                            percentil))
```

Finalmente, `clasificacion.py` se encarga de reconstruir la tabla de posiciones dinámicamente para cualquier jornada de la temporada. Su funcionalidad más compleja es el método `_build_sucesos`, que realiza un cruce relacional entre las estadísticas de cada jugador y los eventos del partido para generar una cronología narrativa. Este proceso transforma datos tabulares en una lista estructurada de sucesos (goles, tarjetas y asistencias) asignados dinámicamente al contenedor local o visitante, permitiendo que el frontend renderice una lista cronológica de lo ocurrido en cada encuentro de la jornada seleccionada.

Listing 8: Reconstrucción de eventos del partido mediante cruce relacional

```
def _build_sucesos(partido_jugado, temporada):
    sucesos = {'goles_local': [], 'goles_visitante': [],
               'amarillas_local': [], ...}
    for stats in
        EstadisticasPartidoJugador.objects.filter(partido=partido_jugado).sel
        ejt =
            EquipoJugadorTemporada.objects.filter(jugador=stats.jugador,
```

```

        temporada=temporada).first()
    if not ejt: continue

    target = 'local' if ejt.equipo ==
        partido_jugado.equipo_local else 'visitante'
    nombre = f"{stats.jugador.nombre}
        {stats.jugador.apellido}".strip()

    if stats.gol_partido > 0:
        for _ in range(stats.gol_partido):
            sucesos[f'goles_{target}'].append({'nombre':
                nombre, 'minuto': stats.min_partido})

    if stats.amarillas > 0:
        sucesos[f'amarillas_{target}'].append({'nombre':
            nombre, 'minuto': stats.min_partido})

    return sucesos

```

9.1.2.3 Detalles de los equipos

El archivo `equipo.py` actúa como un motor de análisis comparativo. Para evitar el problema de rendimiento $N+1$, se implementó una precomputación de consultas (`annotate` y `Count`) que recupera la lista de jugadores por temporada en una única llamada. La vista `EquipoDetailView` construye un objeto `rival_info` dinámico que permite comparar racha y H2H (Head-to-Head) en tiempo real.

Listing 9: Construcción de objeto de análisis comparativo entre equipos rivales

```

rival_info = {
    'nombre': rival.nombre,
    'escudo': shield_name(rival.nombre),
    'racha': racha_rival_detalle,
    'goles_favor': goles_rival_favor,
    'h2h': get_h2h_historico(equipo, rival, temporada),
    'max_goleador_rival': max_goleador_rival,
    'partido_anterior': partido_anterior,
}

```

Además, el sistema muestra un top 3 dinámico según la jornada y ciertas estadísticas significativas:

Listing 10: Generación de ranking top 3

```
jugadores = sorted(jugadores_agrupados.values(), key=lambda x:
    x['total_puntos'], reverse=True)
top_3_puntos = jugadores[:3] # Lideres en puntos
top_3_minutos = sorted(jugadores, key=lambda x:
    x['total_minutos'], reverse=True)[:3] # Lideres en
    participacion
```

9.1.2.4 Perfil del jugador

jugador.py gestiona el perfil detallado y el histórico de carrera (jugador_partidos.py). El mayor reto aquí es la integridad de los datos; los cálculos de percentiles y xG generan a menudo valores compatibles con la lógica matemática pero incompatibles con JSON (NaN o Inf). Se desarrolló el `NaNInfEncoder`, un codificador personalizado que sanitiza la salida para evitar errores de parsing en el cliente cuando aparecen estos valores críticos.

Listing 11: Codificador JSON personalizado

```
class NaNInfEncoder(json.JSONEncoder):
    def iterencode(self, o, _one_shot=False):
        for chunk in super().iterencode(o, _one_shot):
            # Limpieza proactiva para evitar fallos de parsing
            # en el frontend
            chunk = chunk.replace('NaN',
                '0').replace('Infinity',
                '0').replace('-Infinity', '0')
            yield chunk
```

9.1.2.5 Generación y explicación de predicciones

predicciones.py y consejero.py forman el núcleo de IA. El orquestador de ML implementa una estrategia de predicción híbrida: durante las primeras 5 jornadas (muestra insuficiente) activa un fallback de “Media Histórica”; a partir de la 6, conmuta a inferencia pura de ML, devolviendo márgenes de error y rangos de confianza.

Listing 12: Estrategia de predicción híbrida

```
if jornada_num <= 5:
    media_puntos =
        jugador.estadisticas_partidos.aggregate(media=Avg('puntos_fantasy'))[
```

```

    return Response({'status': 'success', 'prediccion':
        round(float(media_puntos), 2), 'modelo': 'Media
        Historica'})
else:
    resultado = predecir_puntos_portero(nombre_jugador,
        jornada, modelo_tipo=modelo_tipo)
    return Response({'prediccion':
        float(resultado['prediccion']), 'margen':
        float(resultado.get('margen', 0))})

```

Se añade además una capa de IA Explicable (XAI) mediante SHAP, permitiendo que el sistema justifique cada decisión basándose y exponiendo los factores de mayor impacto.

9.1.2.6 Gestión de alineaciones

plantilla.py actúa como el motor de construcción de alineaciones, validando formaciones (ej. 4-3-3, 4-4-2, etc.) y calculando puntos en tiempo real. Se apoya en plantilla_notificaciones.py, un monitor reactivo que detecta eventos en vivo (goles, tarjetas o asistencias) de los jugadores alineados por el usuario.

Listing 13: Detección de eventos en vivo con sistema de claves únicas para evitar redundancias

```

for stat in stats:
    if stat.gol_partido > 0:
        key = f"{stat.jugador_id}_gol"
        if key not in seen:
            eventos.append({'jugador_id': stat.jugador_id,
                'evento': 'gol', 'equipo': _get_equipo(stat)})
            seen.add(key)

```

Finalmente, como medida de aviso, la API rastrea la participación reciente e inyecta alertas si un jugador suma menos de 60 minutos en los últimos 4 partidos, protegiendo al usuario ante posibles suplencias:

Listing 14: Sistema de alertas para jugadores con bajo minutaje

```

pocos_minutos_set = {
    jid for jid, mins in _min_sum.items()
    if mins and sum(mins) < 60
}
'pocos_minutos': jugador.id in pocos_minutos_set,

```

9.1.2.7 Orquestación

Dado que el cálculo de percentiles y rankings estadísticos requiere procesar miles de registros de `EstadisticasPartidoJugador`, el coste computacional es elevado. Para mantener tiempos de respuesta óptimos, se implementó un sistema de caché utilizando el backend de caché de Redis.

Las funciones que generan rankings y agregaciones almacenan los resultados en caché mediante claves únicas asociadas a la temporada y jornada, usando `cache.set`. Esto permite servir respuestas instantáneas para consultas repetidas, y el contenido se invalida automáticamente tras un periodo de expiración o cuando se detectan cambios en los datos relevantes.

9.1.3 Arquitectura reactiva y optimización de rendimiento

Tras implementar la lógica de los endpoints, se desarrollaron una serie de componentes transversales encargados de automatizar tareas, asegurar la comunicación bidireccional y garantizar la fluidez de la plataforma bajo carga de datos masiva.

9.1.3.1 Automatización mediante Signals

Para mantener el desacoplamiento del código, se utilizó el patrón Observer mediante los `Django Signals`. Esto permite que el sistema reaccione a eventos en la base de datos sin necesidad de ensuciar las vistas con lógica adicional. Se implementaron tres flujos críticos:

- Indexación en MeiliSearch: cada vez que un Jugador o Equipo es creado o actualizado, una señal `post_save` actualiza automáticamente el índice en el motor de búsqueda, garantizando que los resultados de búsqueda estén siempre sincronizados.
- Auto-predicción en segundo plano: al recibir nuevas estadísticas de un partido, el sistema dispara automáticamente el cálculo de la predicción para la siguiente jornada.
- Gestión de perfiles: creación automática de la entidad `UserProfile` al registrar un nuevo usuario en el sistema auth.

9.1.3.2 Comunicación en tiempo real con WebSockets

Para las notificaciones (goles en directo, solicitudes de amistad), una API REST convencional obligaría al cliente a realizar polling constante, saturando el servidor. Para solventarlo, se implementó una capa de WebSockets utilizando Django Channels.

Se desarrolló el `NotificacionConsumer`, un componente asíncrono que mantiene una conexión persistente con cada cliente autenticado. Cuando una señal detecta un evento relevante, este se envía al grupo de canales del usuario afectado, permitiendo una experiencia de usuario totalmente reactiva.

Listing 15: Signal de Django para emitir notificaciones en tiempo real vía WebSockets

```
@receiver(post_save, sender=Notificacion)
def notificacion_creada_ws(sender, instance, created, **kwargs):
    if created:
        channel_layer = get_channel_layer()
        async_to_sync(channel_layer.group_send)(
            f'notificaciones_{instance.usuario_id}',
            {
                'type': 'notificacion_nueva',
                'notificacion': serialize_notif(instance)
            }
        )
```

9.1.3.3 Estrategia de caching y optimización de agregaciones

Dado que el cálculo de percentiles y rankings estadísticos requiere procesar miles de registros de `EstadisticasPartidoJugador`, el coste computacional es elevado. Para mantener tiempos de respuesta que no afecten a la experiencia de usuario, se diseñó un sistema de caché basado en Redis.

Se implementó un decorador personalizado, `@cache_api_response`, que genera una clave única basada en la URL y los parámetros de la consulta. Esto permite servir datos idénticos de forma instantánea, invalidando el contenido automáticamente según el ciclo de actualización de las jornadas.

9.1.4 Arranque y ciclo de vida

Para garantizar la robustez del sistema desde el momento del despliegue, se ha implementado una lógica de inicialización avanzada en el componente `apps.py`. Este módulo

no solo registra la configuración de la aplicación, sino que actúa como un orquestador de integridad, asegurando que todos los servicios dependientes (Base de Datos, MeiliSearch y Modelos de ML) estén sincronizados y listos para operar.

9.1.4.1 Inicialización asíncrona y gestión de cold start

Uno de los retos técnicos resueltos fue evitar el bloqueo del servidor durante el arranque. Si el sistema realizara comprobaciones pesadas de datos en el hilo principal, el servidor web podría interrumpir el proceso por timeout.

Para solventarlo, se implementó un sistema de inicialización en segundo plano mediante `threading.Thread`. Al arrancar Django, el método `ready()` dispara un hilo que ejecuta un pipeline de 5 pasos sin impedir que el servidor comience a escuchar peticiones.

Listing 16: Inicialización asíncrona en hilo durante startup de Django

```
def ready(self):
    if 'runserver' in sys.argv:
        init_thread =
            threading.Thread(target=self._initialize_data,
                            daemon=True)
        init_thread.start()
```

9.1.4.2 Pipeline de arranque

La función `_initialize_data` ejecuta la lógica de iniciar el servidor, dividida en cinco fases:

1. Auditoría de persistencia: comprueba el volumen de registros en Partido y Estadísticas. Si detecta una base de datos vacía, marca el sistema como “incompleto”.
2. Carga masiva de datos: si la comprobación falla, invoca el módulo de scraping para poblar la base de datos desde cero. Un detalle técnico relevante es la desconexión temporal de Signals durante esta fase; esto evita que se disparen miles de hilos de predicción innecesarios durante una carga masiva, optimizando el uso de CPU y memoria.
3. Verificación de roles: asegura que los campos JSON de roles (corners, penaltis) estén poblados, lanzando procesos de limpieza si detecta una carencia superior al 50 %.

4. Sincronización con el motor de búsqueda: verifica la disponibilidad del cliente MeiliSearch y fuerza una reindexación completa de jugadores y equipos.
5. Pre-generación de inferencias: identifica jornadas de la temporada actual sin predicciones y lanza el comando de generación para que el usuario disponga de datos predictivos desde el primer minuto.

9.1.4.3 Pre-carga de módulos de Machine Learning

Para optimizar el rendimiento de la primera petición (latencia de “primer acceso”), se ha incluido una lógica de pre-importación del módulo `predcir.py`. Esto asegura que el grafo de computación y las librerías pesadas (como Scikit-Learn o XGBoost) ya estén en la memoria RAM del proceso worker antes de que llegue la primera consulta del usuario, reduciendo el tiempo de respuesta inicial de varios segundos a milisegundos.

9.2 Implementación del scraping de datos

La construcción del dataset maestro ha sido uno de los retos de ingeniería más exigentes del proyecto. A diferencia de otros dominios donde existen APIs públicas, el ecosistema del fútbol requiere la extracción de datos desde múltiples fuentes web heterogéneas. La implementación se ha estructurado en scripts especializados que gestionan la descarga, el bypass de protecciones y el parseo de estructuras HTML complejas, garantizando la integridad de más de 23.000 registros históricos.

9.2.1 Origen de los datos y fuentes

Se han integrado tres fuentes principales para cubrir todas las dimensiones del sistema:

- **FBRef**: fuente principal de métricas avanzadas (xG, PSxG, progresiones, estadísticas defensivas y de portería).
- **FutbolFantasy**: utilizada exclusivamente para la obtención de los puntos obtenidos por cada jugador en la plataforma oficial.
- **Transfermarkt**: fuente de contexto competitivo, empleada para extraer las rachas de los equipos, datos de estadios, calendario y nacionalidades.

9.2.2 Gestión de barreras técnicas y bloqueos

Durante el desarrollo, se detectaron barreras críticas de seguridad que obligaron a diseñar estrategias de mitigación específicas:

- Bloqueos de IP en FBRef: este portal implementa políticas extremadamente estrictas de rate limiting. Ante la imposibilidad de realizar un scraping masivo automatizado sin ser baneado, se tomó una decisión de diseño basada en la gestión de riesgos: la descarga controlada y local de los archivos HTML. Se procesaron de forma secuencial los HTML de los partidos de las últimas tres temporadas (23/24, 24/25 y la actual 25/26), asegurando la persistencia de los datos sin comprometer la disponibilidad del sistema.
- Evasión de Cloudflare en FutbolFantasy: el sitio bloquea agentes no humanos. Se implementó el módulo `descargar_puntos.py` utilizando la librería `cloudscraper`, que permite simular una huella de navegador real, gestionando automáticamente los desafíos de JavaScript y cabeceras User-Agent.

9.2.3 Algoritmo de reintentos

Para garantizar la robustez de las descargas en *FutbolFantasy*, se desarrolló un algoritmo con reintentos y esperas aleatorias. Esto evita que el servidor identifique patrones de automatización y permite recuperarse de errores temporales de red.

Listing 17: Algoritmo de reintentos para descargas robustas

```
def descargar_html_con_reintentos(url, max_intentos=5):
    intento = 0
    while intento < max_intentos:
        intento += 1
        try:
            resp = scraper.get(url, timeout=30)
            if resp.status_code == 200:
                return resp.text
        except Exception as e:
            delay = min(3.0 * (2 ** (intento - 1)), 30.0) +
                random.uniform(0, 1.5)
            time.sleep(delay)
    raise RuntimeError(f"Fallo tras {max_intentos} intentos en
                        {url}")
```

9.2.4 Módulos técnicos y responsabilidades

La lógica de scraping se divide en archivos especializados para maximizar la mantenibilidad del pipeline:

A. descargar_puntos.py Este módulo representa el punto de entrada del pipeline, encargado de la descarga masiva de documentos HTML. Su arquitectura contempla la evasión de sistemas de seguridad perimetral comunes en portales de alta demanda.

- Gestión de sesiones: implementa la librería `cloudscraper` para mitigar los bloqueos por desafíos de JavaScript de Cloudflare, simulando un entorno de navegación real (User-Agent dinámico, cabeceras de conexión persistente).
- Estrategia de reintentos: utiliza un algoritmo de backoff exponencial con varianza aleatoria. Esto evita la saturación del servidor y reduce la detección de patrones de tráfico automatizado.
- Comprobaciones: el sistema verifica la integridad y existencia de archivos locales antes de iniciar la descarga.

Listing 18: Reintento con espera aleatoria para mitigación de detección de bots

```
delay_base = min(backoff_inicial * (2 ** (intento - 1)),
                 backoff_max)
delay = delay_base + random.uniform(0, 1.5)
time.sleep(delay)
```

Nota de implementación: debido a las políticas de seguridad extremadamente restrictivas detectadas durante la fase de ejecución (bloqueos por IP tras descargas masivas), este script sirvió como base de pruebas, aunque la obtención del grueso de los datos históricos se complementó con descargas manuales controladas para asegurar la integridad de la base de datos.

B. commons.py Es el motor de procesamiento de texto que garantiza la cohesión entre diferentes fuentes de datos.

- Limpieza de cadenas: implementa expresiones regulares para eliminar información residual en los nombres de los jugadores (como los minutos jugados que FBRef concatena al nombre en ciertos nodos, ej: 90+4').

- Normalización: utiliza la normalización NFD (Normalization Form Decomposition) para separar caracteres base de sus acentos, eliminando sistemáticamente los diacríticos. Esto es vital para asegurar que “Martínez” y “Martinez” sean tratados como el mismo registro.

C. fbref.py Este es el script más complejo a nivel de lógica de negocio, ya que debe unificar múltiples dimensiones estadísticas de un mismo encuentro y enriquecer los datos con atributos derivados. Su arquitectura está centrada en la integración de datos heterogéneos y la verificación de integridad por partido.

- Procesado de tablas: FBRef estructura los datos por encuentro en múltiples tablas HTML con identificadores específicos (tabla ID contiene “_home_” o “_away_” según el equipo). La función `parsear_tabla_fbref()` emplea BeautifulSoup para localizar tables dinámicamente, normalizar nombres de columnas y combinar filas de múltiples tablas usando el nombre normalizado del jugador como clave de unión (inner join en memoria).
- Manejo especializado para porteros: dado que los porteros comparten tablas parcialmente distintas con métricas específicas (PSxG, Save %, GA90), la función `buscar_estadisticas_portero()` implementa una estrategia jerárquica: búsqueda por nombre exacto → búsqueda por apellido. Esto garantiza que el 100 % de porteros sean asociados correctamente incluso cuando existen inconsistencias menores en los datos de FBRef.
- Extracción de calendario: además de las métricas, parsea los resultados históricos y las fechas de los encuentros para construir la base del calendario de la competición.

D. transfermarkt.py Mientras FBRef aporta datos intrínsecos del jugador, este módulo recupera variables extrínsecas mediante `parse_tabla_jornada_transfermarkt()`, extrayendo de forma confiable:

- Posición en clasificación (PJ, PG, PE, PP, puntos)
- Diferencia de goles (GF, GC, DF, etc.)

E. roles.py Módulo avanzado diseñado para detectar jugadores que destacan especialmente en categorías estadísticas específicas (córners, asistencias, etc.).

- Parseo de elementos ocultos: los datos de rankings en FBRef se cargan de forma asíncrona o se encuentran dentro de comentarios HTML. `roles.py` implementa

una técnica de extracción de comentarios mediante BeautifulSoup para acceder a esta información.

- Mapeo semántico: transforma rankings numéricos en descripciones narrativas (insights), permitiendo que el sistema genere automáticamente conclusiones para la interfaz de usuario.

Listing 19: Extracción de nodos comentario HTML con BeautifulSoup para datos ocultos

```
comments = soup.find_all(string=lambda text: isinstance(text,
    Comment))
for c in comments:
    if 'id="div_leaders"' in c:
        frag_soup = BeautifulSoup(c, "lxml")
```

9.2.5 Identificación de roles

Este proceso es vital para el enriquecimiento del dataset. El sistema localiza a los jugadores que ocupan el top 10 en rankings de eficiencia (goles por disparo, pases al último tercio, intercepciones, etc.). Estos datos permiten al modelo de predicción asignar un peso mayor a jugadores que, independientemente de sus puntos brutos, demuestran una importancia táctica superior en la liga.

Los roles se almacenan en un objeto JSON donde la clave principal es el nombre normalizado del jugador, facilitando un acceso rápido durante el entrenamiento. Cada registro contiene una lista de los rankings donde el jugador es protagonista, almacenando la posición y el valor absoluto de la métrica.

Ejemplo de formato JSON generado:

Listing 20: Estructura JSON de roles y rankings de eficiencia para enriquecimiento de features

```
{
  "lamine yamal": [
    {"regates_exitosos": [1, 45.2]},
    {"pases_clave": [3, 12]},
    {"asistencias": [2, 7]}
  ],
  "antonio rudiger": [
    {"despejes": [5, 88]},
    {"intercepciones": [10, 32]}
  ]
}
```

}

9.2.6 Orquestación del scraping

El paso final de scraping es el script `popularDB.py`, que representa la fase de consolidación de datos. Este script ejecuta secuencialmente:

1. Validación de disponibilidad local: comprueba la existencia de los HTML de FBRef descargados previamente, evitando depender de peticiones masivas en tiempo real y reduciendo el riesgo de bloqueos por IP.
2. Carga de entidades base: crea en la base de datos las temporadas, equipos y jornadas necesarias para estructurar el resto de la información.
3. Ensamblaje de estadísticas por partido y jugador: integra los datos procedentes de `fbref.py`, `transfermarkt.py` y `roles.py` en registros unificados con múltiples atributos por jugador-partido.
4. Normalización de datos: aplica las reglas de homogeneización definidas en el sistema para asegurar consistencia entre fuentes antes de la persistencia.
5. Agregación histórica: genera el rendimiento histórico del jugador a partir de sus estadísticas acumuladas en la temporada.
6. Persistencia final: guarda los datos intermedios en CSV y los consolida en PostgreSQL, cerrando así la fase de ingestión.

Esta orquestación permite desacoplar la extracción de datos de su consolidación, facilitando la trazabilidad, la recuperación ante errores y la reutilización de los distintos módulos del pipeline.

9.3 Implementación del matching de entidades

El primer paso es reducir el “ruido” textual. Antes de cualquier comparación, el sistema aplica una normalización agresiva:

- Conversión a minúsculas y eliminación de tildes.
- Eliminación de caracteres especiales (guiones, puntos).
- Limpieza de minutos: FBRef a veces incluye el minuto del evento en el nombre del jugador en el HTML; el sistema lo detecta y limpia mediante expresiones regulares

a través del módulo `commons.py`.

Listing 21: Normalización de texto y limpieza de minutos para entity matching

```
def normalizar_texto(texto):
    texto = str(texto).lower().strip()
    texto = unicodedata.normalize("NFD", texto)
    texto = "".join(c for c in texto if unicodedata.category(c)
                    != "Mn")
    texto = re.sub(r"[-.]", " ", texto)
    texto = re.sub(r"\s+", " ", texto)
    return texto.strip()

def limpiar_minuto(nombre: str) -> str:
    if not nombre: return nombre
    nombre = nombre.replace("+", "").replace("-", "").strip()
    nombre = re.sub(r"\s*\d+(?:\+\d+)?'?$", "", nombre).strip()
    return nombre
```

9.3.1 Alias por temporada y casos especiales

Existen casos donde la normalización no es suficiente (ej. “Pepelu” no se parece a “José Luis García Vaya”). El sistema utiliza un diccionario de alias que mapea nombres conocidos según la temporada y el equipo. Esto permite interceptar estos casos antes de que lleguen a la lógica probabilística.

Ejemplos de flujo:

- (Alias) Isaac Palazón Camacho → Isi (Rayo Vallecano)
- (Normalización) Alexander Sørloth → Sorloth (Villarreal/Atlético)

9.3.2 Generación de propuestas de matching

El proceso no compara un jugador contra toda la base de datos, lo cual generaría falsos positivos masivos. En su lugar, el matching se acota al contexto del partido:

- Se extraen los jugadores del partido en FBRef.
- Se identifican los candidatos de Fantasy que pertenecen exclusivamente a los dos equipos enfrentados.

- Se genera una lista de propuestas que incluye el score de confianza y metadatos como posición y minutos.

Listing 22: Búsqueda contextual de candidatos por partido usando fuzzy matching acotado

```
mejor_norm, mejor_score = obten_coincidencia_nombre(
    nombre_html_norm,
    nombres_fantasy_norm,
    equipo_norm=equipo_fb_norm,
)
```

9.3.3 Lógica principal y Fuzzy Matching

La función `obten_coincidencia_nombre` implementa una estrategia de decisión a tres niveles, no una simple aplicación de un algoritmo único. Este enfoque es fundamental porque diferentes tipos de discrepancias requieren diferentes soluciones.

Nivel 1: Regla de apellidos críticos Ciertos apellidos son extremadamente comunes en el fútbol (García, González, López, Fernández, Rodríguez, etc.). Para estos apellidos, un match “parcial” (donde solo coincide el apellido) es prácticamente inútil y genera colisiones masivas.

Caso real: en el Athletic Club ha habido dos hermanos con apellido Williams (Nico Williams e Iñaki Williams). Si FBRef reporta “Nico Williams” y nuestro algoritmo busca usando solo el apellido, podría devolver erróneamente “Iñaki Williams” con una puntuación engañosamente alta.

Implementación: el sistema mantiene un diccionario de apellidos críticos. Si el nombre de un jugador contiene uno de estos apellidos, se requiere una coincidencia más exigente (nombre completo o al menos dos tokens coincidentes) antes de aceptar el match.

Nivel 2: Coincidencia por prefijo/sufijo Cuando el nombre es un único token y no tiene un apellido crítico.

En el fútbol es habitual usar nombres acortados (“Luis” en lugar de “Luis García”) y apodos (“Pepelu” en lugar de “José Luis García”).

Implementación: para un nombre de un solo token, el sistema busca si coincide con el inicio o fin de algún candidato. Si “Araújo” está en Fantasy y FBRef trae “Ronald

Araújo”, el algoritmo lo detecta como match prefijo/sufijo con un score moderadamente alto y se realiza el match.

Nivel 3: Fallback a WRatio Si los dos niveles anteriores no producen un resultado confiable, se recurre al algoritmo de fuzzy matching más robusto: **WRatio**.

Listing 23: Fallback a WRatio para matching robusto

```
mejor_candidato, score, _ = process.extractOne(  
    nombre_fbref_normalizado,  
    lista_jugadores_fantasy,  
    scorer=fuzz.WRatio  
)  
  
if mejor_candidato and score >= UMBRAL_MATCH:  
    return mejor_candidato, score
```

9.3.3.1 Justificación técnica: ¿Por qué WRatio frente a Levenshtein?

La Distancia de Levenshtein es el enfoque clásico: mide cuántos cambios letra a letra hay que hacer para que una palabra sea igual a otra. Para deportes es demasiado rígido.

Ejemplo comparativo:

Listing 24: Comparación de distancia Levenshtein vs WRatio

```
"Isaac Palazon" vs "Isaac Palazon Camacho"  
Levenshtein: 9 caracteres de diferencia -> 57.1% similitud (por  
debajo del umbral)
```

Con Levenshtein, muchas coincidencias legítimas quedarían por debajo del umbral de aceptación.

WRatio no es un único algoritmo, sino una composición inteligente de varios sub-algoritmos con escalado de pesos. El sistema elige cuál es más adecuado según el contexto:

1. **Partial Ratio:** busca si una cadena está contenida exactamente en la otra.

Listing 25: Partial Ratio: búsqueda de subcadenas

```
"Isaac Palazon" es subcadena exacta de "Isaac Palazon  
Camacho"
```

```
Partial Ratio = 100 %
```

Cuando una fuente usa el nombre completo y otra usa una versión abreviada, el Partial Ratio lo detecta instantáneamente.

2. **Token Sort Ratio:** divide en tokens (palabras), ordena alfabéticamente y compara.

Listing 26: Token Sort Ratio: ordenación de tokens

```
"Garcia Joan" vs "Joan Garcia"
-> Ambos se ordenan como ["Garcia", "Joan"]
Token Sort Ratio = 100 %
```

Este paso es clave para datos procedentes de diferentes portales donde el orden de los nombres es impredecible.

3. **Escalado inteligente de pesos:** el prefijo “W” (Weighted) indica que el algoritmo no promedia los sub-ratios, sino que elige internamente cuál es más fiable según la longitud de los nombres y el tipo de diferencias encontradas.

9.3.4 Resolución de conflictos y desempates

Cuando la lógica difusa propone un mismo jugador de Fantasy para múltiples registros de FBRef, el sistema aplica el siguiente orden de criterios:

- **Prioridad por minutos:** se selecciona la propuesta con más minutos jugados (evita registros residuales con tarjetas en el banquillo).
- **Prioridad por puntos:** si hay empate de minutos o scores similares (<5 % diferencia), se elige el candidato con mayor puntuación Fantasy.
- **Consistencia de apellido:** se valida que el apellido de la propuesta coincida sustancialmente con el del registro original (umbral = 0.85), descartando falsas coincidencias.

9.3.5 Integración con el flujo de scraping

El matching no es un proceso aislado, sino que está integrado en el pipeline de FBRef:

1. **Extracción:** se parsean las tablas de summary, passing, defense, etc.

2. **Propuestas:** se ejecuta `generar_propuestas` para cada equipo, filtrando por contexto de partido.
3. **Resolución:** se consolidan las asignaciones aplicando la cascada de criterios de negocio y se genera el DataFrame final.
4. **Casos especiales (banquillo):** mediante `completar_fantasy_sin_match`, se añaden aquellos jugadores que no aparecen en las estadísticas de juego de FBRef pero sí en Fantasy por haber recibido una tarjeta en el banquillo.

9.4 Implementación del sistema de Machine Learning

El núcleo predictivo del proyecto se basa en un ecosistema de modelos supervisados especializados por posición. Dado que la naturaleza estadística de un portero (centrada en paradas y goles encajados) difiere radicalmente de la de un delantero (basada en xG y remates), se optó por una arquitectura de modelos desacoplados.

9.4.1 Arquitectura del BaseTrainer

Para garantizar la mantenibilidad y evitar la duplicidad de código, se implementó una clase abstracta `BaseTrainer`. Esta clase centraliza las tareas comunes: división temporal, generación de visualizaciones, cálculo de métricas y la ejecución del motor de optimización.

Listing 27: Clase base abstracta para trainers de modelos ML con split temporal

```
class BaseTrainer(ABC):
    """Clase base para el entrenamiento de modelos por
       posicion."""
    @staticmethod
    def temporal_split(X, y, test_size: float):
        split_idx = int(len(X) * (1 - test_size))
        return X.iloc[:split_idx], X.iloc[split_idx:],
            y.iloc[:split_idx], y.iloc[split_idx:]

    @abstractmethod
    def run(self) -> None:
        """Cada entrenador especifico (Portero, Defensa, etc.)
           implementa su flujo."""
        raise NotImplementedError
```

9.4.2 Estrategia de validación y Grid Search

El mayor reto técnico fue evitar la fuga de información. En datos deportivos, el rendimiento de un jugador está ligado a su racha actual; por ello, se prohibió el uso de `train_test_split` aleatorio, utilizando en su lugar un corte temporal estricto (80 % pasado para entrenar, 20 % futuro para testar).

Sobre este split temporal, se orquestó una búsqueda exhaustiva de hiperparámetros mediante `GridSearchCV` con una validación cruzada de 5 folds. Se evaluaron cuatro familias de algoritmos para determinar cuál capta mejor la variabilidad de los puntos Fantasy:

- Random Forest (RF): excelente para capturar relaciones no lineales y robusto frente a outliers.
- XGBoost (XGB): utilizado por su alta capacidad predictiva mediante gradiente boosting.
- Ridge y ElasticNet: modelos lineales con regularización $L1$ y $L2$, ideales para conjuntos de datos con muchas variables correlacionadas.

Espacio de búsqueda de hiperparámetros El sistema no se limitó a parámetros por defecto, sino que exploró un espectro amplio para asegurar la convergencia hacia el error mínimo. En total, se evaluaron 1.790 combinaciones de hiperparámetros por cada posición, desglosadas en 288 para Random Forest, 768 para XGBoost, 14 para Ridge y 720 para ElasticNet. Las configuraciones probadas en el archivo `common_trainer.py` incluyeron:

Listing 28: Espacio de búsqueda exhaustivo de hiperparámetros para grid search multi-algoritmo

```
GRID = {
    'RF': {
        'n_estimators': [200, 300, 400],
        'max_depth': [10, 20, 30],
        'min_samples_split': [2, 3, 5, 7],
        'min_samples_leaf': [2, 3, 4, 5],
        'max_features': ['sqrt', 'log2'],
    },
    'XGB': {
        'max_depth': [5, 7],
        'learning_rate': [0.1, 0.15],
```

```

        'n_estimators': [300, 500],
        'subsample': [0.7, 0.9],
        'gamma': [0.25, 0.5],
        'reg_alpha': [0.05, 0.1],
        'reg_lambda': [1.0, 2.0],
    },
    'Ridge': {
        'regresor__alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0,
                             1000.0],
    },
    'ElasticNet': {
        'regresor__alpha': [0.0001, 0.001, 0.01, 0.1, 1.0,
                             10.0],
        'regresor__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 1.0],
        'regresor__max_iter': [5000, 10000],
    },
}

```

Coste computacional y rendimiento Dada la profundidad del grid, el uso de una validación cruzada de 5 folds y la generación masiva de variables temporales, el proceso exigió una alta carga de cómputo. Al multiplicar las 1.790 combinaciones por las 5 particiones de datos, el sistema ejecutó 8.950 entrenamientos individuales (fits) por cada posición, lo que suma un total de 35.800 modelos evaluados para el conjunto del ecosistema.

Este despliegue supuso un tiempo aproximado de 3 horas por posición, totalizando unas 12 horas de computación intensiva. Para mitigar este coste, se empleó procesamiento paralelo (`n_jobs=-1`), distribuyendo la carga de trabajo entre todos los núcleos de la CPU disponibles.

9.4.3 Robustez y persistencia

Durante el entrenamiento de modelos lineales como ElasticNet, se gestionaron problemas de convergencia habituales en datasets con alta dimensionalidad mediante `warnings.catch_warnings`, evitando que el flujo se detuviera por iteraciones no completadas.

La persistencia de los artefactos se resolvió mediante la serialización en archivos `.pkl` (pickle). Se tomó la decisión estratégica de conservar siempre los cuatro modelos entre-

nados, incluso en escenarios donde modelos lineales como Ridge presentaran un MAE ligeramente inferior.

9.4.4 Criterios de evaluación y selección de métricas

Para determinar la eficacia de los modelos y seleccionar el algoritmo definitivo para cada posición, se estableció un marco de evaluación dual. Este enfoque combina métricas dimensionales (que miden la magnitud del error en puntos reales) y métricas adimensionales (que miden la capacidad del modelo para ordenar a los jugadores por calidad), permitiendo una visión holística del rendimiento predictivo.

1. Métricas dimensionales: MAE y RMSE Se seleccionaron el Error Absoluto Medio (MAE) y la Raíz del Error Cuadrático Medio (RMSE) para medir la precisión de las predicciones en la unidad de medida del problema: los puntos Fantasy.

- MAE (Mean Absolute Error): se estableció como la métrica principal de decisión. Su valor es directamente interpretable para el usuario final: representa cuántos puntos arriba o abajo suele equivocarse el modelo por cada jugador. Al no elevar los errores al cuadrado, el MAE es más robusto frente a eventos imprevisibles del fútbol (como una expulsión inesperada o una lesión en el minuto 5), evitando que un solo registro anómalo penalice injustamente la validez global del modelo.
- RMSE (Root Mean Squared Error): se utilizó como métrica de control para identificar la presencia de errores de gran magnitud. Un RMSE mucho más elevado que el MAE indicaría que el modelo comete fallos muy abultados en ciertos perfiles de jugadores, lo que permitió refinar el filtrado de outliers antes de la producción.

2. Métrica adimensional: Coeficiente de Spearman En el ámbito de los juegos Fantasy, la utilidad de un modelo no reside únicamente en su precisión numérica, sino en su capacidad para establecer una jerarquía correcta. Para un usuario, es más valioso identificar qué jugadores ocuparán los primeros puestos de rendimiento en la jornada que conocer la cifra exacta de sus puntos.

Para evaluar esta capacidad de ordenación, se utilizó el Coeficiente de Correlación de Spearman. A diferencia de otras métricas, Spearman no compara los valores brutos, sino que los transforma en rangos o posiciones (1^o , 2^o , 3^o ...). Técnicamente, evalúa la relación monótona entre las predicciones y los resultados reales:

- **Funcionamiento:** el coeficiente cuantifica la disparidad entre el ranking predicho y el ranking real; una coincidencia exacta en la jerarquía de los jugadores resulta en una correlación de +1, validando la naturaleza monótona de la predicción.
- **Utilidad real:** permite validar el sistema como un motor de recomendación. Un modelo puede tener un error de puntos elevado (por ejemplo, predecir 15 puntos cuando el jugador suma 8), pero si mantiene a ese jugador en el primer puesto de la lista de recomendados y efectivamente es el mejor de la jornada, el valor de decisión para el usuario es máximo.
- **Robustez:** al trabajar con rangos, es una métrica insensible a los valores atípicos y a la escala de la jornada. Independientemente de si ha sido una jornada de muchas o pocas anotaciones, lo que se audita es que la “élite” predicha coincida con la “élite” real.

Esta métrica adimensional (con rango de -1 a 1) fue determinante para confirmar que los modelos son capaces de orientar al usuario en la confección de su alineación, garantizando que los jugadores con mayor probabilidad de éxito aparezcan sistemáticamente en las posiciones superiores del ranking.

3. Justificación del rechazo del coeficiente de determinación (R^2) Durante la fase de validación, se descartó el uso del coeficiente R^2 como criterio de selección. El motivo principal es la alta aleatoriedad intrínseca del fútbol.

Debido a que el rendimiento deportivo está sujeto a una volatilidad extrema que ninguna variable estadística puede capturar por completo (rebotes, decisiones arbitrales, estado anímico), los valores de R^2 en este dominio suelen ser sistemáticamente bajos, situándose habitualmente por debajo de 0.2 . Basar el éxito del proyecto en el R^2 habría llevado al rechazo erróneo de modelos que, a pesar de no explicar toda la varianza del juego, muestran una excelente capacidad de ranking (Spearman alto) y un error absoluto contenido (MAE bajo).

9.5 Implementación del feature engineering

El éxito de los modelos predictivos no reside únicamente en la elección del algoritmo, sino en la capacidad de transformar datos brutos de partidos en señales informativas de alto valor. El feature engineering se diseñó como una capa de transformación intermedia que combina la ingeniería de series temporales con la creación de variables sintéticas de dominio específico.

Mediante la síntesis de métricas base, se han generado indicadores complejos de eficiencia y ratio que permiten capturar la calidad técnica de un jugador más allá del volumen estadístico. La implementación técnica se apoya en el procesamiento vectorial con pandas y numpy, priorizando la eficiencia computacional y la eliminación de sesgos estadísticos.

9.5.1 Generación automatizada de variables temporales

La base de la capacidad predictiva está en cómo evoluciona cada jugador a lo largo de la temporada. Para ello, se desarrolló en **BaseTrainer** un generador genérico de variables temporales sobre ventanas deslizantes. La idea clave fue trabajar siempre con registros desplazados mediante `shift(1)` por jugador, de forma que cada fila use únicamente información anterior y se elimine cualquier fuga temporal.

En la práctica, el sistema combina dos tipos de agregación: medias móviles y EWMA. Las medias móviles resumen el comportamiento reciente en ventanas fijas de 3 y 5 partidos, mientras que la EWMA da más peso a los encuentros más cercanos y permite captar cambios de forma con mayor sensibilidad. Durante el desarrollo se valoró una ventana de 7 partidos, pero quedó relegada porque suavizaba demasiado la señal y hacía menos visible la reacción ante cambios de rol, minutaje o rendimiento.

Listing 29: Agregación temporal con medias móviles y EWMA para capturar evolución de jugadores

```
nuevas_cols[col_roll] = df.groupby("player")[columna].transform(
    lambda x: x.shift().rolling(ventana, min_periods=1).mean()
).fillna(default_value)
nuevas_cols[col_ewma] = df.groupby("player")[columna].transform(
    lambda x: x.shift().ewm(span=ventana, adjust=False).mean()
).fillna(default_value)
```

9.5.2 Heurísticas de limpieza

Uno de los principales problemas del feature engineering fue la calidad irregular de los datos. Antes de generar variables nuevas, el pipeline convierte columnas a formato numérico, reemplaza valores no válidos y elimina registros poco informativos. En concreto, los NaN e infinitos se corrigen con estrategias de imputación por mediana o relleno controlado según la fase del procesamiento, evitando que el entrenamiento se rompa por valores incompatibles.

Además, `diagnosticar_y_limpiar` aplica dos filtros importantes. Primero, descarta partidos con menos de 10 minutos, porque esos registros suelen producir métricas poco estables o directamente engañosas. Segundo, exige un mínimo de 5 partidos por jugador para que las variables temporales tengan una base histórica mínimamente fiable. Esta decisión reduce ruido y evita que una sola actuación aislada distorsione medias, rankings o medias exponenciales.

9.5.3 Especialización asimétrica por posición

El sistema no reutiliza las mismas variables para todas las posiciones. En su lugar, cada modelo incorpora únicamente las señales que tienen sentido futbolístico para ese perfil. En porteros, el pipeline prioriza variables como `porcentaje_paradas`, `psxg`, `save_pct_roll5` y combinaciones de forma y disponibilidad. En defensas, se enfatizan entradas, intercepciones, despejes y duelos. En mediocampistas, el foco se desplaza hacia pases clave, asistencias, regates y continuidad de minutos. En delanteros, se da más peso a goles, tiros a puerta, penaltis y eficiencia ofensiva.

Esta especialización evita que el modelo aprenda ruido de variables irrelevantes para cada demarcación. También mejora la interpretabilidad, porque cada conjunto de features refleja la lógica real del rol en el campo en lugar de forzar una representación homogénea de todos los jugadores.

9.5.4 Inyección de contexto externo

Para enriquecer el dataset con contexto extra, se integraron dos tipos de señales. La primera procede de los roles destacados de FBRef. Estos rankings se convierten en variables numéricas a través de un factor de posición que penaliza progresivamente los puestos inferiores, de manera que un primer puesto pesa mucho más que un décimo o un vigésimo. La segunda procede del contexto de partido, con variables derivadas de cuotas y dificultad del partido, que ayudan a estimar el escenario competitivo antes del encuentro.

Este bloque fue especialmente útil para introducir señales sin depender solo de estadísticas acumuladas. Por ejemplo, el módulo `role_enricher.py` crea interacciones entre la condición de jugador destacado y la intensidad real de su rol, como `elite_paradas_interact` en porteros o `elite_entradas_interact` en defensas. Además, se añadieron variables de disponibilidad-forma, como la combinación entre minutos recientes y producción reciente, porque en Fantasy un jugador solo aporta si juega y mantiene continuidad.

Listing 30: Inyección de interacciones entre elite y rol para enriquecer representación de características

```
df["elite_paradas_interact"] = (  
    df["es_portero_elite"].astype(float) *  
    df["rol_paradas_ponderado"]  
)
```

9.5.5 Selección de características mediante Spearman

Dada la alta dimensionalidad tras generar ventanas temporales (más de 150 variables por posición), se aplicó una selección de características basada en la Correlación de Spearman. Se descartaron sistemáticamente todas las variables con un $|\rho| < 0,02$, reduciendo la complejidad del modelo y mejorando el tiempo de entrenamiento.

9.6 Implementación de IA Explicable (XAI)

Uno de los mayores retos al trabajar con modelos de Machine Learning complejos es su opacidad, el problema de la “caja negra”. Para un usuario de la plataforma, recibir una predicción de “8.5 puntos” sin contexto no genera confianza y dificulta la toma de decisiones.

Para resolver este problema, se ha implementado una capa de IA Explicable (XAI). Esta arquitectura no solo entrega una predicción numérica, sino que desglosa de forma determinista qué factores han influido y en qué medida en cada cálculo individual, garantizando la transparencia del sistema.

9.6.1 Fundamentos de la metodología SHAP

El sistema utiliza la librería SHAP. Bajo este enfoque, la predicción final del modelo se considera un “premio” y las variables estadísticas del jugador (goles, minutos, precisión de pases, etc.) se consideran los “jugadores” que cooperan para obtener dicho premio.

SHAP calcula los Valores de Shapley, que representan el promedio de la contribución marginal de cada característica a través de todas las permutaciones posibles de las variables. Matemáticamente, el sistema descompone la predicción mediante una función aditiva:

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i$$

Donde:

- $f(x)$ es la predicción final.
- ϕ_0 es el valor base del modelo (la media de salida del conjunto de entrenamiento).
- ϕ_i es la contribución (positiva o negativa) de cada feature i .

A diferencia de los métodos de importancia de variables tradicionales, que ofrecen una visión global del modelo, SHAP proporciona explicaciones locales. Esto permite al sistema explicar por qué un delantero tiene una predicción alta debido a su racha de goles, mientras que para otro puede ser debido exclusivamente a la debilidad defensiva del rival en esa jornada concreta.

9.6.2 Motor de explicación unificado

Para gestionar la heterogeneidad de los modelos (árboles y lineales) y las diferentes posiciones, se desarrolló el módulo `explicaciones.py`, que actúa como una capa de abstracción entre los valores matemáticos de SHAP y el lenguaje natural deportivo.

- Variedad de modelos: el sistema detecta automáticamente la arquitectura del modelo cargado. Si es un bosque de decisión (Random Forest o XGBoost), instancia un `TreeExplainer`; si es un modelo lineal (Ridge o ElasticNet), utiliza un `LinearExplainer`.
- Mapeo semántico: se implementó un diccionario exhaustivo (`EXPLICACIONES_FEATURES`) que asocia cada variable técnica con una narrativa de impacto positivo y negativo, permitiendo que el usuario final comprenda la lógica deportiva detrás del dato.

Listing 31: Diccionario semántico para traducir variables a narrativas deportivas interpretables

```
EXPLICACIONES_FEATURES = {
    'pf_ewma5': {
        'positivo': 'Tendencia ascendente. Mejora en las
                    ultimas jornadas.',
        'negativo': 'Tendencia bajista. Caída en las ultimas
                    jornadas.'
    },
}
```

```

        'is_home': {
            'positivo': 'Juega en su estadio. Ventaja local.',
            'negativo': 'Juega fuera de casa. Menor apoyo.'
        }
    }
}

```

9.6.3 Integración en el flujo de la API

La capa XAI está integrada de forma síncrona en el contrato de respuesta de los endpoints de predicción. Cuando se invoca a `PredecirPorteroView` o `PredecirJugadorView`, el sistema ejecuta un pipeline de tres fases:

1. Inferencia: se cargan los archivos .pkl y se calcula la predicción.
2. Atribución: se calculan los valores SHAP para el vector de entrada del jugador.
3. Formateo: se seleccionan los 7 factores con mayor impacto absoluto y se transforman en texto legible.

Listing 32: Contrato de respuesta API con metadatos de predicción y explicación para transparencia

```

return Response({
    'status': 'success',
    'prediccion': float(prediccion),
    'modelo': resultado.get('modelo', 'Random Forest'),
    'features_impacto': impacts[:7],
    'explicacion_texto': explicacion_texto
})

```

9.6.4 Seguridad deportiva y estrategia híbrida

Para garantizar la fiabilidad de las predicciones, el sistema implementa una lógica de transición inteligente. Durante las primeras 5 jornadas de la competición, el histórico es insuficiente para que las ventanas temporales (especialmente las de tamaño 5) tengan significancia estadística.

En este periodo de “arranque en frío”, la vista `PredecirJugadorView` activa automáticamente un modelo de media histórica. A partir de la jornada 6, el orquestador cambia al motor de Machine Learning y activa las explicaciones XAI. Además, el sistema inyecta alertas preventivas de “pocos minutos” si detecta que un jugador tiene una

tendencia de inactividad que las métricas de rendimiento podrían estar enmascarando, añadiendo una capa de seguridad heurística a la predicción estadística.

9.7 Implementación del frontend

La capa de frontend se desarrolló con React y Vite, con una arquitectura orientada a consumo intensivo de API y a la gestión de estado global compartido entre pantallas.

La estructura del frontend se organizó en componentes reutilizables, páginas, contextos y una capa de servicios centralizada para las peticiones HTTP. Esta separación permitió desacoplar la lógica de presentación de la lógica de comunicación con el backend, y facilitó controlar mejor los problemas de sincronización que aparecen cuando varias vistas dependen de la misma información global.

9.7.1 Estructura general de la interfaz

La aplicación se compone de una serie de vistas especializadas, cada una orientada a una parte concreta de la experiencia de usuario. Algunas pantallas son públicas y otras requieren autenticación, lo que obligó a incorporar control de acceso a nivel de rutas. Además, varias vistas comparten información global, como la jornada activa, la sesión autenticada o el estado del tour guiado.

Las vistas principales del frontend son las siguientes:

- LoginPage: pantalla de acceso y registro de usuarios. Permite iniciar sesión, crear una cuenta y recuperar el estado autenticado desde el backend.
- MenuPage: panel principal del sistema. Resume la información más relevante: clasificación, próximos partidos, jugadores destacados y predicciones generadas por los modelos.
- ClasificacionPage: muestra la tabla clasificatoria y partidos de cada jornada.
- EquiposPage: listado general de equipos con acceso a sus fichas individuales.
- EquipoPage: vista detallada de un equipo, con información de plantilla, racha, resultados y contexto competitivo.
- JugadorPage: ficha completa de un jugador, con estadísticas históricas, percentiles, roles, radar visual y explicaciones generadas automáticamente.
- EstadisticasPage: pantalla de consulta global de estadísticas y comparativas.

- `MiPlantillaPage`: gestión de la plantilla del usuario autenticado.
- `PerfilPage`: edición de datos personales, foto, estado y preferencias del usuario.
- `AmigosPage`: gestión de amistades, solicitudes y relaciones sociales.
- `AmigoPlantillaPage`: consulta de la plantilla de otro usuario.
- `SelectFavoritesPage`: selección de equipos favoritos para personalizar la navegación.

Cada una de estas vistas se implementó como un componente de alto nivel encargado de orquestar la recuperación de datos mediante el uso de Custom Hooks, garantizando que la lógica de negocio no ensucie la capa de representación.

9.7.2 Gestión del estado global

La gestión del estado se resolvió mediante una combinación de Context API, estado local y persistencia en el navegador. El estado de autenticación se centralizó en un contexto propio, de modo que la aplicación pudiera reconstruir la sesión al arrancar y decidir de forma fiable si el usuario podía acceder a determinadas rutas.

Este mecanismo resultó especialmente importante en las rutas protegidas, ya que evita mostrar contenido restringido antes de comprobar si existe una sesión válida. De esta forma, la aplicación no depende de que el usuario navegue manualmente hasta una pantalla concreta, sino que valida el acceso desde el inicio y mantiene la coherencia entre frontend y backend.

Para evitar la redundancia de código y facilitar el mantenimiento, se encapsuló la lógica de estado compartido en ganchos personalizados.

- `useAuth`: centraliza las comprobaciones de token JWT y la persistencia de la sesión en el `sessionStorage`.
- `useJornada`: hook que permite a cualquier componente suscribirse a los cambios de la jornada activa sin necesidad de pasar props a través de múltiples niveles de la jerarquía, evitando el prop drilling.
- `usePredictor`: encapsula las llamadas a los modelos de Machine Learning, gestionando automáticamente los estados de error y reintento.

9.7.3 Problemas reales de sincronización y carga

Durante el desarrollo del frontend, uno de los principales retos fue la sincronización entre vistas que consumen la misma información de contexto. El cambio de jornada, por ejemplo, afecta a varias pantallas al mismo tiempo. Para resolver este problema, se adoptó un mecanismo de actualización global que notifica a los distintos componentes cuando la jornada activa cambia, evitando incoherencias entre lo que se muestra en una vista y lo que se muestra en otra.

Otro problema frecuente fue la carga asíncrona de datos. Muchas pantallas dependen de varias peticiones a la vez, y no siempre todas responden con la misma rapidez. En lugar de bloquear la interfaz completa, se optó por manejar estados de carga separados y permitir que la vista siga siendo utilizable aunque una petición concreta falle. Esta estrategia mejora la robustez percibida del sistema y evita que un error puntual degrade por completo la experiencia del usuario.

También fue necesario controlar el proceso de autenticación al cargar la aplicación. Al refrescar la página, el frontend debe reconstruir la sesión antes de decidir si muestra contenido privado o redirige al login. Esta sincronización inicial es una de las partes más delicadas de cualquier aplicación con autenticación por sesión, y en este caso se resolvió mediante un estado de carga transitorio que impide renderizados incorrectos.

9.7.4 Elementos destacados de la interfaz

Entre las funcionalidades más llamativas del frontend destaca la vista de jugador, que integra un radar chart para representar visualmente el perfil estadístico del futbolista. Esta visualización permite condensar en una sola figura varias dimensiones de rendimiento, haciendo más sencilla la comparación entre jugadores y la identificación de fortalezas y debilidades. Frente a una lectura puramente numérica, el radar ofrece una interpretación más intuitiva y cercana al usuario.

Otro elemento especialmente relevante es la gestión de la plantilla mediante drag and drop. Esta funcionalidad aporta una interacción mucho más natural que la simple edición por formularios, ya que permite reorganizar jugadores visualmente dentro del campo. Además de mejorar la experiencia de uso, esta solución refuerza la comprensión táctica de la alineación y hace que el usuario perciba la gestión de su equipo como una acción directa e inmediata.

También merece mención el uso de explicaciones contextuales y percentiles en la ficha de jugador. La interfaz no se limita a mostrar estadísticas aisladas, sino que acompaña



Figura 27: Radar chart de Pedri González.

los datos con interpretaciones y comparativas que ayudan a entender el rendimiento del jugador dentro de su rol y de su contexto competitivo. Esto convierte al frontend en una herramienta de análisis y apoyo a la decisión, no solo en un visor de datos.

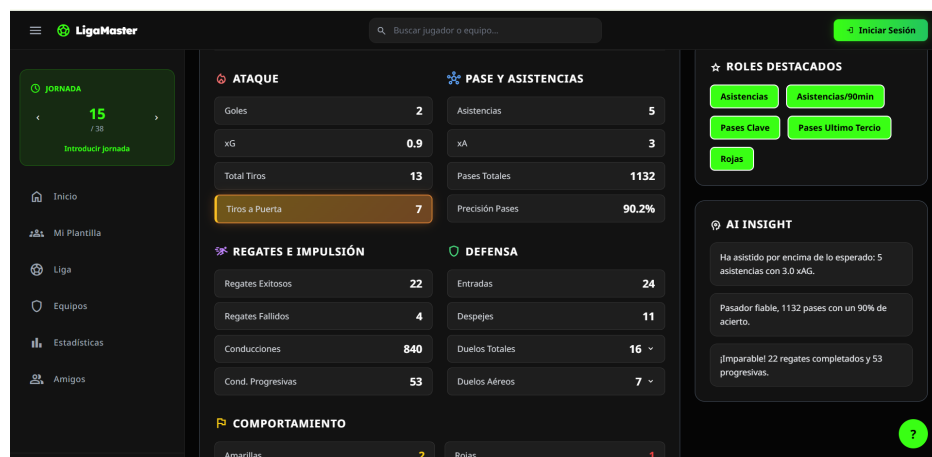


Figura 28: Uso de percentiles y roles en la interfaz en la ficha de Pedri González.

Finalmente, se incorporó un tour guiado para facilitar la exploración inicial de la aplicación. Esta funcionalidad resulta útil porque introduce al usuario en las pantallas más importantes y reduce la curva de aprendizaje, especialmente en un sistema con varias capas de información y navegación.

9.8 Principales problemas enfrentados

A lo largo del ciclo de vida del proyecto, surgieron diversos obstáculos técnicos y metodológicos que obligaron a replantear decisiones de diseño y a aplicar soluciones creativas. Estos retos, lejos de ser simples contratiempos, permitieron profundizar en el conocimiento de las herramientas y en la robustez de la arquitectura final.

9.8.1 Optimización del despliegue

Durante las fases iniciales de desarrollo, se detectó un problema crítico de eficiencia en el flujo de trabajo con Docker. En las primeras versiones de la orquestación, cada vez que se levantaban los contenedores, el sistema procedía a borrar y recrear la base de datos desde cero por defecto.

Dada la magnitud del dataset y la complejidad del pipeline de scraping (que, como se ha detallado en el punto 9.2, incluye miles de registros estadísticos), este proceso de población inicial demoraba más de 20 minutos en cada arranque. Esta carga inicial bloqueante penalizaba severamente la productividad y la disponibilidad del servicio.

Solución: Se implementó una lógica de integridad en el componente `apps.py` (explicada en el punto 9.1.4). El sistema ahora realiza una auditoría de persistencia mediante consultas de salud (health checks) y comprobaciones de volumen. Si el sistema detecta que la base de datos ya está poblada, omite el flujo de carga masiva, reduciendo el tiempo de preparación de minutos a escasos segundos.

9.8.2 Ambigüedad en el matching de entidades y barreras de scraping

El proceso de unificar datos procedentes de tres fuentes heterogéneas (FBRef, Fútbol-Fantasy y Transfermarkt) supuso un reto de limpieza de datos superior al previsto. La principal dificultad residió en la inconsistencia de los nombres de los futbolistas.

Casos como los de los hermanos Williams (Iñaki y Nico, I. Williams y N. Williams) en el Athletic Club, o jugadores con nombres extremadamente comunes como Eric García y Joan García, generaban colisiones en los algoritmos de búsqueda. A esto se sumó la política restrictiva de bloqueos por IP de los portales de origen, que identificaban las ráfagas de peticiones como tráfico malicioso.

Solución: Se abandonó la comparación simple por cadenas de texto en favor de la lógica difusa y el matching por contexto de partido (detallado en el punto 9.3). Además, para evadir los bloqueos, se diseñó la estrategia de backoff exponencial y la gestión de descargas locales controladas para los datos históricos más pesados.

9.8.3 Refactorización arquitectónica: de Django Templates a React

Originalmente, el frontend se concibió utilizando el sistema de vistas y plantillas nativo de Django. Sin embargo, a medida que los requisitos de interactividad crecieron

(especialmente con el sistema de drag and drop y los gráficos en tiempo real), se hizo evidente que una arquitectura monolítica no era suficiente.

Se decidió migrar toda la capa de presentación a React. En un intento inicial por optimizar tiempos, se recurrió a herramientas de inteligencia artificial para automatizar la conversión de código Python/HTML a componentes JavaScript. No obstante, el resultado fue insatisfactorio: el código generado presentaba una deuda técnica inasumible, con errores de lógica en el manejo de hooks y una integración deficiente con la API.

Lección aprendida: La migración tuvo que realizarse de forma manual e íntegra. Aunque esto supuso una inversión de tiempo considerable y un retraso en el cronograma original, garantizó la calidad del código, la correcta gestión del estado global (como se analiza en el punto 9.7) y una experiencia de usuario fluida que la automatización no pudo alcanzar.

9.8.4 Integridad predictiva: detección y corrección de Data Leakage

Uno de los errores más sutiles pero peligrosos ocurrió durante el entrenamiento inicial de los modelos de Machine Learning. En las primeras aproximaciones, al no haber trabajado previamente con series temporales deportivas de esta escala, se aplicaron divisiones aleatorias de datos (random splits).

Esto provocó un fenómeno de data leakage: el modelo “predecía” el pasado utilizando información del futuro, arrojando métricas de precisión extraordinariamente altas que no se correspondían con la realidad. Al intentar predecir jornadas nuevas sin datos almacenados, el rendimiento caía drásticamente.

Solución: Se rediseñó por completo el motor de entrenamiento en el `BaseTrainer` (ver punto 9.4.2), imponiendo un corte temporal estricto. Se aseguró que para predecir la jornada n , el modelo solo tuviera acceso a datos de la jornada $n - 1$ hacia atrás, garantizando así la validez científica de las predicciones y la utilidad real de la herramienta para el usuario.

La implementación del sistema ha sido un proceso iterativo de refinamiento técnico. La transición de un modelo monolítico a uno desacoplado, la superación de las barreras de extracción de datos y la corrección de sesgos en los modelos predictivos han consolidado una plataforma que no solo es funcional, sino también escalable y robusta ante las inconsistencias del mundo real. Los problemas enfrentados han servido para validar la arquitectura propuesta, demostrando que la integración de Machine Learning en entornos de producción exige un control riguroso tanto de la infraestructura como de la integridad de los datos.

9.8.5 Elección del motor de búsqueda

Durante el desarrollo del sistema, la elección del motor de búsqueda ha sido un aspecto clave debido a la necesidad de realizar consultas eficientes sobre grandes volúmenes de datos deportivos.

En una primera fase, se optó por utilizar **ElasticSearch**, aprovechando su integración con servicios gestionados en la nube y su potente sistema de indexación. Sin embargo, esta solución presentaba una limitación importante: tras un periodo inicial gratuito, el servicio requería una suscripción de pago, lo cual no se ajustaba a los requisitos del proyecto.

Como alternativa, se evaluó **OpenSearch**, un fork open-source de ElasticSearch que eliminaba dicha restricción económica. No obstante, durante las pruebas de despliegue se observó que su consumo de recursos era elevado, especialmente en términos de memoria RAM. Esto lo hacía inviable en entornos con recursos limitados, como las máquinas virtuales gratuitas o de bajo coste disponibles en Azure.

Solución: Finalmente, se optó por **MeiliSearch**, un motor de búsqueda ligero y optimizado para rendimiento, que permite su despliegue en entornos con recursos reducidos sin comprometer significativamente la velocidad de respuesta. Esta elección permitió mantener la funcionalidad de búsqueda avanzada dentro de las restricciones de infraestructura del proyecto, asegurando al mismo tiempo una experiencia de usuario fluida.

Esta transición evidencia la importancia de considerar no solo las capacidades técnicas de una herramienta, sino también su viabilidad en entornos reales de despliegue, especialmente cuando existen limitaciones económicas y de recursos.

9.8.6 Despliegue en producción

El despliegue en producción planteó desafíos significativos, especialmente en relación con la gestión de costes y la sostenibilidad de la infraestructura a lo largo del tiempo.

Inicialmente, se consideró una arquitectura monolítica basada en una única máquina virtual en Azure que albergara todos los componentes del sistema. Sin embargo, durante la fase de estimación de costes se determinó que esta solución supondría un gasto mensual mayor al crédito estudiantil de Azure, lo cual resultaba inviable para un proyecto académico cuyo objetivo es mantenerse accesible durante varios meses para su evaluación.

Solución: Se rediseñó la arquitectura hacia un modelo distribuido, delegando distintos componentes en servicios especializados y, en muchos casos, con planes gratuitos o de bajo coste:

- El **frontend** se desplegó en Vercel, aprovechando su integración con aplicaciones SPA y su red de distribución global.
- La **base de datos** se externalizó a Supabase, proporcionando un servicio PostgreSQL gestionado.
- El sistema de **caché** se implementó mediante Upstash Redis, optimizado para entornos serverless.
- En Azure se mantuvieron únicamente los componentes críticos: el **backend en Django** y el motor de búsqueda **MeiliSearch**.

Esta estrategia permitió reducir drásticamente los costes de infraestructura, manteniendo al mismo tiempo la escalabilidad y disponibilidad del sistema. Además, el enfoque distribuido mejora la resiliencia de la aplicación y refleja una arquitectura más cercana a entornos reales de producción.

En conclusión, el proceso de despliegue no solo ha servido para optimizar recursos, sino también para validar decisiones arquitectónicas orientadas a la sostenibilidad y eficiencia del sistema.

10. Calidad del software y pruebas

Este capítulo recoge la estrategia de validación y aseguramiento de la calidad aplicada al sistema. El planteamiento seguido es incremental y orientado al riesgo: primero se verifican las entidades y reglas de dominio, después las rutas de la API y, por último, los flujos de usuario y los escenarios de integración más relevantes. El objetivo no es alcanzar una cobertura artificialmente máxima, sino una validación sólida, coherente con el alcance del proyecto y representativa del uso real de la plataforma.

10.1 Organización del entorno de pruebas

La validación se articula en una suite automatizada de 30 pruebas, distribuidas en seis bloques según el nivel de validación y la responsabilidad funcional del código evaluado. La batería combina pruebas unitarias, de integración, extremo a extremo, negativas, algorítmicas y un caso manual, con el fin de cubrir tanto la lógica de negocio como la interacción entre la API, la base de datos y los servicios auxiliares.

Cuadro 7: Suite de pruebas automatizadas

Archivo	Nivel de prueba	Nº de tests
<code>test_unit_models.py</code>	Unitario	9
<code>test_integration_api.py</code>	Integración	8
<code>test_e2e_user_journey.py</code>	Extremo a extremo	2
<code>test_negative_api.py</code>	Negativo	6
<code>test_matching.py</code>	Algorítmico	4
<code>test_manual.py</code>	Datos / Manual	1
Total		30

A modo de resumen, las características principales de la suite son las siguientes:

- **Total de pruebas:** 30.
- **Tiempo de ejecución:** aproximadamente 145 segundos en entorno local.
- **Tecnologías utilizadas:** Django `TestCase`, `APIClient`, `SimpleTestCase`, `mock` y `Coverage.py`.

- **Áreas cubiertas:** modelos, autenticación, relaciones sociales, API REST, algoritmo de *matching* y escenarios manuales.

10.2 Niveles de pruebas y resultados

10.2.1 Pruebas unitarias y de integración

Las pruebas unitarias se centran en la validación de reglas de negocio, restricciones de integridad y funciones auxiliares del dominio. Las pruebas de integración, por su parte, ejercitan la API sobre datos reales de base de datos en un entorno de test aislado, lo que permite comprobar el comportamiento conjunto de vistas, modelos y serialización.

- **Modelos (92,79 %):** Verificación de restricciones, unicidad, relaciones foráneas y validación explícita de entidades clave como *Partido*, *Jugador* y *EquipoJugadorTemporada*.
- **Autenticación (84,68 %):** Registro, inicio y cierre de sesión, consulta del usuario autenticado y validación de credenciales incorrectas, incluyendo duplicidades y campos obligatorios.
- **Relaciones sociales:** Gestión de equipos favoritos, acceso autenticado, solicitudes de amistad y administración de notificaciones.
- **API de Equipos (71,91 %):** Listado, detalle, comportamiento dependiente de temporada y cálculo de datos derivados, como la sugerencia de temporada activa.
- **API de Jugadores (72,30 %):** Detalle individual, radar de métricas, partidos recientes y agregaciones sobre el historial del jugador.
- **API del Consejero (73,91 %):** Validación de entrada, control de errores y generación del veredicto de recomendación.
- **API de Estadísticas (82,78 %):** Comparación de jugadores y agregaciones de rendimiento sobre temporadas concretas.

La integración con PostgreSQL se verifica mediante conjuntos de datos de prueba completos, lo que permite comprobar el filtrado por temporada y jornada, las agregaciones de clasificación y la consistencia de los resultados devueltos por la API.

10.2.2 Validación de la API

La validación funcional de la API cubre rutas públicas, rutas autenticadas y respuestas ante entradas inválidas. El objetivo no se limita a comprobar que un *endpoint* responde, sino que la respuesta sea semánticamente correcta, estable y consumible por el *frontend*.

En una ejecución focalizada de las clases `NegativeApiTests` e `IntegrationApiTests`, la batería de 14 pruebas concluyó sin errores. Durante dicha ejecución se observaron códigos de estado como 401, 404, 405, 503 y 400; sin embargo, todos ellos correspondían a escenarios negativos diseñados de forma intencional. Estos códigos no reflejan fallos del sistema, sino comportamientos esperados frente a autenticación ausente, recursos no localizados, métodos no permitidos, validaciones incorrectas o dependencias no disponibles.

Cuadro 8: Ejecución focalizada de validación de la API

Clase	Tests	Resultado verificado
<code>NegativeApiTests</code>	6	Respuestas esperadas ante autenticación inválida, recursos inexistentes, métodos no permitidos y errores de validación.
<code>IntegrationApiTests</code>	8	Flujo correcto sobre perfil, notificaciones, clasificación, jugador, consejero, estadísticas y rutas v2.
Total	14	Ejecución estable, sin errores de test y con códigos de respuesta negativos intencionales.

- **Pruebas positivas:** Se comprueba que los documentos JSON incluyan los campos requeridos por la interfaz y que los parámetros de temporada, jornada, posición o búsqueda alteren la respuesta de forma controlada y predecible.
- **Pruebas negativas:** Se validan los códigos 401, 400, 404, 405 y 503 en los escenarios previstos, evitando que un error de validación o una dependencia externa no disponible se materialice en un fallo silencioso o en una excepción no controlada.

10.2.3 Pruebas de ciclo de vida

Se han implementado flujos extremo a extremo que reproducen el ciclo de vida más representativo del usuario en la plataforma: alta, inicio de sesión, consulta de informa-

ción, gestión de favoritos y tramitación de solicitudes de amistad. Las etapas del flujo son las siguientes:

1. **Registro y autenticación:** creación de usuario y obtención del token JWT.
2. **Acceso a perfil:** lectura de `/api/me/` y comprobación del estado autenticado.
3. **Interacción con favoritos:** alta y baja de un equipo favorito sobre datos persistentes.
4. **Relaciones sociales:** envío, aceptación y reflejo de una solicitud de amistad sobre el estado persistente.
5. **Cierre de sesión:** validación del *logout* y desconexión del flujo autenticado.

Este enfoque confirma que autenticación, persistencia y autorización se comportan correctamente cuando se ejecutan como un flujo completo y no como llamadas aisladas entre sí.

10.2.4 Pruebas del algoritmo de *matching*

El algoritmo de *matching* se valida de forma aislada mediante casos contruïdos manualmente. Las pruebas comprueban la desambiguación de nombres, la priorización por minutos jugados y puntuación, y la gestión de propuestas sin coincidencia clara. Aunque su cobertura es inferior a la del resto de módulos, se trata de una lógica específica y fuertemente dependiente de datos heterogéneos, por lo que la validación por escenarios resulta más representativa que la búsqueda de una cobertura puramente cuantitativa.

10.3 Pruebas de carga y rendimiento

Para evaluar la capacidad, rendimiento y escalabilidad de la API bajo condiciones realistas, se ha ejecutado una batería de pruebas de carga utilizando la herramienta Locust. El entorno de pruebas se ha configurado con 3 *workers* distribuidos, evaluando distintos volúmenes de tráfico para identificar el comportamiento del sistema y detectar sus límites físicos.

Las pruebas se han estructurado en fases de intensidad creciente. Aunque el objetivo teórico era alcanzar los 300 usuarios concurrentes, el análisis se centra en las fases de validación inicial y carga moderada, donde se localizó el punto exacto de saturación arquitectónica del sistema.

Cuadro 9: Resultados agregados de las pruebas de estrés con Locust

Métrica	Fase 1 (10 usuarios)	Fase 2 (100 usuarios)
Duración de la prueba	3 minutos	5 minutos
Tasa de generación (<i>Spawn rate</i>)	2 usr/s	10 usr/s
Total de peticiones procesadas	612	2 187
Throughput (RPS)	~5,8 req/s	~4,6 req/s
Tasa de error (Fallos)	4,7 %	14,5 %
Latencia Mediana	130 ms	5 800 ms
Percentil 90 (P90)	2 400 ms	25 000 ms

Análisis de resultados y comportamiento del sistema:

Durante la **Fase 1 (10 usuarios concurrentes)**, la plataforma demostró un comportamiento estable y funcional, con una tasa de error baja (4,7 %) y un tiempo de respuesta mediano bastante rápido (130 ms). Si bien el comportamiento general es positivo para una carga base, algunas rutas que exigen mayor procesamiento, como `/api/clasificacion` o `POST /api/cambiar-jornada`, comenzaron a mostrar latencias significativas en el percentil 90 superior a los 2 segundos.

Al escalar a la **Fase 2 (100 usuarios concurrentes)**, el sistema alcanzó rápidamente su límite operativo. A pesar de aumentar la concurrencia en un factor de 10, el *throughput* (RPS) no logró escalar, estancándose e incluso reduciéndose ligeramente a 4,6 peticiones por segundo. La tasa de error se elevó al 14,5 %, y las latencias sufrieron una degradación drástica: el tiempo mediano de respuesta ascendió a 5,8 segundos y el percentil 90 alcanzó los 25 segundos. Bajo estas condiciones métricas, el paso a la Fase 3 (300 usuarios) supone un colapso completo del servicio debido a la incapacidad previa de procesar la carga de la Fase 2.

Diagnóstico del cuello de botella:

El análisis detallado del colapso en la Fase 2 confirma que la degradación del servicio no se debe a ineficiencias en el código de la aplicación o del ORM sino que está originado por la configuración límite de conexiones concurrentes en PostgreSQL.

Al recibir la avalancha de peticiones (10 nuevos usuarios por segundo), el límite máximo de conexiones (`max_connections`) tolerado por la base de datos se agota casi de inmediato. Esto obliga a las instancias de Django a poner en cola el resto de las solicitudes a la espera de que se libere un hilo en la base de datos. Este encolamiento

pasivo es el responsable directo de que la latencia se dispare a decenas de segundos y de que un 14,5 % de las peticiones terminen en error por agotamiento de tiempo de espera (*timeout*).

Conclusión:

El *backend* en su configuración actual soporta de forma fluida un tráfico simultáneo de aproximadamente 10 a 20 usuarios. Para alcanzar el objetivo de 100 usuarios o más con latencias inferiores a 1000 ms, es imprescindible modificar la topología de la base de datos. La mejora inmediata si pasase a producción sería implementar un agrupador de conexiones (*Connection Pooler*) como PgBouncer, aumentar el parámetro `max_connections` en PostgreSQL, y expandir el uso de Redis para cachear operaciones pesadas de lectura, aliviando así la presión directa sobre el gestor relacional.

10.4 Resultados de cobertura y calidad

Para auditar la calidad del código se han combinado dos métricas complementarias: cobertura dinámica con `Coverage.py` y análisis estático con SonarQube. La primera cuantifica el comportamiento ejecutado por la suite de pruebas; la segunda permite identificar deuda técnica, *hotspots* de seguridad y problemas de mantenibilidad sin necesidad de ejecutar el sistema.

10.4.1 Cobertura de código

Se estableció un umbral mínimo del 70 %. Tras la ejecución de la suite consolidada de 30 pruebas, el sistema alcanzó un 83,02 % de cobertura global sobre el alcance definido en `.coveragerc`, superando el objetivo planteado sin aproximarse a un valor artificialmente perfecto.

Cuadro 10: Cobertura de código por módulo

Módulo	Cobertura	Descripción
<code>main.models</code>	92,79 %	Restricciones, relaciones FK e integridad referencial.
<code>main.api.auth</code>	84,68 %	Registro, <i>login</i> , <i>logout</i> y estado de sesión.
<code>main.api.consejero</code>	73,91 %	Validación de entrada y veredicto de recomendación.
<code>main.api.equipo</code>	71,91 %	Listado y detalle de equipos.
<code>main.api.jugador</code>	72,30 %	Detalle individual, radar, histórico y partidos.
<code>main.api.menu</code>	79,81 %	Resumen principal y <i>top</i> jugadores.
<code>main.api.estadisticas</code>	82,78 %	Comparación de jugadores y agregaciones de rendimiento.
<code>main.drf_views</code>	100,00 %	Adaptadores v2 y rutas equivalentes.
<code>main.scrapping.matching</code>	66,02 %	Agrupación y resolución del algoritmo de <i>matching</i> .
Total del sistema	83,02 %	Cobertura global integrada mediante 30 pruebas.

El módulo de *matching* queda por debajo del resto porque concentra una lógica algorítmica compleja cuya cobertura por exhaustividad resulta poco práctica. Aun así, su validación específica con casos sintéticos evita regresiones en los criterios de resolución y mantiene controlado el comportamiento en los escenarios más sensibles.

10.4.2 Análisis estático con SonarQube

El análisis estático realizado con SonarQube ofrece una visión complementaria del estado del proyecto. Los resultados permiten mantener el nivel de calidad dentro de un rango alto pero realista, sin forzar una cobertura total que no aportaría valor adicional al proceso de validación.

- **Mantenibilidad (Rating A):** La base de código presenta un buen nivel de calidad estructural, con una deuda técnica reducida en relación con el volumen analizado. Se han identificado 147 *code smells*, con un esfuerzo de mejora estimado en 4 días de trabajo.
- **Seguridad (Rating E):** Se han identificado 10 *security hotspots*, revisados manualmente, asociados principalmente a la gestión de sesiones y a la configuración de red. Entre ellos se incluyen credenciales embebidas en el código fuente de los tests, que SonarQube no distingue automáticamente de credenciales reales de producción.
- **Fiabilidad (Rating A):** La herramienta no detecta ningún *bug* en el código analizado.

En conjunto, la combinación de cobertura automatizada, validación de casos negativos y análisis estático proporciona una base sólida para evolucionar la plataforma con menor riesgo de regresión, manteniendo un equilibrio razonable entre la amplitud de las pruebas y el coste de su mantenimiento.

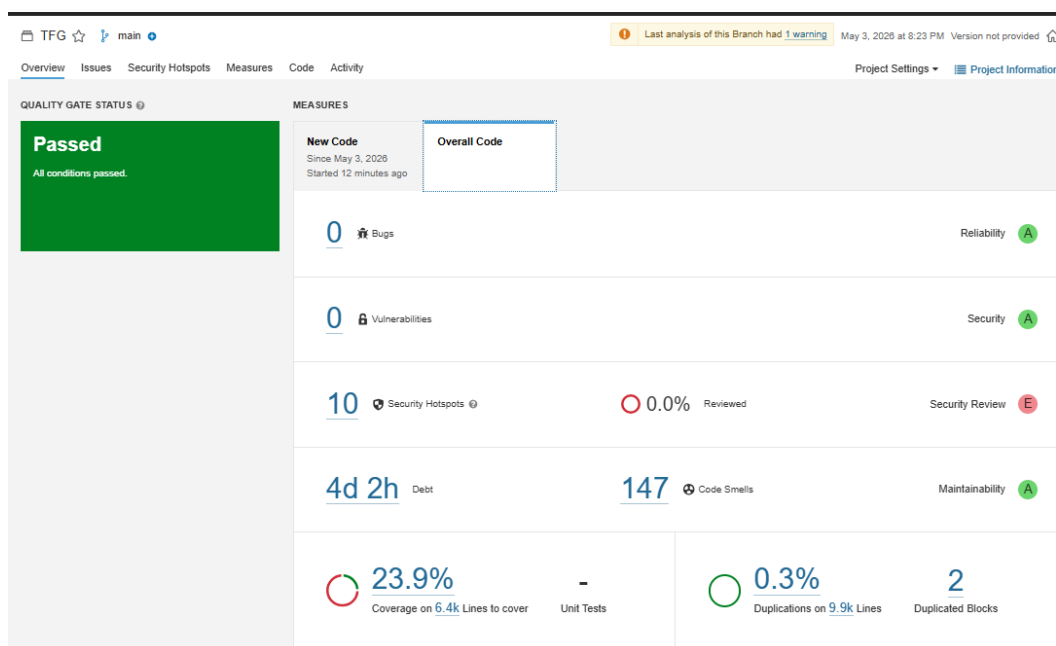


Figura 29: Estado del *dashboard* de SonarQube tras el análisis del proyecto.

11. Resultados

Como se detalló en el apartado 7.3.3, para evaluar la calidad predictiva de los modelos desarrollados, se establecieron umbrales de rendimiento basados en la distribución del Error Absoluto Medio (MAE) observada en el estudio previo. Estos objetivos permiten clasificar la precisión de los modelos según la posición táctica.

Objetivos de rendimiento por posición

Posición	Élite	Aceptable	Alerta
Porteros (PT)	$\leq 2,93$	$\leq 3,18$	$> 3,63$
Defensas (DF)	$\leq 2,15$	$\leq 2,79$	$> 3,18$
Mediocentros(MC)	$\leq 2,10$	$\leq 2,60$	$> 3,25$
Delanteros (DT)	$\leq 2,25$	$\leq 2,75$	$> 3,25$

Cuadro 11: Umbrales de rendimiento por posición táctica.

Resultados obtenidos por demarcación

Tras el entrenamiento y la optimización de hiperparámetros mediante Grid Search, se presentan los resultados obtenidos para cada posición. Se han evaluado modelos de regresión lineal regularizada (Ridge, ElasticNet) y modelos basados en árboles (Random Forest, XGBoost).

A. Defensas (DF) El modelo Ridge destaca como el más preciso, situándose dentro del rango de rendimiento aceptable con un $MAE = 2,67$, superando el objetivo establecido.

Modelo	MAE	RMSE	Spear.	Best Params
Ridge	2,6748	3,3017	0,0889	alpha: 1000,0
ElasticNet	2,6756	3,3022	0,0874	alpha: 0,05
Random Forest	2,6837	3,3098	0,0841	max_depth: 10
XGBoost	2,7366	3,4026	0,0744	lr: 0,1

Cuadro 12: Resultados de los modelos evaluados para la posición Defensas (DF).

B. Delanteros (DT) En esta demarcación, ElasticNet obtuvo el mejor desempeño, quedando muy cerca del umbral aceptable (2,75) con una correlación de Spearman

significativamente superior a la de los defensas ($\rho = 0,289$), lo que indica una mejor capacidad de *ranking*.

Modelo	MAE	RMSE	Spear.	Best Params
ElasticNet	2,7928	3,6878	0,2893	alpha: 0,01
Ridge	2,7981	3,6914	0,2859	alpha: 100,0
Random Forest	2,8514	3,7055	0,2662	max_depth: 10
XGBoost	2,9387	3,8463	0,2279	n_est: 300

Cuadro 13: Resultados de los modelos evaluados para la posición Delanteros (DT).

C. Mediocentros (MC) El modelo Ridge lidera la precisión en el centro del campo, mostrando una estabilidad notable y situándose apenas 0,06 puntos por encima del objetivo «Aceptable». La correlación de Spearman ($\rho = 0,228$) confirma la capacidad del modelo para ordenar correctamente a los jugadores.

Modelo	MAE	RMSE	Spear.	Best Params
Ridge	2,6650	3,4443	0,2277	alpha: 250,0
ElasticNet	2,6672	3,4457	0,2257	alpha: 0,01
Random Forest	2,6701	3,4470	0,2259	max_depth: 10
XGBoost	2,7621	3,5537	0,1566	lr: 0,1

Cuadro 14: Resultados de los modelos evaluados para la posición Mediocentros (MC).

D. Porteros (PT) Para la posición de portero, ElasticNet ofrece el mejor ajuste, cumpliendo prácticamente de forma exacta con el objetivo de rendimiento aceptable establecido en 3,18 ($\text{MAE} = 3,19$).

Modelo	MAE	RMSE	Spear.	Best Params
ElasticNet	3,1878	3,7598	0,1149	alpha: 0,05
Ridge	3,2068	3,7750	0,0900	alpha: 250,0
Random Forest	3,2246	3,8273	0,0210	max_depth: 20
XGBoost	3,3537	4,0727	-0,0407	lr: 0,1

Cuadro 15: Resultados de los modelos evaluados para la posición Porteros (PT).

Comparativa final y análisis del cumplimiento de objetivos

La tabla 16 resume el cumplimiento de los objetivos por posición utilizando el mejor modelo obtenido en cada caso.

Posición	Modelo	MAE	Obj.	Estado
Defensas	Ridge	2,67	2,79	Cumplido
Delanteros	ElasticNet	2,79	2,75	+0,04
Mediocentros	Ridge	2,66	2,60	+0,06
Porteros	ElasticNet	3,19	3,18	Cumplido

Cuadro 16: Cumplimiento de los objetivos de rendimiento por posición.

Observaciones generales En líneas generales, los modelos de regresión lineal regularizada (Ridge y ElasticNet) han demostrado una mayor capacidad de generalización frente a modelos complejos como XGBoost en este conjunto de datos. Los resultados son satisfactorios: en todas las posiciones el MAE se mantiene significativamente alejado del Umbral de Alerta (rango P75), validando la utilidad de los modelos para la predicción de métricas en el contexto del estudio.

A pesar de que en las categorías de delanteros y mediocentros el MAE se sitúa ligeramente por encima de la mediana fijada como «objetivo aceptable», los modelos se consideran válidos y funcionales para el propósito del estudio por las siguientes razones:

1. Capacidad de clasificación Las demarcaciones de delanteros ($\rho = 0,289$) y mediocentros ($\rho = 0,228$) presentan los coeficientes de Spearman más elevados del proyecto. Esto indica que el modelo tiene una notable capacidad para ordenar correctamente a los jugadores según su rendimiento real, lo cual constituye el objetivo crítico de la herramienta. La capacidad de *ranking* resulta más relevante que la precisión absoluta cuando el objetivo es identificar talento. Además, las desviaciones respecto al objetivo son mínimas:

- Delanteros: $\Delta\text{MAE} = +0,04$ (1,5 % por encima del objetivo).
- Mediocentros: $\Delta\text{MAE} = +0,06$ (2,3 % por encima del objetivo).

Ambas desviaciones se mantienen muy alejadas del Umbral de Alerta (P75), con un margen de seguridad de 0,46 y 0,59 puntos respectivamente, lo que demuestra la robustez de las predicciones.

2. Consistencia predictiva y generalización El hecho de que los modelos lineales regularizados (Ridge y ElasticNet) sean los ganadores en todas las demarcaciones sugiere que el modelo ha capturado tendencias generales del juego que no dependen

del ruido ni del *overfitting*. Esta característica garantiza una predicción real sobre datos no vistos, lo que es esencial para su aplicación en competiciones o temporadas futuras.

Resulta altamente revelador que dichos modelos superen sistemáticamente a algoritmos basados en árboles de decisión complejos (Random Forest y XGBoost). Esto confirma que el rendimiento individual en el fútbol contiene un componente intrínseco de aleatoriedad y ruido: mientras que XGBoost tiende a sobreajustarse memorizando dicho ruido, los modelos lineales demuestran una mayor capacidad de generalización al capturar estrictamente la tendencia subyacente.

3. La paradoja varianza *vs.* consistencia Los resultados exponen un patrón estadístico que refleja fielmente la naturaleza estocástica del deporte:

- **Atacantes (DT y MC):** presentan la mayor capacidad de clasificación ($\rho \approx 0,289$) pero un mayor error absoluto (MAE). Es más sencillo ordenar la calidad de los atacantes porque los jugadores de élite destacan claramente en sus métricas base; sin embargo, predecir su puntuación exacta es complejo al depender de eventos de altísima varianza, como los goles o las asistencias.
- **Defensores y porteros (DF y PT):** cumplen con holgura los objetivos de MAE pero registran una capacidad de *ranking* pobre ($\rho < 0,12$). Sus puntuaciones base son mucho más estables y predecibles (dependen del volumen de juego, los despejes o las porterías a cero), pero dada la alta homogeneidad en sus calificaciones empíricas, el modelo encuentra dificultades para establecer una jerarquía predictiva clara.

4. Alta penalización de características El análisis de los hiperparámetros óptimos arroja valores de penalización notablemente altos, en especial en los modelos Ridge (p. ej. $\alpha = 1000,0$ en Defensas y $\alpha = 250,0$ en Mediocentros). Esto refleja la necesidad del algoritmo de suprimir el impacto de ciertas variables, indicando la presencia de multicolinealidad entre los predictores extraídos mediante *scraping*. Se confirma que la regularización ha sido una técnica crítica y exitosa para filtrar métricas redundantes que no aportaban valor real a la predicción final.

Resumen de la validación final

Posición	Modelo	MAE	Spearman	Evaluación
Defensas	Ridge	2,67	0,089	Aceptable (error controlado)
Delanteros	ElasticNet	2,79	0,289	Excelente (buena capacidad ranking)
Mediocentros	Ridge	2,66	0,228	Bueno (error mínimo + ranking)
Porteros	ElasticNet	3,19	0,115	Aceptable (cumple exactamente)

Cuadro 17: Resumen de la validación final por posición.

Se confirma la utilidad y robustez de la herramienta desarrollada. Mientras que el MAE proporciona una medida de proximidad al valor exacto predicho, el coeficiente de Spearman confirma que el modelo comprende la jerarquía y estructura del rendimiento de los jugadores en el mercado real de la Liga EA Sports, permitiendo:

1. Identificar correctamente el talento infravalorado (jugadores con rendimiento superior al esperado según sus métricas base).
2. Detectar jugadores en mala dinámica (con rendimiento inferior a sus métricas base).
3. Proporcionar un *ranking* fiable para procesos de fichaje y toma de decisiones.

En consecuencia, los modelos desarrollados son adecuados para su implementación en sistemas de recomendación de transferencias y en el análisis predictivo del rendimiento en el contexto de la competición analizada.

12. Conclusiones

12.1. Cumplimiento de objetivos

El desarrollo de la plataforma *LigaMaster* ha concluido con éxito, logrando el objetivo general propuesto al inicio del trabajo: diseñar e implementar una plataforma web de apoyo a la toma de decisiones en el Fantasy Football mediante análisis predictivo y explicabilidad.

Se ha logrado una integración completa entre un motor de *Machine Learning* desarrollado en Python, un *backend* robusto con Django REST Framework y un *frontend* interactivo y desacoplado creado con React. Los objetivos funcionales se han cubierto en su totalidad, permitiendo la visualización de estadísticas, la gestión de plantillas y la predicción de rendimiento basada en explicabilidad (XAI). Del mismo modo, los requisitos no funcionales de rendimiento (RNF1), calidad del dato (RNF3) y seguridad (RNF5) han sido verificados mediante la suite de pruebas automatizadas, las métricas de cobertura y los análisis estáticos con SonarQube descritos en el capítulo anterior.

12.2. Limitaciones encontradas

A lo largo del proyecto se han identificado ciertas limitaciones inherentes a la obtención y al procesamiento de los datos, así como a las decisiones de diseño adoptadas bajo las restricciones propias de un desarrollo individual. Las más relevantes se describen a continuación:

- **Ausencia de APIs oficiales para el mercado.** La falta de *endpoints* públicos en la API oficial de LaLiga Fantasy impidió extraer e integrar los valores económicos dinámicos de los jugadores en tiempo real de forma automatizada. Esto ha limitado la capacidad de desarrollar un módulo financiero completo en la plataforma, funcionalidad que habría enriquecido considerablemente la propuesta y habría permitido correlacionar el precio de mercado con la predicción de rendimiento.
- **Bloqueos de IP durante el *scraping*.** Debido a los mecanismos anti-*scraping* activos en las fuentes consultadas, fue necesario descargar manualmente el HTML de todos los partidos históricos, proceso que resultó costoso en tiempo y difícilmente automatizable. Esta restricción implica que la actualización del *dataset* requiere intervención manual periódica, lo que supone un cuello de botella operativo en un entorno de producción continua.

- **Capacidad de carga del sistema.** Las pruebas de rendimiento con Locust evidenciaron que, en su configuración actual, el sistema comienza a degradarse significativamente a partir de los 20–30 usuarios concurrentes, con tasas de error del 14,5 % a 100 usuarios. El diagnóstico apunta al agotamiento del número máximo de conexiones simultáneas de PostgreSQL como causa principal, lo que constituye una limitación relevante de cara a un despliegue en producción con tráfico real.

12.3. Líneas de trabajo futuro

El sistema desarrollado posee una arquitectura escalable que abre la puerta a diversas mejoras y nuevas vías de desarrollo. A continuación se presentan las líneas de trabajo consideradas de mayor impacto, tanto a nivel técnico como de producto:

- **Optimización del rendimiento y escalabilidad.** El cuello de botella identificado en las pruebas de carga puede resolverse mediante la incorporación de un *connection pooler* como PgBouncer entre Django y PostgreSQL, lo que permitiría gestionar un número elevado de peticiones concurrentes sin saturar el límite de conexiones de la base de datos. Complementariamente, ampliar el uso de Redis como capa de caché para cubrir los *endpoints* de mayor coste computacional —especialmente los de predicción y clasificación— reduciría la presión directa sobre el gestor relacional. Estas dos mejoras, de bajo coste de implementación, permitirían escalar el sistema a un orden de magnitud superior sin modificar la arquitectura existente.
- **Consolidación y refactorización de la capa API.** Durante el desarrollo del proyecto coexistieron dos capas de acceso a datos: la API principal estructurada en el módulo `main/api/` y un conjunto de adaptadores de compatibilidad en `main/drf_views.py`, introducidos para mantener rutas equivalentes durante una migración interna. Esta situación genera deuda técnica al duplicar puntos de entrada, dificultar el mantenimiento y aumentar la superficie de posibles inconsistencias. Como trabajo futuro prioritario se propone unificar ambas capas bajo una única interfaz coherente, eliminando los adaptadores de compatibilidad y normalizando el contrato de todos los *endpoints* bajo un versionado explícito (`/api/v1/`).
- **Automatización del *pipeline* de reentrenamiento.** En su estado actual, los modelos de Machine Learning son entrenados de forma manual fuera de línea y serializados como artefactos `.pkl` estáticos. Este enfoque, adecuado para el alcance del TFG, no es sostenible en producción: el rendimiento de los modelos se degrada con el tiempo a medida que el contexto de la competición evoluciona, y la incor-

poración de nuevas jornadas requiere intervención manual. Una línea de trabajo natural sería implementar un *pipeline* de reentrenamiento periódico y automatizado —por ejemplo, al cierre de cada jornada— utilizando una cola de tareas asíncronas como Celery. Este enfoque se enmarca dentro de las prácticas de MLOps y garantizaría que los modelos desplegados reflejen siempre el estado más reciente de la competición.

- **Predicción de titularidades.** Integrar un modelo secundario que evalúe la probabilidad de que un jugador parta en el once inicial. Mediante el análisis de rotaciones tácticas, el histórico de minutos jugados, la acumulación de tarjetas y los partes médicos disponibles, se refinaría notablemente la precisión de las puntuaciones esperadas por jornada. Actualmente el sistema asume la disponibilidad plena del jugador, lo que constituye una simplificación relevante que limita la fiabilidad de las predicciones en jornadas con alta rotación.
- **Desarrollo de una aplicación móvil nativa.** Aprovechando que el *backend* está completamente desacoplado y expuesto a través de una API REST con DRF, el siguiente paso natural sería implementar una aplicación nativa para dispositivos iOS y Android. Esto mejoraría la experiencia de usuario en el entorno donde la mayoría de personas acceden a los juegos de Fantasy Football y permitiría habilitar notificaciones *push* en tiempo real, complementando el sistema de WebSockets ya implementado.

12.4. Valoración personal

La realización de este Trabajo de Fin de Grado ha supuesto un verdadero reto que me ha permitido consolidar y poner en práctica los conocimientos adquiridos a lo largo de la titulación. A través de este proyecto he gestionado todas las etapas del ciclo de vida del software: desde la captura de requisitos y el diseño de procesos ETL, hasta la implementación de modelos de aprendizaje automático, el desarrollo *full-stack* y el despliegue final.

Si bien estas tareas han sido recurrentes durante mi formación, la ejecución de este proyecto ha supuesto un hito personal al asumir, por primera vez, la autonomía y la responsabilidad total sobre un sistema complejo de principio a fin.

Especialmente valioso ha resultado el aprendizaje autodidacta en técnicas de Explainable AI (XAI), lo que me ha dotado de una perspectiva diferente y realista sobre las exigencias actuales en el desarrollo de software y en el mundo de la inteligencia artificial. Asimismo, el proyecto ha permitido profundizar en el tratamiento de datos

a gran escala y en los fundamentos prácticos de la ciencia de datos aplicada a un dominio real. En conjunto, considero que *LigaMaster* refleja fielmente el perfil técnico que una titulación en Ingeniería del Software debe desarrollar: no solo la capacidad de construir sistemas funcionales, sino de razonar sobre sus limitaciones, documentar sus decisiones y diseñar una hoja de ruta que permita su evolución responsable.

Bibliografía

SHAP. *SHAP Documentation*.

<https://shap.readthedocs.io/en/latest/>

Esri. *¿Cómo funciona XGBoost?*.

<https://pro.arcgis.com/es/pro-app/3.4/tool-reference/geoai/how-xgboost-works.htm>

QuestionPro. *Coeficiente de correlación de Spearman*.

<https://www.questionpro.com/blog/es/coeficiente-de-correlacion-de-spearman/>

Minitab. *¿Cómo interpretar el R cuadrado?*.

<https://blog.minitab.com/es/blog/analisis-de-regresion-como-puedo-interpretar-el-r-cuadrado-y-evaluar-la-bondad-de-ajuste>

IBM. *¿Qué es Random Forest?*.

<https://www.ibm.com/es-es/think/topics/random-forest>

scikit-learn developers. *scikit-learn: Machine Learning in Python*.

<https://scikit-learn.org/stable/>

Lundberg, S. M., & Lee, S.-I. *A Unified Approach to Interpreting Model Predictions*. arXiv preprint arXiv:1705.07874, 2017.

<https://doi.org/10.48550/arXiv.1705.07874>

Harish S., Abishek Kevin A., Harsha Vardhan U., & Sharon Femi P. *Expected Goals Prediction in Football using XGBoost*. ESP Journal of Engineering & Technology Advancements, 3(1), 21–26, 2023.

<https://doi.org/10.56472/25832646/JETA-V3I1P104>

Spearman, W. *Beyond Expected Goals*. MIT Sloan Sports Analytics Conference, Boston, MA, 2018.

https://www.researchgate.net/publication/327136620_Beyond_Expected_Goals

Anexo I. Glosario de términos

A continuación, se definen los conceptos técnicos, acrónimos y vocabulario especializado utilizados a lo largo de este Trabajo de Fin de Grado, con el objetivo de facilitar la lectura y comprensión del documento.

CAPEX (Capital Expenditure): Gastos de capital en bienes físicos, inversión inicial en infraestructura.

CSRF (Cross-Site Request Forgery): Vulnerabilidad de seguridad web que permite a un atacante inducir a los usuarios a realizar acciones que no pretenden realizar en una aplicación en la que están autenticados.

Data Leakage (Fuga de datos): Error en el aprendizaje automático que ocurre cuando se utiliza información externa al conjunto de entrenamiento para crear el modelo, lo que resulta en métricas de rendimiento irrealmente optimistas que no se generalizan a datos nuevos.

ETL (Extract, Transform, Load): Proceso de integración de datos que consiste en extraer datos de diversas fuentes, transformarlos para cumplir con requisitos de negocio o técnicos, y cargarlos en un sistema de destino.

EWMA (Exponentially Weighted Moving Average): Método estadístico para series temporales que aplica pesos decrecientes a los datos históricos, priorizando la información más reciente.

Feature Engineering (Ingeniería de características): Proceso de transformar datos en bruto en variables que representen mejor el problema subyacente a los algoritmos predictivos, mejorando así la precisión del modelo.

GridSearch: Técnica de optimización de hiperparámetros que realiza una búsqueda exhaustiva a través de un subconjunto especificado manualmente del espacio de parámetros de un algoritmo de aprendizaje automático.

H2H (Head-to-Head): Comparación directa entre dos entidades o modelos bajo las mismas condiciones.

JWT (JSON Web Token): Estándar abierto basado en JSON para la creación de tokens de acceso que permiten la propagación de identidad y privilegios de forma segura entre un cliente y un servidor.

Kanban: Metodología ágil para la gestión del flujo de trabajo que se centra en la visualización del trabajo, la limitación del trabajo en curso (WIP) y la mejora continua

del proceso de entrega.

MVC (Model-View-Controller): Patrón de arquitectura de software que separa la lógica de negocio (Modelo), la interfaz de usuario (Vista) y la gestión de eventos (Controlador) para facilitar el mantenimiento y la escalabilidad.

OPEX (Operational Expenditure): Gastos operativos recurrentes, asociado comúnmente al modelo de pago por uso de servicios, mantenimiento o salarios.

Outliers (Valores atípicos): Observaciones en un conjunto de datos que se encuentran a una distancia anormal de otros valores y que pueden indicar variabilidad en la medición, errores experimentales o novedades de interés.

Prop Drilling: Fenómeno en frameworks de desarrollo frontend donde los datos se pasan a través de múltiples niveles de componentes que no los necesitan, solo para llegar a un componente anidado.

Proxy: Servidor o programa que actúa como intermediario entre las peticiones de recursos de un cliente y el servidor de destino, utilizado a menudo para seguridad, filtrado o mejora del rendimiento.

Random Forest: Algoritmo de aprendizaje supervisado compuesto por una multitud de árboles de decisión, cuya salida es la clase que representa la moda de las clases o la media de las predicciones de los árboles individuales.

Scraping: Técnica automatizada para extraer grandes cantidades de información de sitios web de forma estructurada para su posterior análisis o almacenamiento.

SHAP (SHapley Additive exPlanations): Método basado en la teoría de juegos para explicar el resultado de modelos de aprendizaje automático, asignando a cada característica un valor de importancia para una predicción específica.

XAI (Explainable Artificial Intelligence): Conjunto de procesos y métodos que permiten a los usuarios humanos comprender y confiar en los resultados y predicciones creados por algoritmos de aprendizaje automático.

XGBoost (eXtreme Gradient Boosting): Implementación optimizada de algoritmos de aumento de gradiente diseñada para ser altamente eficiente, flexible y portátil, siendo uno de los algoritmos más utilizados en ciencia de datos.

Anexo II. Enlaces externos

Enlaces utilizados durante el análisis comparativo de repositorios:

- github.com/carlosgeos/laligafantasy
- github.com/diegoparrilla/marca-fantasy-scraper
- github.com/VictorBusque/laliga-fantasy-bot
- github.com/alxgarci/marca-fantasy-api-scraper-updated
- github.com/Ayush-Mgr/Football-Prediction
- github.com/eharshit/LaLiga-Prediction-Analytics

Anexo III. Registro de horas

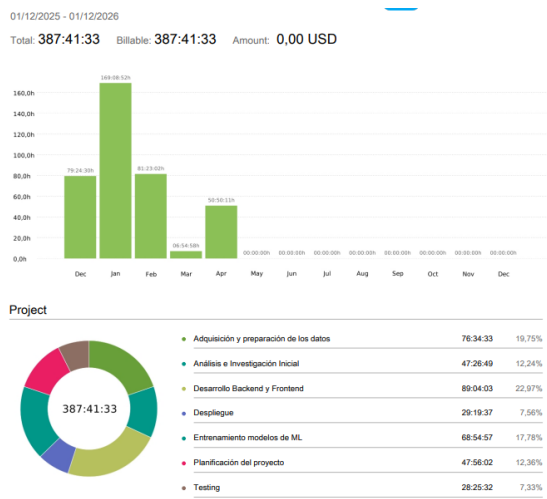
Este anexo contiene el detalle de todas las entradas registradas en Clockify durante el desarrollo del proyecto.

Descripción

Clockify es la herramienta de monitorización de tiempo utilizada para registrar todas las horas invertidas en el proyecto. Los registros incluyen:

- **Fecha:** Día de realización de la actividad
- **Duración:** Horas invertidas en la tarea
- **Descripción:** Detalle específico de la actividad realizada
- **Proyecto/Categoría:** Paquete de trabajo de la EDT asociado
- **Tags:** Etiquetas para clasificación (metodología, fase del proyecto)

Exportación de datos



Los registros completos pueden consultarse directamente en la plataforma Clockify en la siguiente URL:

<https://app.clockify.me/shared/69ef5bade8cb7f630a26ebc3>

Anexo IV. Manuales de instalación y usuario

Este anexo tiene como objetivo proporcionar las instrucciones necesarias para la puesta en marcha de la aplicación en diferentes entornos, así como una guía básica para el usuario final.

1. Manual de instalación

1.1 Objetivo

Este manual detalla cómo inicializar y ejecutar LigaMaster en tres configuraciones distintas:

1. **Desarrollo local:** backend ejecutándose en la máquina local, con servicios de apoyo (PostgreSQL, Meilisearch, Redis) en contenedores Docker.
2. **Docker completo:** toda la pila de aplicación containerizada y orquestada mediante Docker Compose.
3. **Acceso remoto:** conexión directa a la versión desplegada en Vercel, sin instalación local.

1.2 Requisitos previos

Antes de proceder con cualquiera de los escenarios, verifica que tu sistema dispone de:

- **Python 3.11:** verifica ejecutando `python --version`. Debe mostrar `Python 3.11.x`.
- **Node.js 18+:** verifica ejecutando `node --version`. Debe mostrar `v18.x.x` o superior.
- **Docker Desktop:** requerido solo para los escenarios que utilizan contenedores. Descargable desde <https://www.docker.com/products/docker-desktop>.
- **Git:** sistema de control de versiones para clonar el repositorio.

Además, el proyecto utiliza un entorno virtual Python denominado `.venv311`.

1.3 Escenario A: Desarrollo local

Este escenario es recomendado para desarrollo activo del código. El backend se ejecuta directamente en la máquina del desarrollador, mientras que los servicios de apoyo (especialmente la base de datos PostgreSQL, además de Meilisearch y Redis) se ejecutan en contenedores Docker para mantener el entorno aislado.

Nota importante: si ya tienes un PostgreSQL local usando el puerto 5432, la base de datos en Docker se publica en el puerto 5433 del host para evitar conflictos.

1.3.1 Configuración inicial (ejecutar una sola vez)

1. Clonar el repositorio:

```
git clone https://github.com/pabloarrabalh/TFG.git
cd TFG
```

2. Iniciar Docker Desktop: Abre la aplicación desde el menú inicio y espera a que muestre el estado “running” (indicador verde). Verifica la disponibilidad ejecutando:

```
docker ps
```

3. Levantar servicios de apoyo en Docker: Levanta PostgreSQL, Meilisearch y Redis en contenedores Docker; la aplicación Django conectará a PostgreSQL ejecutándose en Docker en ‘localhost:5433’.

```
docker compose -f docker-compose.local.yml up -d db meilisearch redis
```

Espera aproximadamente 30 segundos a que los servicios alcancen estado “healthy”. Verifica con:

```
docker compose -f docker-compose.local.yml ps
```

Todos los servicios deben mostrar estado “Up (healthy)”. Si necesitas comprobar la codificación del servidor Postgres desde el contenedor.

4. Crear entorno virtual Python:

```
python -m venv .venv311
& .\.venv311\Scripts\Activate.ps1
```

Verifica que el indicador de la terminal muestra (.venv311).

5. Instalar dependencias Python:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Este proceso requiere aproximadamente 2-3 minutos.

6. Configurar variables de entorno: Copia el archivo de configuración local:

```
copy .env.local .env
```

Verifica que el archivo `.env` contiene las siguientes variables con los valores correctos para desarrollo local usando servicios en Docker (el puerto 5433 de Postgres se expondrá en el host):

```
DB_HOST=localhost
DB_PORT=5433
MEILISEARCH_HOST=http://localhost:7700
REDIS_URL=redis://localhost:6379/1
```

7. Aplicar migraciones de base de datos:

```
python manage.py makemigrations
python manage.py migrate
```

8. (Recomendado) Generar o regenerar predicciones manualmente: si quieres generarlas en local, ejecuta:

```
python manage.py generar_predicciones --temporada 25_26
--all-jornadas --sin-filtro-minutos --workers 4 --force
```

Este proceso es largo y se necesita tanto en local como en Docker, se recomienda consultar la versión desplegada para evitar esperas.

9. (Opcional) Crear superusuario para acceso administrativo:

```
python manage.py createsuperuser
```

Comprobaciones:

1. PostgreSQL:

```
python manage.py shell -c "from django.db import connection;
connection.ensure_connection();
print('DB OK:',
connection.settings_dict['HOST'], connection.settings_dict['PORT'])"
```

2. Meilisearch: `curl http://localhost:7700/health` Seleccionar opción S/Y

1.3.2 Ejecución en cada sesión de desarrollo Una vez completada la configuración inicial, para iniciar el sistema en sesiones posteriores:

Terminal 1 - Backend:

```
cd TFG
& .\venv311\Scripts\Activate.ps1
python manage.py runserver
```

Este comando inicia el servidor Django en `http://127.0.0.1:8000/`.

Terminal 2 - Frontend:

```
cd TFG/frontend-web
npm install # solo en la primera ejecución
npm run dev
```

Este comando inicia el servidor de desarrollo Vite en `http://localhost:5173/`.

La aplicación estará completamente operativa con ambos servidores en ejecución.

Servicio	Puerto	URL
Backend Django	8000	<code>http://127.0.0.1:8000/</code>
Frontend Vite	5173	<code>http://localhost:5173/</code>
Meilisearch	7700	<code>http://localhost:7700/</code>
Redis	6379	<code>localhost:6379</code>
PostgreSQL	5432	<code>localhost:5432</code>

1.3.3 Puertos y URLs en desarrollo local

1.4 Escenario B: Docker completo

Este escenario encapsula toda la aplicación en contenedores, replicando exactamente el entorno de producción. Se recomienda para verificaciones de integración y despliegues.

1.4.1 Configuración e inicialización

1. Clonar el repositorio:

```
git clone https://github.com/pabloarrabalh/TFG.git
cd TFG
```

2. **Iniciar Docker Desktop:** Abre la aplicación y verifica que se encuentra en estado “running”.
3. **Configurar variables de entorno para Docker:** Copia la configuración específica para Docker:

```
copy .env.docker .env
```

Verifica que el archivo `.env` contiene:

```
DB_HOST=db  
MEILISEARCH_HOST=http://meilisearch:7700  
REDIS_URL=redis://redis:6379/1
```

Nota crítica: En este escenario, los hosts son nombres de servicios Docker (definidos en `docker-compose.local.yml`), no `localhost`.

4. **Construir e iniciar contenedores (primera ejecución):**

```
docker compose -f docker-compose.local.yml up --build
```

Este proceso incluye la compilación de imágenes Docker, la creación de volúmenes, y la inicialización de servicios. Puede requerir 5-10 minutos en la primera ejecución. La población inicial de la base de datos (carga de datos CSV, cálculo de predicciones ML, indexación en Meilisearch) puede requerir 20-30 minutos adicionales.

Nota: en Docker no hace falta lanzar un comando extra para generar predicciones si la BD está vacía; el backend las genera automáticamente en el arranque inicial mediante `docker-entryptoint.sh`.

5. **Ejecutar en sesiones posteriores:**

```
docker compose -f docker-compose.local.yml up
```

Los contenedores conservarán el estado de volúmenes persistentes (base de datos, índices, etc.).

1.4.2 Acceso a la aplicación en Docker Una vez que los servicios se encuentren en ejecución y los logs muestren “Daphne running”, accede a:

- **Frontend + Backend:** `http://localhost/`
- **Admin Django:** `http://localhost/admin/`

- **API REST:** `http://localhost/api/`

El proxy Nginx en puerto 80 redirige las solicitudes hacia el backend Django en puerto 8000 (interno, no expuesto directamente).

1.4.3 Gestión de contenedores Detener servicios sin eliminar datos:

```
docker compose -f docker-compose.local.yml stop
```

Detener y eliminar contenedores (preservando volúmenes):

```
docker compose -f docker-compose.local.yml down
```

Ver logs en tiempo real:

```
docker compose -f docker-compose.local.yml logs -f backend
```

1.5 Escenario C: Versión desplegada

Para usuarios que no desean realizar instalación local, la aplicación está disponible en una instancia de producción:

`https://tfg-delta-three.vercel.app`

Esta versión se encuentra completamente funcional y requiere únicamente un navegador web. No es necesario instalar dependencias, ejecutar migraciones, ni gestionar contenedores locales.

1.6 Resolución de problemas

- **“Cannot connect to Docker daemon”:** Docker Desktop no está en ejecución. Abre la aplicación desde el menú inicio y espera a que alcance estado “running”.
- **“Port XXX already in use”:** El puerto está en uso por otro proceso. Para desarrollo local, inicia el backend en otro puerto o termina el otro proceso que esté corriendo:

```
python manage.py runserver 8001
```

- **“Connection refused” en PostgreSQL:** El servicio Docker no está operativo o el puerto expuesto es incorrecto. Verifica:

```
docker compose -f docker-compose.local.yml ps
```

Si el estado no es “healthy”, espera 30 segundos y repite. Si persiste, reinicia los servicios y comprueba que Django apunta a ‘localhost:5433’:

```
docker compose -f docker-compose.local.yml down
docker compose -f docker-compose.local.yml up -d
```

- **“ModuleNotFoundError” en Python:** Las dependencias no están instaladas. Verifica que el entorno virtual está activado y ejecuta:

```
pip install -r requirements.txt
```

- **Meilisearch tardanza en indexación:** En la primera ejecución, la indexación de todos los jugadores puede requerir 1-3 minutos. Este comportamiento es normal. No interrumpas el proceso.
- **“django.core.exceptions.ImproperlyConfigured”:** Las variables de entorno son incorrectas o inexistentes. Verifica que el archivo `.env` existe y contiene todas las variables requeridas. Revisa los valores según el escenario (desarrollo local vs. Docker).

2. Manual de Usuario

2.1 Objetivo

Este manual explica cómo usar LigaMaster desde el punto de vista del usuario final.

2.2 Formas de acceso

Puedes usar la aplicación de tres formas:

- en local, si la has arrancado en tu equipo;
- en Docker, si estás ejecutando el proyecto con contenedores;
- directamente en la versión desplegada en <https://tfg-delta-three.vercel.app>.

2.3 Acceso inicial

Paso 1: Abrir la aplicación Entra en la URL correspondiente a tu entorno:

- local: `http://localhost:5173/` o `http://localhost:80/`;
- desplegada: `https://tfg-delta-three.vercel.app`.

Paso 2: Registrarse o iniciar sesión Si es tu primer acceso:

1. entra en la pantalla de registro;
2. completa el formulario con usuario, email y contraseña;
3. elige si deseas hacer un tour guiado por la aplicación.

Si ya tienes cuenta:

1. abre el formulario de login;
2. introduce usuario y contraseña;
3. accede al panel principal.

2.4 Navegación principal

Una vez dentro, la aplicación te permite consultar varias áreas:

- **Menú principal:** resume la jornada actual, partidos de los equipos favoritos y todos los correspondientes a la próxima jornada. Así como una sección de jugadores destacados de acuerdo a los modelos predictivos en la próxima jornada. Pulsando en ellos se accede a la explicación de la predicción y un botón para navegar a la vista del propio jugador.
- **Clasificación:** muestra la tabla de la liga, con filtros para elegir la jornada deseada.
- **Equipos:** lista los equipos disponibles y permite abrir su detalle.
- **Jugadores:** muestra el detalle del jugador, su radar y estadísticas asociadas.
- **Mi Plantilla:** vista en la cual el usuario puede gestionar su alineación mediante un sistema de drag and drop con cartas para cada jugador.
- **Búsqueda:** localiza jugadores y equipos por nombre.
- **Favoritos:** guarda tus equipos preferidos.

- **Estadísticas:** consulta y compara el rendimiento de jugadores.
- **Perfil:** actualiza tus datos personales, foto de perfil y preferencias.

2.5 Uso paso a paso

2.5.1 Consultar la clasificación

1. Abre la sección de clasificación.
2. Selecciona la temporada o jornada que desee consultar.
3. Revisa la tabla y el rendimiento de cada equipo.

2.5.2 Ver el detalle de un equipo

1. Entra en la sección de equipos.
2. Pulsa sobre el nombre del equipo.
3. Consulta su información, jugadores y contexto de temporada.

2.5.3 Ver el detalle de un jugador

1. Abre la sección de jugador (desde la vista de su equipo o el modal de jugadores destacados) o busca al jugador por nombre.
2. Entra en su ficha.
3. Revisa el radar, partidos recientes y estadísticas.

2.5.4 Buscar equipos o jugadores

1. Usa el buscador de la aplicación.
2. Escribe parte del nombre de un equipo o jugador.
3. Selecciona el resultado que te interese.

2.5.5 Gestionar favoritos Opción A:

1. Entra en la vista de favoritos.
2. Pulsa la opción de favorito.
3. El equipo quedará guardado en tu perfil.

Opción B:

1. Accede al perfil.
2. Elimina aquellos equipos que desees en la sección de favoritos.

Nota: Si es tu primer inicio, accede a favoritos desde el componente de equipos favoritos.

2.5.6 Usar el consejero

1. Abre la sección de consejero.
2. Selecciona el jugador y la acción que quieres analizar.
3. Revisa la recomendación generada.

2.5.7 Revisar notificaciones

1. Entra en notificaciones.
2. Marca como leídas las que ya hayas visto.
3. Elimina las que no necesites conservar.

2.5.8 Editar perfil

1. Abre tu perfil.
2. Cambia los datos permitidos.
3. Guarda los cambios.

2.6 Recomendaciones de uso

- Usa siempre la misma cuenta para conservar favoritos y notificaciones.
- Si trabajas en local, asegúrate de que Meiliseach está arrancado.
- Si la app no responde, refresca la sesión y vuelve a iniciar sesión.
- En la versión desplegada, espera unos segundos si una búsqueda tarda más de lo normal.

2.7 Resolución de problemas básica

- **No aparecen resultados de búsqueda:** revisa que el buscador y Meilisearch estén activos.
- **No aparecen resultados, equipos ni jugadores:** dado que la BBDD está en Supabase, cada semana, si no tiene tráfico se suspende. Se ha intentado evitar esto mediante GitHub Actions y otras herramientas, pero ha resultado imposible. Se intenta mantener activa manualmente, si en algún caso no estuviese disponible, contactar con el autor del proyecto.

2.8 Análisis de calidad con SonarQube

Este apartado describe cómo ejecutar el análisis estático de calidad del proyecto utilizando SonarQube y el scanner CLI.

Requisitos previos

- **Docker** instalado y en ejecución
- **Servidor SonarQube** en ejecución en `http://localhost:9000`
- **Token de SonarQube** válido con permisos de análisis
- **coverage.xml** generado mediante Coverage.py

1. Generar reporte de cobertura Antes de ejecutar el análisis, es necesario generar el reporte de cobertura:

```
.venv311\Scripts\Activate.ps1
coverage run --source=main,config manage.py test
coverage xml
```

Esto generará el archivo `coverage.xml` en la raíz del proyecto.

2. Ejecutar análisis de SonarQube En una PowerShell

```
docker --% run --rm --network host `
-e SONAR_HOST_URL=http://host.docker.internal:9000 `
-e SONAR_TOKEN=<tu_token_aqui> `
-v c:\Users\<usuario>\Desktop\TFG:/usr/src `
sonarsource/sonar-scanner-cli:latest `
```

```
-Dsonar.sources=main,config `
-Dsonar.tests=main/tests `
-Dsonar.python.coverage.reportPaths=coverage.xml `
-Dsonar.python.xunit.reportPath=xunit-result.xml
```

Nota: El flag - % es obligatorio en PowerShell para evitar problemas de parsing.

Parámetro	Valor	Descripción
SONAR_HOST_URL	http://host.docker.internal:9000	URL del servidor SonarQube
SONAR_TOKEN	Token generado	Token de autenticación
-v	Ruta local:/usr/src	Vinculación de volumen
sonar.sources	main,config	Código fuente
sonar.tests	main/tests	Tests
sonar.python.coverage.reportPaths	coverage.xml	Reporte de cobertura
sonar.python.xunit.reportPath	xunit-result.xml	Reporte de tests

3. Parámetros de configuración

4. Gestión de tokens Crear token:

1. Acceder a <http://localhost:9000> con las credenciales por defecto (admin/admin)
2. Ir a **My Account** → **Security**
3. Generar nuevo token
4. Guardarlo (solo visible una vez)

Uso del token:

```
-e SONAR_TOKEN=squ_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

5. Validación de ejecución

- **Éxito:** EXECUTION SUCCESS
- **Error:** mensajes ERROR

Ejemplo esperado:

```
INFO  Sensor Cobertura Sensor for Python coverage
DEBUG Using pattern 'coverage.xml'
```

6. Revisión de resultados Accede a `http://localhost:9000` y revisa:

- Quality Gate
- Coverage
- Security Hotspots
- Code Smells
- Bugs

2.9 Pruebas de carga con Locust

Este apartado describe cómo ejecutar pruebas de carga distribuidas contra la API de LigaMaster utilizando Locust con Docker. Locust permite simular múltiples usuarios concurrentes contra la API. La configuración utiliza una arquitectura master-worker distribuida en Docker:

- 1 servidor master (interfaz web en puerto 8089)
- 3 workers conectados automáticamente al master

1. Construcción de la imagen Ejecuta el siguiente comando para construir la imagen Docker de Locust sin utilizar caché:

```
docker compose -f docker-compose.locust.yml build --no-cache
```

2. Levantar el cluster (master + 3 workers) Inicia el cluster distribuido:

```
docker compose -f docker-compose.locust.yml up -d
```

Este comando inicia:

- 1 servidor master (puerto 8089 para la interfaz web)
- 3 trabajadores (workers)

3. Acceder a la interfaz web Abre tu navegador y accede a:

`http://localhost:8089`

En la interfaz ingresa los parámetros de la prueba:

- **Number of users:** cantidad de usuarios virtuales.
- **Spawn rate:** usuarios por segundo a generar.

Una vez ingresados los parámetros, haz clic en **Start swarming** para iniciar la prueba de carga.

4. Parar el cluster Para detener y eliminar los contenedores:

```
docker compose -f docker-compose.locust.yml down
```

Notas

- **Host en la prueba:** `http://backend:8000`
- **Usuario de prueba:** las pruebas utilizan credenciales `testlocust / TestLocust123!` para autenticación automática en el bootstrap inicial. (es posible que no estén en la BBDD de producción)

Anexo V. Autodeclaración del uso de IA

Se declara que durante el desarrollo de este proyecto se han utilizado herramientas basadas en Inteligencia Artificial generativa como apoyo técnico, operativo y de consulta. El uso de dichas tecnologías se ha realizado bajo la constante supervisión, revisión y criterio del autor, garantizando en todo momento la integridad académica del trabajo.

Concretamente, la asistencia de la IA se ha aplicado en las siguientes tareas:

- **Prototipado y estructuración inicial:** Generación de la estructura base del proyecto y desarrollo de un frontend provisional, lo que permitió agilizar las fases iniciales de diseño y visualización de la interfaz.
- **Soporte en codificación y refactorización:** Asistencia técnica en el proceso de migración de módulos del frontend hacia JavaScript, así como apoyo en la resolución de bloqueos lógicos, optimización de sintaxis y tareas de debugging.
- **Generación de pruebas:** Apoyo en la redacción y estructuración de casos de prueba unitarios para diversas funciones, contribuyendo a asegurar la robustez y calidad del software desarrollado.
- **Documentación técnica:** Asistencia en la generación de comentarios estructurados y documentación de métodos y funciones, mejorando la legibilidad del código fuente.
- **Consulta y síntesis de información:** Uso de la herramienta como asistente de consulta técnica para consultar rápidamente conceptos específicos y resolver dudas de implementación, actuando como complemento y síntesis de la documentación oficial de las bibliotecas utilizadas.
- **Maquetación y conversión a LaTeX:** Transformación de documentos de Google Docs al lenguaje de marcado $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, incluyendo la automatización del formato de tablas y la organización del glosario de términos.
- **Revisión de estilo:** Apoyo en la corrección gramatical y el pulido tipográfico del documento final para asegurar un tono académico coherente.

Es importante destacar que el diseño de la arquitectura del software, la toma de decisiones tecnológicas, la implementación de la lógica de negocio central, así como la interpretación de los resultados, son fruto exclusivo del trabajo del autor. La inteligencia artificial ha actuado únicamente como una herramienta de productividad y soporte

técnico, manteniendo intacta la autoría y la responsabilidad sobre el contenido total de este Trabajo de Fin de Grado.